

Duboko učenje 1 – Neslužbena rješenja ispitnih zadataka

by: Antonio Macan

10. lipnja 2026.

Zadaci riješeni u ovoj skripti preuzeti su s javno dostupnih [neslužbenih stranica predmeta](#) ili su rekonstruirani prema dobrom pamćenju kolega :)

Sadržaj

1	MI 2016./2017.	3
	Zadatak 1	3
	Zadatak 2	6
	Zadatak 3	8
	Zadatak 4	10
	Zadatak 5	11
	Zadatak 6	12
2	MI 2017./2018.	14
	Zadatak 1	14
	Zadatak 2	15
	Zadatak 3	17
	Zadatak 4	19
	Zadatak 5	22
	Zadatak 6	23
3	MI 2018./2019.	26
	Zadatak 1	26
	Zadatak 2	27
	Zadatak 3	29
	Zadatak 4	31
	Zadatak 5	32
4	MI 2021./2022.	36
	Zadatak 1	36
	Zadatak 2	37
	Zadatak 3	38
	Zadatak 4	43
	Zadatak 5	47
5	MI 2022./2023.	50
	Zadatak 1	50
	Zadatak 2	53
	Zadatak 3	59

Zadatak 4	61
Zadatak 5	62
6 MI 2024./2025.	65
Zadatak 1	65
Zadatak 2	65
Zadatak 3	67
Zadatak 4	67
Zadatak 5	69

1 MI 2016./2017.

Zadatak 1

Razmatramo klasifikacijsku konvolucijsku mrežu s arhitekturom:

- konvolucijski sloj bez nadopunjavanja: dvije jezgre 3×3 , korak 1, aktivacija ReLU;
- sažimanje maksimumom 2×2 ;
- pretvaranje u vektor;
- potpuno povezani sloj dimenzije 2 te aktivacijom softmaxs.

Parametri mreže zadani su matricama (nema pomaka!):

$$k_1 = \begin{bmatrix} 0 & 0 & 0 \\ -1 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad k_2 = \begin{bmatrix} 0 & -1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad W = \begin{bmatrix} 0 & 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 & 1 \end{bmatrix} \cdot \ln(2).$$

Zadaci:

- Odredite veličinu ulazne slike.
- Odredite izlaze mreže ako je na ulazu zadana slika čiji su svi pikseli jednaki nuli osim piksela $(2, 3)$, $(3, 2)$ i $(3, 3)$ koji su postavljeni na jedan (indeksi redaka i stupaca počinju od nule).

Rješenje (a):

Za početak, zapišimo tablično sve slojeve i dimenzije njihovih izlaznih tenzora u posve općenitoj formi, a potom ćemo na temelju podataka zadanih u tekstu zadatka vidjeti koje su konkretne vrijednosti poznate.

Sloj	Dimenzija izlaza
Ulaz	$C_{\text{ulaz}} \times H_{\text{ulaz}} \times W_{\text{ulaz}}$
Conv($C_{\text{in}} = C_{\text{ulaz}}$, $C_{\text{out}} = C_{\text{conv}}$, $k = k_{\text{conv}}$, $s = s_{\text{conv}}$, $p = p_{\text{conv}}$)	$C_{\text{conv}} \times H_{\text{conv}} \times W_{\text{conv}}$
ReLU	$C_{\text{conv}} \times H_{\text{conv}} \times W_{\text{conv}}$
MaxPool($k = k_{\text{pool}}$, $s = s_{\text{pool}}$, $p = p_{\text{pool}}$)	$C_{\text{conv}} \times H_{\text{pool}} \times W_{\text{pool}}$
Flatten	$C_{\text{conv}} \cdot H_{\text{pool}} \cdot W_{\text{pool}}$
FC	D_{FC}
Softmax	D_{FC}

Pri pisanju ove tablice, treba se znati sljedeće:

- broj ulaznih kanala konvolucijskog sloja jednak je broju izlaznih kanala prethodnog sloja
- ReLU ne mijenja dimenzije ulaza
- MaxPool ne mijenja semantičku dimenziju

- Softmax je vektorska funkcija vektorske varijable pri čemu ulazni i izlazni vektori imaju istu dimenzionalnost

Iz teksta zadatka mogu se iščitati sljedeće vrijednosti dimenzionalnosti:

- $C_{\text{ulaz}} = 1$ jer su zadane konvolucijske jezgre jednokanalne
- $C_{\text{conv}} = 2$ jer su zadane dvije konvolucijske jezgre za konvolucijski sloj (k_1, k_2)
- $C_{\text{conv}} \cdot H_{\text{pool}} \cdot W_{\text{pool}} = 8$ jer je ulaz FC sloja dimenzije 8 ($W \in \mathbb{R}^{2 \times 8}$)
- $D_{\text{FC}} = 2$ jer je izlaz FC sloja dimenzije 2 ($W \in \mathbb{R}^{2 \times 8}$)

Iz teksta zadatka mogu se iščitati sljedeće vrijednosti parametara slojeva:

- $k_{\text{conv}} = 3$
- $s_{\text{conv}} = 1$
- $p_{\text{conv}} = 0$
- $k_{\text{pool}} = 2$
- $s_{\text{pool}} = 2$ (ako nije eksplicitno navedeno, pretpostavljamo da je $s_{\text{pool}} = k_{\text{pool}}$)
- $p_{\text{pool}} = 0$

Pretpostavimo da je prostorna dimenzija ulazne slike kvadratna te uzmimo u obzir da radimo s konvolucijskim slojevima i slojevima sažimanja za koje su parametri k, s, p isti za obje prostorne dimenzije. Tada vrijedi da će izlazni tenzor za svaki od slojeva u tom dijelu mreže imati kvadratnu prostornu dimenzionalnost. Dakle, vrijedi:

$$H_{\text{ulaz}} = W_{\text{ulaz}} = D_{\text{ulaz}}, \quad H_{\text{conv}} = W_{\text{conv}} = D_{\text{conv}}, \quad H_{\text{pool}} = W_{\text{pool}} = D_{\text{pool}}.$$

Napišimo prethodnu tablicu s novim oznakama i uvrštenim poznatim vrijednostima:

Sloj	Dimenzija izlaza
Ulaz	$1 \times D_{\text{ulaz}} \times D_{\text{ulaz}}$
Conv($C_{\text{in}} = 1, C_{\text{out}} = 2, k = 3, s = 1, p = 0$)	$2 \times D_{\text{conv}} \times D_{\text{conv}}$
ReLU	$2 \times D_{\text{conv}} \times D_{\text{conv}}$
MaxPool($k = 2, s = 2, p = 0$)	$2 \times D_{\text{pool}} \times D_{\text{pool}}$
Flatten	$2 \cdot D_{\text{pool}}^2$
FC	2
Softmax	2

Lako možemo izračunati sljedeće:

$$2 \cdot D_{\text{pool}}^2 = 8 \iff D_{\text{pool}}^2 = 4 \Rightarrow D_{\text{pool}} = 2.$$

Kako bismo mogli izračunati prostorne dimenzije kroz konvolucijske slojeve i slojeve sažimanja, moramo znati formulu koja nam opisuje kako te operacije mijenjaju prostornu dimenzionalnost svog ulaza. Neka je D_{in} prostorna dimenzija (npr. W_{in} ili H_{in}) ulaza u konvolucijski sloj ili sloj sažimanja te neka je D_{out} prostorna dimenzija izlaza tog sloja. Sloj ima parametre k, s, p koji su redom veličina jezgre, korak (stride), padding. Tada vrijedi:

$$D_{\text{out}} = \left\lfloor \frac{D_{\text{in}} + 2p - k}{s} \right\rfloor + 1. \quad (1)$$

Prema formuli (1) računamo:

$$\begin{aligned} D_{\text{pool}} &= \left\lfloor \frac{D_{\text{conv}} + 2p_{\text{pool}} - k_{\text{pool}}}{s_{\text{pool}}} \right\rfloor + 1 \\ 2 &= \left\lfloor \frac{D_{\text{conv}} + 2 \cdot 0 - 2}{2} \right\rfloor + 1 \\ 1 &= \left\lfloor \frac{D_{\text{conv}} - 2}{2} \right\rfloor. \end{aligned}$$

Vrijedi:

$$\begin{aligned} 1 = \left\lfloor \frac{D_{\text{conv}} - 2}{2} \right\rfloor &\iff 1 \leq \frac{D_{\text{conv}} - 2}{2} < 2 \\ &\iff 2 \leq D_{\text{conv}} - 2 < 4 \\ &\iff 4 \leq D_{\text{conv}} < 6. \end{aligned}$$

Dakle, imamo dva slučaja: (1) $D_{\text{conv}} = 4$ ili (2) $D_{\text{conv}} = 5$.
Za slučaj (1) $D_{\text{conv}} = 4$ prema formuli 1 računamo:

$$\begin{aligned} D_{\text{conv}} &= \left\lfloor \frac{D_{\text{ulaz}} + 2p_{\text{conv}} - k_{\text{conv}}}{s_{\text{conv}}} \right\rfloor + 1 \\ 4 &= \left\lfloor \frac{D_{\text{ulaz}} + 2 \cdot 0 - 3}{1} \right\rfloor + 1 \\ 3 &= \lfloor D_{\text{ulaz}} - 3 \rfloor. \end{aligned}$$

Budući da je D_{ulaz} prirodan broj, vrijedi:

$$3 = \lfloor D_{\text{ulaz}} - 3 \rfloor \iff 3 = D_{\text{ulaz}} - 3 \iff D_{\text{ulaz}} = 6.$$

Za slučaj (2) $D_{\text{conv}} = 5$ prema formuli 1 računamo:

$$\begin{aligned} D_{\text{conv}} &= \left\lfloor \frac{D_{\text{ulaz}} + 2p_{\text{conv}} - k_{\text{conv}}}{s_{\text{conv}}} \right\rfloor + 1 \\ 5 &= \left\lfloor \frac{D_{\text{ulaz}} + 2 \cdot 0 - 3}{1} \right\rfloor + 1 \\ 4 &= \lfloor D_{\text{ulaz}} - 3 \rfloor. \end{aligned}$$

Budući da je D_{ulaz} prirodan broj, vrijedi:

$$4 = \lfloor D_{\text{ulaz}} - 3 \rfloor \iff 4 = D_{\text{ulaz}} - 3 \iff D_{\text{ulaz}} = 7.$$

Konačno rješenje (a): $\text{Ulaz} \in \mathbb{R}^{1 \times 6 \times 6}$ ili $\text{Ulaz} \in \mathbb{R}^{1 \times 7 \times 7}$.

Rješenje (b):

Budući da smo u podzadaku (a) dobili 2 moguća rješenja, za ovaj podzadatak ćemo između njih proizvoljno odabrati da je ulazna slika $\mathbf{x} \in \mathbb{R}^{1 \times 6 \times 6}$.

$$\mathbf{x} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

Neka je $\mathbf{s}$ izlaz iz konvolucijskog sloja te neka je $\mathbf{s}^{(1)}$ prvi kanal izlaza, a $\mathbf{s}^{(2)}$ drugi kanal izlaza.

$$\mathbf{s}^{(1)} = k_1 \star \mathbf{x} = \begin{bmatrix} 0 & 0 & 0 \\ -1 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} \star \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & -1 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

$$\mathbf{s}^{(2)} = k_2 \star \mathbf{x} = \begin{bmatrix} 0 & -1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} \star \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & -1 & -1 & 0 \end{bmatrix}.$$

Neka je $\mathbf{h}$ izlaz iz ReLU sloja te neka je $\mathbf{h}^{(1)}$ prvi kanal izlaza, a $\mathbf{h}^{(2)}$ drugi kanal izlaza.

$$\mathbf{h}^{(1)} = \text{ReLU}(\mathbf{s}^{(1)}) = \text{ReLU} \left(\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & -1 \\ 0 & 1 & 0 & -1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \right) = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

$$\mathbf{h}^{(2)} = \text{ReLU}(\mathbf{s}^{(2)}) = \text{ReLU} \left(\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & -1 & -1 & 0 \end{bmatrix} \right) = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

Neka je $\mathbf{f}$ izlaz iz MaxPool sloja te neka je $\mathbf{f}^{(1)}$ prvi kanal izlaza, a $\mathbf{f}^{(2)}$ drugi kanal izlaza.

$$\mathbf{f}^{(1)} = \mathbf{f}^{(2)} = \text{MaxPool} \left(\begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \right) = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}.$$

Neka je g vektor dobiven vektorizacijom tenzora $\mathbf{f}$.

$$g = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 \end{bmatrix}^\top.$$

Neka je z izlaz iz potpuno povezanog sloja.

$$z = Wg = \ln(2) \cdot \begin{bmatrix} 0 & 1 & 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 \end{bmatrix}^\top = \ln(2) \cdot \begin{bmatrix} 3 \\ 1 \end{bmatrix} = \begin{bmatrix} \ln(8) \\ \ln(2) \end{bmatrix}.$$

Neka je p izlaz softmaxa.

$$p = \text{softmax} \left(\begin{bmatrix} \ln(8) & \ln(2) \end{bmatrix}^\top \right) = \begin{bmatrix} \frac{\exp(\ln(8))}{\exp(\ln(8)) + \exp(\ln(2))} \\ \frac{\exp(\ln(2))}{\exp(\ln(8)) + \exp(\ln(2))} \end{bmatrix} = \begin{bmatrix} \frac{4}{5} \\ \frac{1}{5} \end{bmatrix}.$$

Zadatak 2

Razmatramo klasifikacijsku mrežu s arhitekturom:

- potpuno povezani sloj: dimenzija 2, aktivacija ReLU;
- potpuno povezani sloj: dimenzije 2, aktivacija softmax.

Parametri mreže zadani su matricama:

$$W_1 = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \quad b_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad W_2 = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \cdot \ln(2), \quad b_2 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}.$$

Zadaci:

- Napišite jednadžbe modela kojeg implementira mreža.
- Odredite izlaze mreže ako je na ulazu zadan podatak $(5, 2)$.
- Odredite gradijente po retcima matrice W_1 ako gubitak odgovara negativnoj log-izglednosti, a željeni izlaz mreže je $(0, 1)$.

Rješenje (a):

Neka je $x \in \mathbb{R}^2$ ulazni podatak. Neka je s_1 izlaz iz prvog potpuno povezanog sloja. Vrijedi:

$$s_1 = W_1 x + b_1.$$

Neka je h izlaz iz aktivacije ReLU koja slijedi prvi FC. Vrijedi:

$$h = \text{ReLU}(s_1).$$

Neka je s_2 izlaz iz drugog potpuno povezanog sloja. Vrijedi:

$$s_2 = W_2 h + b_2.$$

Neka je p izlaz iz aktivacije softmax koja slijedi drugi FC. Vrijedi:

$$p = \text{softmax}(s_2).$$

Rješenje (b):

Na temelju jednadžbi napisanih u (a) dijelu zadatka, vrijedi:

$$\begin{aligned} x &= \begin{bmatrix} 5 & 2 \end{bmatrix}^\top \\ s_1 &= \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 5 \\ 2 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 5 \\ -2 \end{bmatrix} \\ h &= \text{ReLU} \left(\begin{bmatrix} 5 \\ -2 \end{bmatrix} \right) = \begin{bmatrix} 5 \\ 0 \end{bmatrix} \\ s_2 &= \ln(2) \cdot \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 5 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \ln(2) \cdot \begin{bmatrix} 5 \\ 0 \end{bmatrix} \\ \hat{y} &= \text{softmax} \left(\ln(2) \cdot \begin{bmatrix} 5 & 0 \end{bmatrix}^\top \right) = \text{softmax} \left(\begin{bmatrix} \ln(32) & \ln(1) \end{bmatrix}^\top \right) = \begin{bmatrix} \frac{32}{33} & \frac{1}{33} \end{bmatrix}^\top. \end{aligned}$$

Rješenje (c):

Neka je y stvarna oznaka primjera x , tj. stvarna razdioba vjerojatnosti pripadnosti klasama za primjer x te neka je $\hat{y}$ izlaz modela, tj. prediktirana razdioba vjerojatnosti pripadnosti klasama za primjer x . Gubitak za primjer x se definira kao negativna log-izglednost na sljedeći način:

$$L = - \sum_{j=1}^N y_j \cdot \ln(\hat{y}_j). \quad (2)$$

Ako razmatramo slučaj jednojedininičnih oznaka te ako pretpostavimo da je $j = k$ točna oznaka primjera x , onda se formula (2) pojednostavljuje u:

$$L = - \ln(\hat{y}_k). \quad (3)$$

U našem slučaju, zadano je: $y = \begin{bmatrix} 0 & 1 \end{bmatrix}^\top$.

Za gubitak negativne log-izglednosti, postupak unatražnog prolaza započinjemo računanjem gradijenta gubitka po ulazu u softmaks, dakle računamo:

$$\frac{\partial L}{\partial s_2} = (\hat{y} - y)^\top = \left(\begin{bmatrix} \frac{32}{33} \\ \frac{1}{33} \end{bmatrix} - \begin{bmatrix} 0 \\ 1 \end{bmatrix} \right)^\top = \begin{bmatrix} \frac{32}{33} & -\frac{32}{33} \end{bmatrix}.$$

Nadalje, po pravilu ulančavanja računamo gradijent gubitka po h :

$$\frac{\partial L}{\partial h} = \frac{\partial L}{\partial s_2} \frac{\partial s_2}{\partial h} = \frac{\partial L}{\partial s_2} W_2 = \begin{bmatrix} \frac{32}{33} & -\frac{32}{33} \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \cdot \ln(2) = \ln(2) \cdot \begin{bmatrix} \frac{32}{33} & \frac{32}{33} \end{bmatrix}.$$

Nadalje, po pravilu ulančavanja računamo gradijent gubitka po s_1 :

$$\frac{\partial L}{\partial s_1} = \frac{\partial L}{\partial h} \frac{\partial h}{\partial s_1} = \frac{\partial L}{\partial h} \odot \llbracket s_1 > 0 \rrbracket^\top = \ln(2) \cdot \begin{bmatrix} \frac{32}{33} & \frac{32}{33} \end{bmatrix} \odot \begin{bmatrix} 1 & 0 \end{bmatrix} = \ln(2) \cdot \begin{bmatrix} \frac{32}{33} & 0 \end{bmatrix}.$$

Konačno, računamo gradijent gubitka po W_1 :

$$\frac{\partial L}{\partial W_1} = \left(\frac{\partial L}{\partial s_1} \right)^\top x^\top = \ln(2) \cdot \begin{bmatrix} \frac{32}{33} \\ 0 \end{bmatrix} \begin{bmatrix} 5 & 2 \end{bmatrix} = \ln(2) \cdot \begin{bmatrix} \frac{160}{33} & \frac{64}{33} \\ 0 & 0 \end{bmatrix}.$$

Zadatak 3

Razmatramo klasifikacijsku konvolucijsku mrežu koja je prilagođena obradi sivih slika dimenzija 28×28 . Arhitektura mreže je:

- konvolucijski sloj s nadopunjavanjem (rezolucija ostaje jednaka): 16 jezgara 5×5 korak 1, aktivacija ReLU;
- sažimanje maksimumom 2×2 ;
- konvolucijski sloj s nadopunjavanjem (rezolucija ostaje jednaka): 32 jezgre 5×5 , korak 1, aktivacija ReLU;
- sažimanje maksimumom 2×2 ;

- pretvaranje u vektor;
- potpuno povezani sloj dimenzije 512, aktivacija ReLU;
- potpuno povezani sloj dimenzije 2, aktivacija softmax.

Zadaci:

- Koliko ukupno parametara ima ova mreža?
- Koliko ukupno receptivno polje imaju značajke iz drugog konvolucijskog sloja?
- Koliko bi parametara imala ova mreža kad bismo konvolucijske slojeve zamijenili potpuno povezanim slojevima s jednakim brojem izlaznih aktivacija.

Rješenje (a, b):

Bit će nam potrebna formula (1) kako bismo računali dimenzije izlaznih tenzora kroz slojeve. Također, bit će nam potrebna formula za računanje receptivnog polja:

$$R_i = R_{i-1} + (k_i - 1) \cdot \prod_{j=1}^{i-1} s_j,$$

gdje je R_i receptivno polje i -tog sloja, k_i je veličina jezgre i -tog sloja, a s_j je korak (eng. *stride*) j -tog sloja. Početni uvjet za ulaznu sliku je uvijek $R_0 = 1$. Da bismo odredili broj parametara, dimenzije mapa značajki te receptivno polje, prikazimo arhitekturu mreže kroz sljedeću tablicu:

Sloj	Broj parametara	Dimenzija izlaza	Receptivno polje
Slika	0	$1 \times 28 \times 28$	1×1
Conv1($C_{in} = 1$, $C_{out} = 16$, $k = 5$, $s = 1$, $p = 2$)	$16 \cdot 1 \cdot 5 \cdot 5 + 16$	$16 \times 28 \times 28$	5×5
ReLU	0	$16 \times 28 \times 28$	5×5
MaxPool1($k = 2$, $s = 2$)	0	$16 \times 14 \times 14$	6×6
Conv2($C_{in} = 16$, $C_{out} = 32$, $k = 5$, $s = 1$, $p = 2$)	$32 \cdot 16 \cdot 5 \cdot 5 + 32$	$32 \times 14 \times 14$	14×14
ReLU	0	$32 \times 14 \times 14$	14×14
MaxPool2($k = 2$, $s = 2$)	0	$32 \times 7 \times 7$	16×16
Flatten	0	1568	16×16
FC1	$512 \cdot 1568 + 512$	512	28×28
ReLU	0	512	28×28
FC2	$2 \cdot 512 + 2$	2	28×28
Softmax	0	2	28×28

- Ukupan broj parametara: $\sum \text{Broj parametara} = 817602$
- Receptivno polje značajki iz drugog konvolucijskog sloja: 14×14

Rješenje (c):

Radimo tablicu kao i u prethodnom zadatku, uz modifikaciju da konvolucijske slojeve mijenjamo potpuno povezanim slojevima s istim brojem izlaznih aktivacija. Zbog ove modifikacije u obzir moramo uzeti sljedeće:

- Budući da je ulazna slika 3D tenzor, vektorizaciju moramo provesti odmah na početku kako bismo imali valjani ulaz za potpuno povezani sloj.
- MaxPool operacija je ranije bila definirana kao operacija koja djeluje nad 2D prostornim dimenzijama tenzora, čime se ukupan broj aktivacija smanjivao za faktor 4 (jezgra 2×2 te pomak 2). Budući da je sada ulaz u MaxPool sloj 1D tenzor, odnosno vektor, pretpostavit ćemo da MaxPool sada djeluje nad vektorom po njegovoj jednoj dimenziji pomoću 1D jezgre veličine 4 i s korakom 4 ($k = 4, s = 4$) kako bi zadržao isti faktor smanjenja dimenzionalnosti kao u originalnoj mreži.

Sloj	Broj parametara	Dimenzija izlaza	Receptivno polje
Slika	0	$1 \times 28 \times 28$	1×1
Flatten	0	784	1×1
FC1	$12544 \cdot 784 + 12544$	12544	28×28
ReLU	0	12544	28×28
MaxPool1($k = 4, s = 4$)	0	3136	28×28
FC2	$6272 \cdot 3136 + 6272$	6272	28×28
ReLU	0	6272	28×28
MaxPool2($k = 4, s = 4$)	0	1568	28×28
FC3	$512 \cdot 1568 + 512$	512	28×28
ReLU	0	512	28×28
FC4	$2 \cdot 512 + 2$	2	28×28
Softmax	0	2	28×28

Ukupan broj parametara: $\sum \text{Broj parametara} = 30326658$

Zadatak 4

Razmatramo sloj dubokog modela koji ulazni vektor q preslikava u izlazni vektor p , a može se opisati sljedećim jednadžbama: $p_i = w_1 \cdot 1/q_i + w_2 \cdot q_i^2$. Predložite implementaciju tog sloja razredom koji nasljeđuje sučelje `Layer` iz prvog zadatka druge laboratorijske vježbe te implementira metode: `forward(self, inputs)`, `backward_inputs(self, grads)` i `backward_parameters(self, grads)`. U rješenju možete pretpostaviti da je veličina mini-grupe 1.

Rješenje:

Za implementaciju metoda `backward_inputs` i `backward_parameters` bit će nam potrebni gradijenti izlaznog vektora p s obzirom na ulazni vektor q te gradijenti izlaznog vektora p s obzirom na parametre w_1 i w_2 . Provedimo te izračune.

Gradijent izlaza s obzirom na ulaz po komponentama iznosi:

$$\frac{\partial p_i}{\partial q_j} = \begin{cases} 0, & \text{ako je } i \neq j \\ -w_1 \frac{1}{q_i^2} + 2w_2 q_i, & \text{ako je } i = j \end{cases}$$

Prikazano u obliku Jakobijeve matrice, jasno se vidi da je to dijagonalna matrica:

$$\frac{\partial p}{\partial q} = \begin{bmatrix} -w_1 \frac{1}{q_1^2} + 2w_2 q_1 & 0 & \dots & 0 \\ 0 & -w_1 \frac{1}{q_2^2} + 2w_2 q_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & -w_1 \frac{1}{q_k^2} + 2w_2 q_k \end{bmatrix}.$$

Zatim računamo gradijente izlaza s obzirom na parametre mreže. Prvo ćemo raspisati derivacije po pojedinim komponentama, a zatim ih prikazati u vektorskom obliku. Za parametar w_1 vrijedi:

$$\frac{\partial p_i}{\partial w_1} = \frac{1}{q_i}, \quad \frac{\partial p}{\partial w_1} = \begin{bmatrix} \frac{1}{q_1} & \frac{1}{q_2} & \dots & \frac{1}{q_k} \end{bmatrix}.$$

Za parametar w_2 vrijedi:

$$\frac{\partial p_i}{\partial w_2} = q_i^2, \quad \frac{\partial p}{\partial w_2} = \begin{bmatrix} q_1^2 & q_2^2 & \dots & q_k^2 \end{bmatrix}.$$

Sada možemo implementirati traženi razred. Zbog optimizacije memorije i brzine, umjesto stvaranja dijagonalne matrice (koja može biti velikih dimenzija), ulančavanje lokalnog gradijenta za ulaz provodimo *element-wise* množenjem vektora. To je moguće zato jer je ta operacija ekvivalentna množenju vektora s dijagonalnom matricom. U nastavku je dana implementacija rješenja u programskom jeziku Python.

```

1 import numpy as np
2
3 class MyLayer(Layer):
4     def __init__(self, name="MyLayer"):
5         self.w1 = np.random.randn()
6         self.w2 = np.random.randn()
7         self.q = None
8         self.name = name
9         self.has_params = True
10
11     def forward(self, inputs):
12         self.q = inputs
13         p = self.w1 * 1/inputs + self.w2 * inputs**2
14         return p
15
16     def backward_inputs(self, grads):
17         dp_dq_diag = -self.w1 * 1/(self.q * self.q) + 2 * self.w2 * self.q
18         return grads * dp_dq_diag
19
20     def backward_params(self, grads):
21         dp_dw1 = 1 / self.q
22         dp_dw2 = self.q * self.q
23
24         grad_w1 = grads @ dp_dw1
25         grad_w2 = grads @ dp_dw2
26
27     return [[self.w1, grad_w1], [self.w2, grad_w2], self.name]
```

Zadatak 5

Promotrite sljedeće dvije funkcije:

$$(a) f(x) = \sqrt{1 + \max(0, x)^2}$$

$$(b) g(x) = \frac{\exp(x)}{1 + \exp(x)}$$

Odredite derivacije funkcija f i g obzirom na ulaze. Na temelju dobivenih izraza odredite koja od dvije funkcija bi bila prikladniji izbor nelinearnosti za skrivene slojeve dubokog modela.

Rješenje:

Raspišimo najprije funkciju pod (a), odnosno $f(x)$, po slučajevima:

$$f(x) = \begin{cases} 1, & \text{ako je } x \leq 0 \\ \sqrt{1 + x^2}, & \text{ako je } x > 0 \end{cases}.$$

Derivirajmo sada funkciju $f(x)$ obzirom na ulaz x :

$$\frac{\partial f(x)}{\partial x} = \begin{cases} 0, & \text{ako je } x \leq 0 \\ \frac{x}{\sqrt{1+x^2}}, & \text{ako je } x > 0 \end{cases}.$$

Ova se derivacija može sažeto zapisati kao:

$$\frac{\partial f(x)}{\partial x} = \frac{\max(0, x)}{\sqrt{1 + x^2}}.$$

Funkcija pod (b), odnosno $g(x)$, je upravo sigmoida, $\sigma(x)$. Njezinu derivaciju znamo kao činjenicu i ona iznosi:

$$\frac{\partial g(x)}{\partial x} = g(x) \cdot (1 - g(x)).$$

Usporedba prikladnosti:

Funkcija $f(x)$ bila bi prikladniji izbor nelinearnosti za skrivene slojeve dubokog modela. S predavanja nam je poznato da je sigmoida (funkcija g) iznimno loš izbor za skrivene slojeve zbog problema nestajućeg gradijenta. Za vrijednosti ulaza koje su po apsolutnoj vrijednosti relativno velike, funkcija odlazi u zasićenje te njezina derivacija teži prema 0, što blokira unatražni prolaz gradijenta kroz slojeve modela.

S druge strane, funkcija $f(x)$ ima slična svojstva kao aktivacijska funkcija ReLU, za koju smo zaključili da je zadovoljavajuća. Za negativne vrijednosti ulaza gradijent je 0, a za pozitivne vrijednosti gradijent funkcije nikada ne iščezava. Također, možemo vidjeti da za vrlo velike pozitivne ulaze vrijedi:

$$\lim_{x \rightarrow \infty} \frac{x}{\sqrt{1 + x^2}} = 1.$$

Budući da za velike aktivacije derivacija teži k 1, gradijenti se tijekom unatražnog prolaza neće prigušiti, što rješava problem nestajućeg gradijenta u pozitivnoj domeni te omogućava stabilnije učenje dubokog modela. Iščezavanje gradijenta u negativnoj domeni nije problem te to možemo promatrati kao svojevrsnu regularizaciju koja uvodi rijetkost u model.

Zadatak 6

Napišite potpuni program za određivanje korijena kvadratne jednadžbe $a \cdot x^2 + b \cdot x + c = 0$ gradijentnim spustom. U vašem rješenju iskoristite mogućnost automatske diferencijacije koju nudi **Tensorflow PyTorch**. Podsjetnik: koristite operaciju koju vraća metoda `minimize` primjerka razreda `tf.train.GradientDescentOptimizer`.

Rješenje:

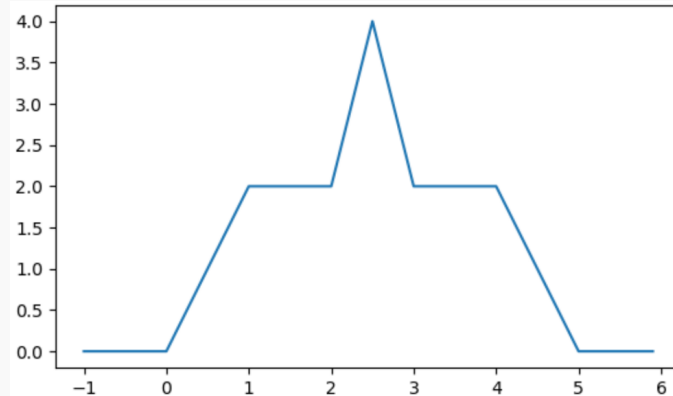
Traženje korijena zadane jednadžbe ekvivalentno je traženju minimuma kvadrata te jednadžbe. Kvadrat jednadžbe funkcionira kao funkcija gubitka jer poprima minimalnu vrijednost (0) točno u onim točkama koje su korijeni jednadžbe. U nastavku je dana implementacija rješenja u programskom jeziku Python koristeći Pytorch kao okvir za automatsku diferencijaciju.

```
1 import torch
2
3 # Inicijalizacija hiperparametara
4 niter = 10000
5 step = 0.01
6
7 # Parametri kvadratne jednadzbe (npr. a=1, b=0, c=-4 za jednadzbu x^2 -
   4 = 0)
8 a = 1.0
9 b = 0.0
10 c = -4.0
11
12 # Inicijalizacija pocetne tocke (mora zahtijevati gradijent)
13 x = torch.tensor(5.0, requires_grad=True)
14
15 for i in range(niter):
16     # Forward pass: kvadratna funkcija gubitka
17     y = (a * x**2 + b * x + c)**2
18
19     # Backward pass: automatsko racunanje gradijenta
20     y.backward()
21
22     # Azuriranje vrijednosti x
23     x.data -= step * x.grad
24
25     # Nuliranje gradijenta prije iduceg koraka
26     x.grad.zero_()
27
28 print(f"Pronadeni korijen: {x.item():.4f}")
```

2 MI 2017./2018.

Zadatak 1

Konstruirajte dvorazinski regresijski model koji kombinira affine transformacije i zglobnicu da bi aproksimirao funkciju zadanu slikom.



Rješenje:

Intuicija za rješavanje ovog zadatka oslanja se na svojstva zglobnice. Na svakom “prijelomu” zadane funkcije želimo “upaliti” novu zglobnicu koja će poništavati nagib pravca iz prethodnog intervala te pridonijeti nagibu pravca u novom intervalu koji tom točkom započinje.

Razmotrimo funkciju po dijelovima domene, gdje svaki dio odgovara intervalu između dva uzastopna prijeloma, te za svaki od tih intervala odredimo nagib pravca a_j . Neka su točke prijeloma označene s c_j .

Interval	$[-1, 0)$	$[0, 1)$	$[1, 2)$	$[2, 2.5)$	$[2.5, 3)$	$[3, 4)$	$[4, 5)$	$[5, 6]$
Nagib (a_j)	0	2	0	4	-4	0	-2	0

Svakom podintervalu pridružujemo jednu zglobnicu koja se “pali” u njenoj početnoj točki c_j . Izlazi prvog (ujedno i jedinog skrivenog) sloja ovog modela predstavljeni su sljedećim jednadžbama afinih transformacija i zglobnica:

$$h_0 = \text{ReLU}(x - (-1))$$

$$h_1 = \text{ReLU}(x - 0)$$

$$h_2 = \text{ReLU}(x - 1)$$

$$h_3 = \text{ReLU}(x - 2)$$

$$h_4 = \text{ReLU}(x - 2.5)$$

$$h_5 = \text{ReLU}(x - 3)$$

$$h_6 = \text{ReLU}(x - 4)$$

$$h_7 = \text{ReLU}(x - 5)$$

Konačno rješenje, odnosno jednadžbu izlaznog sloja modela, dobit ćemo tako da uzmemo linearnu kombinaciju ovih zglobnica pri čemu koeficijent w_j uz zglobnicu koja se pali u točki loma c_j odgovara razlici nagiba nakon i prije te točke loma:

$$w_j = a_j - a_{j-1}.$$

Iznimka je početna točka domene, za koju možemo pretpostaviti da je nagib prije nje $a_{-1} = 0$, pa vrijedi $w_0 = a_0$.

Dakle, dobivamo:

$$y = 0 \cdot h_0 + 2 \cdot h_1 - 2 \cdot h_2 + 4 \cdot h_3 - 8 \cdot h_4 + 4 \cdot h_5 - 2 \cdot h_6 + 2 \cdot h_7.$$

Zadatak 2

Napišite jednadžbu gubitka negativne log-izglednosti za grupu od N podataka. Izvedite izraz za gradijente tog gubitka s obzirom na ulaz softmaksa u svim podacima grupe.

Unakrsnu entropiju predikcije modela u podatku x_i s obzirom na željenu distribuciju y_i možemo izraziti jednadžbom: $\mathcal{L}_{CE} = -\sum_{j=1}^C y_{ij} \log p(Y = y_{ij} | x_i)$. Odredite gradijent gubitka $\mathcal{L}_{CE}$ s obzirom na ulaz softmaksa u podatku x_i .

Napomena: U tekstu zadatka krije se pogreška u izrazu za unakrsnu entropiju. Naime, umjesto $p(Y = y_{ij} | x_i)$, trebalo bi pisati $p(Y = j | x_i)$.

Rješenje:

Pretpostavimo da imamo C klasa: $\{1, \dots, C\}$. Neka su zadani podatci $X = \{x_i\}_{i=1}^N$. Neka je $\hat{y}_i$ prediktirana distribucija pripadnosti klasama za primjer x_i .

Budući da je negativna log-izglednost specijalan slučaj unakrsne entropije za slučaj jedno-jediničnih oznaka, najprije ćemo odrediti gradijent gubitka $\mathcal{L}_{CE}$ s obzirom na ulaz softmaksa u podatku x_i . Kako bismo si pojednostavili raspisivanje za samo jedan primjer, možemo zanemariti indeks i .

Vjerojatnost $p(Y = j | x)$ predstavlja j -ti izlaz softmaksa za logite s :

$$\mathcal{L}_{CE} = -\sum_{j=1}^C y_j \log(p(Y = j | x)) = -\sum_{j=1}^C y_j \log(\text{softmax}(s)_j). \quad (4)$$

Sada je dobra ideja srediti izraz $\log(\text{softmax}(s)_j)$:

$$\log(\text{softmax}(s)_j) = \log\left(\frac{\exp(s_j)}{\sum_{k=1}^C \exp(s_k)}\right) = s_j - \log\left(\sum_{k=1}^C \exp(s_k)\right). \quad (5)$$

Uvrštavamo izraz (5) natrag u jednadžbu (4). Pri tome koristimo činjenicu da je suma svih elemenata stvarne distribucije klasa jednaka 1, odnosno $\sum_{j=1}^C y_j = 1$:

$$\begin{aligned} \mathcal{L}_{CE} &= -\sum_{j=1}^C y_j \left(s_j - \log\left(\sum_{k=1}^C \exp(s_k)\right) \right) \\ &= -\sum_{j=1}^C y_j s_j + \sum_{j=1}^C y_j \log\left(\sum_{k=1}^C \exp(s_k)\right) \\ &= -\sum_{j=1}^C y_j s_j + \log\left(\sum_{k=1}^C \exp(s_k)\right) \sum_{j=1}^C y_j \\ &= -\sum_{j=1}^C y_j s_j + \log\left(\sum_{k=1}^C \exp(s_k)\right) \cdot 1 \\ &= \log\left(\sum_{k=1}^C \exp(s_k)\right) - \sum_{j=1}^C y_j s_j. \end{aligned}$$

Sada smo izraz sredili do točke gdje možemo odrediti gradijent s obzirom na ulaz u softmaks. Najprije odredimo gradijent s obzirom na l -tu komponentu ulaza u softmaks, odnosno s_l :

$$\begin{aligned}\frac{\partial \mathcal{L}_{CE}}{\partial s_l} &= \frac{\exp(s_l)}{\sum_{k=1}^C \exp(s_k)} - y_l \\ &= p(Y = l \mid x) - y_l \\ &= \hat{y}_l - y_l.\end{aligned}$$

Iz ovoga slijedi da je gradijent s obzirom na cijeli vektor ulaza u softmaks jednak:

$$\frac{\partial \mathcal{L}_{CE}}{\partial s} = \hat{y} - y.$$

Posebno, ako sada ponovno uzmemo u obzir da ovo razmatramo za i -ti primjer, dobivamo:

$$\frac{\partial (\mathcal{L}_{CE})_i}{\partial s_i} = \hat{y}_i - y_i.$$

Radi jednostavnosti, označimo $(\mathcal{L}_{CE})_i$ kao $\mathcal{L}_i$

Gubitak negativne log-izglednosti za cijelu grupu (mini-grupu) od N podataka definiramo kao prosjek gubitaka pojedinačnih primjera:

$$\mathcal{L}_{CE} = \frac{1}{N} \sum_{j=1}^N \mathcal{L}_j.$$

Kada računamo gradijent tog ukupnog gubitka s obzirom na ulaz softmaksu specifičnog i -tog primjera (s_i), primjećujemo da s_i utječe samo na i -ti član sume. Svi ostali članovi sume su u odnosu na varijablu s_i konstante te njihova derivacija iznosi 0. Stoga vrijedi:

$$\frac{\partial \mathcal{L}_{CE}}{\partial s_i} = \frac{\partial}{\partial s_i} \left(\frac{1}{N} \sum_{j=1}^N \mathcal{L}_j \right) = \frac{1}{N} \frac{\partial \mathcal{L}_i}{\partial s_i} = \frac{1}{N} (\hat{y}_i - y_i).$$

Nadalje, logite grupe možemo spremiti u matricu $S \in \mathbb{R}^{N \times C}$ tako da u i -ti redak matrice S smjestimo vektor redak logita i -tog primjera. Neka je matrica $\hat{Y} \in \mathbb{R}^{N \times C}$ matrica predikcija gdje se u i -tom retku nalazi vektor redak predikcija za i -ti primjer. Neka je matrica $Y \in \mathbb{R}^{N \times C}$ matrica stvarnih oznaka gdje se u i -tom retku nalazi vektor redak stvarnih oznaka za i -ti primjer.

Tada, gradijent ukupnog gubitka s obzirom na cijelu matricu S jednak je:

$$\frac{\partial \mathcal{L}_{CE}}{\partial S} = \frac{1}{N} (\hat{Y} - Y).$$

Razmotrimo sada gubitak negativne log-izglednosti. Rekli smo da je gubitak negativne log-izglednosti specijalan slučaj gubitka unakrsne entropije za slučaj jednojedinčnih oznaka. Prema tome, vrijedi:

$$\mathcal{L}_{NLL} = \frac{1}{N} \sum_{i=1}^N \mathcal{L}_i = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^C y_{ij} \log(\hat{y}_{ij})$$

Budući da su oznake jednojedinčne vrijedi da za i -ti primjer samo jedna vrijednost y_{ij} poprima vrijednost 1, dok ostale poprimaju vrijednost 0. Prema tome, možemo pisati:

$$\mathcal{L}_{NLL} = -\frac{1}{N} \sum_{i=1}^N \log(\hat{y}_{i c_i})$$

gdje je c_i točna klasa i -tog primjera.

Nadalje, budući da smo već izveli izraze za gradijente gubitka negativne log-izglednosti, možemo uočiti da uvođenje pretpostavke o jednojedinčnim oznakama neće promijeniti te izraze. Prema tome, vrijedi:

$$\frac{\partial \mathcal{L}_{NLL}}{\partial s_i} = \frac{1}{N}(\hat{y}_i - y_i).$$

$$\frac{\partial \mathcal{L}_{NLL}}{\partial S} = \frac{1}{N}(\hat{Y} - Y),$$

Zadatak 3

Razmatramo klasifikacijski duboki model koji podatak $\mathbf{x}$ preslikava u distribuciju $\mathbf{p}$:

$$\mathbf{h}_A = \text{ReLU}(\mathbf{W}_A \cdot \mathbf{x} + \mathbf{b}_A)$$

$$\mathbf{s} = \mathbf{W}_B \cdot \mathbf{h}_A + \mathbf{b}_B$$

$$\mathbf{p} = \text{softmax}(\mathbf{s})$$

- Odredite izlaze mreže ako je na ulazu zadan podatak $(2, 4)$, a početni parametri su: $\mathbf{W}_A = \begin{bmatrix} -1 & 1 \end{bmatrix}$, $\mathbf{b}_A = [-1]$, $\mathbf{W}_B = \begin{bmatrix} 1 & -1 \end{bmatrix} \cdot \ln(2)$, $\mathbf{b}_B = \begin{bmatrix} 1 & 1 \end{bmatrix}$
- odredite gubitak negativne log-izglednosti ako je podatak označen distribucijom $y = (0, 1)$;
- odredite gradijente gubitka negativne log-izglednosti s obzirom na težine $\mathbf{W}_A$ i $\mathbf{W}_B$;
- odredite parcijalnu derivaciju $\partial \mathbf{s} / \partial \text{vec}(\mathbf{W}_B)$ i komentirajte svrsishodnost njenog korištenja u praksi; napomena: $\text{vec}(\mathbf{W}_B)$ označava vektor sa svim elementima tenzora $\mathbf{W}_B$;
- komentirajte zalihost modela: koji dio modela bi se mogao izostaviti bez utjecaja na kapacitet?

Rješenje (a):

Za početak, uvedimo novu oznaku za logite prvog sloja $\mathbf{s}_A$:

$$\mathbf{s}_A = \mathbf{W}_A \cdot \mathbf{x} + \mathbf{b}_A = \begin{bmatrix} -1 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 4 \end{bmatrix} + [-1] = (-2 + 4) - 1 = 1.$$

Račun nastavljamo prolazom kroz ostale slojeve:

$$\mathbf{h}_A = \text{ReLU}(\mathbf{s}_A) = \text{ReLU}(1) = 1.$$

$$\mathbf{s} = \mathbf{W}_B \cdot \mathbf{h}_A + \mathbf{b}_B = \ln(2) \cdot \begin{bmatrix} 1 \\ -1 \end{bmatrix} \cdot 1 + \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} \ln(2) + 1 \\ -\ln(2) + 1 \end{bmatrix} = \begin{bmatrix} \ln(2) + \ln(e) \\ \ln(1/2) + \ln(e) \end{bmatrix} = \begin{bmatrix} \ln(2e) \\ \ln(e/2) \end{bmatrix}.$$

$$\mathbf{p} = \text{softmax}(\mathbf{s}) = \begin{bmatrix} 2e/(2e + e/2) \\ (e/2)/(2e + e/2) \end{bmatrix} = \begin{bmatrix} 2/(5/2) \\ (1/2)/(5/2) \end{bmatrix} = \begin{bmatrix} 4/5 \\ 1/5 \end{bmatrix}.$$

Rješenje (b):

Zadana je stvarna distribucija $\mathbf{y} = \begin{bmatrix} 0 & 1 \end{bmatrix}^\top$. Gubitak negativne log-izglednosti iznosi:

$$\mathcal{L}_{NLL} = -y_0 \ln(p_0) - y_1 \ln(p_1) = -0 \cdot \ln(4/5) - 1 \cdot \ln(1/5) = -\ln(1/5) = \ln(5).$$

Rješenje (c):

Zadana je stvarna distribucija $\mathbf{y} = \begin{bmatrix} 0 & 1 \end{bmatrix}^\top$.

Postupak unatragnog prolaza započinjemo računanjem gradijenta gubitka po ulazu u soft-maks $\mathbf{s}$:

$$\frac{\partial \mathcal{L}_{NLL}}{\partial \mathbf{s}} = (\mathbf{p} - \mathbf{y})^\top = \left(\begin{bmatrix} 4/5 \\ 1/5 \end{bmatrix} - \begin{bmatrix} 0 \\ 1 \end{bmatrix} \right)^\top = \begin{bmatrix} 4/5 & -4/5 \end{bmatrix}.$$

Zatim, po pravilu ulančavanja, računamo gradijent gubitka po matrici težina $\mathbf{W}_B$:

$$\frac{\partial \mathcal{L}_{NLL}}{\partial \mathbf{W}_B} = \frac{\partial \mathcal{L}_{NLL}}{\partial \mathbf{s}} \frac{\partial \mathbf{s}}{\partial \mathbf{W}_B} = \left(\frac{\partial \mathcal{L}_{NLL}}{\partial \mathbf{s}} \right)^\top \mathbf{h}_A^\top = \begin{bmatrix} 4/5 \\ -4/5 \end{bmatrix} \cdot 1 = \begin{bmatrix} 4/5 \\ -4/5 \end{bmatrix}.$$

Nadalje, računamo gradijent gubitka po izlazu skrivenog sloja $\mathbf{h}_A$:

$$\frac{\partial \mathcal{L}_{NLL}}{\partial \mathbf{h}_A} = \frac{\partial \mathcal{L}_{NLL}}{\partial \mathbf{s}} \frac{\partial \mathbf{s}}{\partial \mathbf{h}_A} = \frac{\partial \mathcal{L}_{NLL}}{\partial \mathbf{s}} \mathbf{W}_B = \begin{bmatrix} 4/5 & -4/5 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} \cdot \ln(2) = \frac{8}{5} \ln(2).$$

Slijedi gradijent gubitka po ulazu u aktivacijsku funkciju $\mathbf{s}_A$:

$$\frac{\partial \mathcal{L}_{NLL}}{\partial \mathbf{s}_A} = \frac{\partial \mathcal{L}_{NLL}}{\partial \mathbf{h}_A} \frac{\partial \mathbf{h}_A}{\partial \mathbf{s}_A} = \frac{\partial \mathcal{L}_{NLL}}{\partial \mathbf{h}_A} \odot \llbracket \mathbf{s}_A > 0 \rrbracket = \frac{8}{5} \ln(2) \cdot 1 = \frac{8}{5} \ln(2).$$

Konačno, računamo gradijent gubitka po matrici težina $\mathbf{W}_A$:

$$\frac{\partial \mathcal{L}_{NLL}}{\partial \mathbf{W}_A} = \frac{\partial \mathcal{L}_{NLL}}{\partial \mathbf{s}_A} \frac{\partial \mathbf{s}_A}{\partial \mathbf{W}_A} = \left(\frac{\partial \mathcal{L}_{NLL}}{\partial \mathbf{s}_A} \right)^\top \mathbf{x}^\top = \frac{8}{5} \ln(2) \begin{bmatrix} 2 & 4 \end{bmatrix} = \frac{\ln(2)}{5} \begin{bmatrix} 16 & 32 \end{bmatrix}.$$

Rješenje (d):

Vektorizirana matrica $\mathbf{W}_B$ dimenzija 2×1 iznosi:

$$\text{vec}(\mathbf{W}_B) = \begin{bmatrix} W_{B,11} & W_{B,21} \end{bmatrix}^\top.$$

Napišimo kako se računa jedan element vektora $\mathbf{s}$:

$$s_i = \sum_j (W_{B,ij} \cdot h_{A,j}) + b_{B,i}.$$

Razmotrimo sada gradijent jednog elementa vektora $\mathbf{s}$ po proizvoljnom elementu $\text{vec}(\mathbf{W}_B)$:

$$\frac{\partial s_i}{\partial W_{B,kl}} = \begin{cases} h_{A,l}, & \text{ako je } i = k \\ 0, & \text{inače} \end{cases}.$$

U našem slučaju, izlaz skrivenog sloja $\mathbf{h}_A$ je skalar pa imamo samo indeks $j = l = 1$. Imamo dvije komponente izlaza $\mathbf{s}$:

$$\begin{aligned}s_1 &= W_{B,11} \cdot h_{A,1} + b_{B,1} \\ s_2 &= W_{B,21} \cdot h_{A,1} + b_{B,2}\end{aligned}$$

Parcijalne derivacije po elementima su:

$$\begin{aligned}\frac{\partial s_1}{\partial W_{B,11}} &= h_{A,1} = 1, & \frac{\partial s_1}{\partial W_{B,21}} &= 0 \\ \frac{\partial s_2}{\partial W_{B,11}} &= 0, & \frac{\partial s_2}{\partial W_{B,21}} &= h_{A,1} = 1\end{aligned}$$

Prikazano u obliku Jakobijeve matrice, dobivamo:

$$\frac{\partial \mathbf{s}}{\partial \text{vec}(\mathbf{W}_B)} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

Komentar o svrsishodnosti: Eksplicitno instanciranje ove Jakobijeve matrice u praksi je nesvrsishodno jer su takve matrice najčešće ogromnih dimenzija i rijetke (većina elemenata su nule), što bi dovelo do neprihvatljivog zauzeća memorije. Shodno tome, umjesto ovoga se koriste posebni izrazi koji se temelje na matričnim umnošcima odgovarajućih jakobijana.

Rješenje (e):

Dio modela koji bi se mogao izostaviti bez utjecaja na kapacitet je vektor pomaka zadnjeg sloja, odnosno vektor $\mathbf{b}_B$.

Razlog tome je to da je funkcija softmax invarijantna na translaciju (dodavanje konstante). Budući da vektor $\mathbf{b}_B = \begin{bmatrix} 1 & 1 \end{bmatrix}^T$ dodaje istu konstantnu vrijednost objema komponentama vektora logita $\mathbf{s}$, ta će se konstanta prilikom prolaska kroz softmax izlučiti i u brojniku i u nazivniku te pokratiti, što znači da neće imati utjecaja na prediktiranu distribuciju.

Zadatak 4

Razmatramo konvolucijski duboki model koji 2D tenzor $\mathbf{x}$ preslikava u vjerojatnost p :

$$\begin{aligned}\mathbf{h}_A &= \text{ReLU}(\text{conv}(\mathbf{x}, \mathbf{w}_A) + \mathbf{b}_A) \\ \mathbf{h}_B &= \text{conv}(\mathbf{h}_A, \mathbf{w}_B) + \mathbf{b}_B \\ s &= \text{GMP}(\mathbf{h}_B) \\ p &= \sigma(s \cdot \ln(2))\end{aligned}$$

Pri tome ReLU označava zglobnicu, conv – konvoluciju bez nadopunjavanja, a GMP – globalno sažimanje maksimalnom vrijednošću. Tenzor kojeg proizvodi GMP ima prostorne dimenzije 1×1 , dok broj kanala izlaza odgovara broju kanala ulaza.

Tenzor $\mathbf{w}_A$ ima oblik $2 \times 1 \times 3 \times 3$. Prve dvije dimenzije odgovaraju broju kanala izlaza i ulaza. Posljednje dvije dimenzije odgovaraju retcima i stupcima po kojima provodimo konvoluciju i sažimanje. Početne vrijednosti elemenata $\mathbf{w}_A$ su: $\mathbf{w}_{A0010} = -1$, $\mathbf{w}_{A0011} = 1$, $\mathbf{w}_{A1021} = -1$, $\mathbf{w}_{A1011} = 1$, a svi ostali elementi su 0.

Tenzor $\mathbf{w}_B$ ima oblik $1 \times 2 \times 1 \times 1$ te jednaku organizaciju kao i $\mathbf{w}_A$. Početne vrijednosti elemenata $\mathbf{w}_B$ su: $\mathbf{w}_{B0000} = -1$, $\mathbf{w}_{B0100} = 1$, a svi ostali elementi su 0.

Početne vrijednosti svih elemenata pomaka $\mathbf{b}_A$ i $\mathbf{b}_B$ su 0.

Ulazni tenzor $\mathbf{x}$ ima oblik 4×4 , $\mathbf{x}_{11} = \mathbf{x}_{12} = \mathbf{x}_{22} = 1$, a svi ostali elementi su 0. Točan razred podatka $y = 1$.

Odredite izlaz modela te izračunajte gradijente negativne log-izglednosti s obzirom na elemente konvolucijske jezgre $\mathbf{w}_B$.
 Svi indeksi počinju od nule.
 BONUS: izračunajte gradijente negativne log-izglednosti s obzirom na elemente konvolucijske jezgre $\mathbf{w}_A$.

Rješenje:

Zadani su sljedeći podaci:

$$\mathbf{w}_{A00} = \begin{bmatrix} 0 & 0 & 0 \\ -1 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{w}_{A10} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -1 & 0 \end{bmatrix}$$

$$\mathbf{w}_{B00} = [-1], \quad \mathbf{w}_{B01} = [1]$$

$$\mathbf{b}_A = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad b_B = 0$$

$$\mathbf{x} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad y = 1$$

Unaprijedni prolaz započinjemo računanjem konvolucije za prvi sloj:

$$\mathbf{g}_{A0} = \text{conv}(\mathbf{x}, \mathbf{w}_{A00}) = \mathbf{w}_{A00} \star \mathbf{x} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$\mathbf{g}_{A1} = \text{conv}(\mathbf{x}, \mathbf{w}_{A10}) = \mathbf{w}_{A10} \star \mathbf{x} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Dodajemo pomak i propuštamo kroz aktivacijsku funkciju:

$$\mathbf{f}_{A0} = \mathbf{g}_{A0} + b_{A0} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{h}_{A0} = \text{ReLU}(\mathbf{f}_{A0}) = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$\mathbf{f}_{A1} = \mathbf{g}_{A1} + b_{A1} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{h}_{A1} = \text{ReLU}(\mathbf{f}_{A1}) = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Nadalje, računamo konvoluciju drugog sloja zbrajanjem doprinosa po kanalima:

$$\begin{aligned} \mathbf{g}_B &= \text{conv}(\mathbf{h}_{A0}, \mathbf{w}_{B00}) + \text{conv}(\mathbf{h}_{A1}, \mathbf{w}_{B01}) \\ &= (\mathbf{w}_{B00} \star \mathbf{h}_{A0}) + (\mathbf{w}_{B01} \star \mathbf{h}_{A1}) \\ &= \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix} + \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \end{aligned}$$

Nakon dodavanja pomaka, primjenjujemo globalno sažimanje maksimumom (GMP):

$$\mathbf{h}_B = \mathbf{g}_B + b_B = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \implies s = \text{GMP}(\mathbf{h}_B) = 0$$

Konačno, računamo vjerojatnost množenjem s konstantom i propuštanjem kroz sigmoidu:

$$s_1 = s \cdot \ln(2) = 0 \implies p = \sigma(s_1) = \sigma(0) = 0.5$$

Gubitak negativne log-izglednosti za binarnu klasifikaciju glasi:

$$\begin{aligned}\mathcal{L} &= -y \log(p(y=1|\mathbf{x})) - (1-y) \log(p(y=0|\mathbf{x})) \\ &= -y \log(\sigma(z)) - (1-y) \log(1-\sigma(z))\end{aligned}$$

gdje je $z = f(\mathbf{x})$, a f neka transformacija nad $\mathbf{x}$.

Iskoristimo ovu priliku da se prisjetimo izvoda gradijenta gubitka po ulazu u sigmoidu:

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial z} &= -y \cdot \frac{1}{\sigma(z)} \cdot \sigma(z)(1-\sigma(z)) - (1-y) \cdot \frac{1}{1-\sigma(z)} \cdot (-1) \cdot \sigma(z)(1-\sigma(z)) \\ &= -y(1-\sigma(z)) + (1-y)\sigma(z) \\ &= -y + y\sigma(z) + \sigma(z) - y\sigma(z) \\ &= \sigma(z) - y\end{aligned}$$

U našem slučaju je $z = s_1$ i $y = 1$, pa započnimo s izračunom gradijenta propagacijom unatrag:

$$\frac{\partial \mathcal{L}}{\partial s_1} = \sigma(s_1) - y = p - y = 0.5 - 1 = -0.5$$

Slijedi gradijent po izlazu iz globalnog sažimanja s :

$$\frac{\partial \mathcal{L}}{\partial s} = \frac{\partial \mathcal{L}}{\partial s_1} \frac{\partial s_1}{\partial s} = -0.5 \cdot \ln(2)$$

Na ovom mjestu valja se prisjetiti kako računamo gradijent izlaza GMP-a s obzirom na njegov ulaz $\mathbf{h}_B$. Budući da GMP daje maksimum za svaki od ulaznih kanala, to znači da je u svakom izlaznom kanalu samo jedna vrijednost koja dolazi s pozicije maksimuma odgovarajućeg ulaznog kanala. Prema tome, gradijent je tenzor čija struktura odgovara strukturi ulaza te ima jedinice na onim pozicijama iz kojih su izvučeni maksimumi, a na svim ostalim pozicijama ima vrijednost 0. Dobro je još napomenuti da u slučaju višestrukog pojavljivanja maksimalne vrijednosti, dogovorno razmatramo samo prvu poziciju na kojoj se pojavila ta vrijednost.

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_B} = \frac{\partial \mathcal{L}}{\partial s} \frac{\partial s}{\partial \mathbf{h}_B} = \frac{\partial \mathcal{L}}{\partial s} \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} = -0.5 \ln(2) \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$$

Gradijent po ulazu u operaciju dodavanja pomaka $\mathbf{g}_B$ je jednak:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{g}_B} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_B} \frac{\partial \mathbf{h}_B}{\partial \mathbf{g}_B} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_B} \cdot 1 = -0.5 \ln(2) \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$$

Računanje gradijenta gubitka s obzirom na elemente konvolucijske matrice $\mathbf{w}_B$ podrazumijeva računanje gradijenata s obzirom na svaki kanal konvolucijske jezgre. Ovaj izraz za računanje gradijenta gubitka po težinama konvolucije može se pronaći u prezentaciji predmeta, no valja napomenuti da ovdje nismo dodali *padding* na ulaz konvolucije budući da ga nije bilo ni u unaprijednom prolazu.

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}_{B00}} = \frac{\partial \mathcal{L}}{\partial \mathbf{g}_B} \star \mathbf{h}_{A0} = -0.5 \ln(2) \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \star \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = -0.5 \ln(2) \cdot 1 = -0.5 \ln(2)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}_{B01}} = \frac{\partial \mathcal{L}}{\partial \mathbf{g}_B} \star \mathbf{h}_{A1} = -0.5 \ln(2) \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \star \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = -0.5 \ln(2) \cdot 1 = -0.5 \ln(2)$$

BONUS:

Najprije moramo izračunati gradijent s obzirom na $\mathbf{h}_A$. To je 3D tenzor, pa računanje gradijenta radimo po svakom od kanala.

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial \mathbf{h}_{A0}} &= \text{flip2d}(\mathbf{w}_{B00}) \star \text{pad}\left(\frac{\partial \mathcal{L}}{\partial \mathbf{g}_B}, k_B - 1\right) \\ \frac{\partial \mathcal{L}}{\partial \mathbf{h}_{A1}} &= \text{flip2d}(\mathbf{w}_{B01}) \star \text{pad}\left(\frac{\partial \mathcal{L}}{\partial \mathbf{g}_B}, k_B - 1\right)\end{aligned}$$

Ovaj izraz za računanje gradijenta gubitka po ulazu konvolucije može se pronaći u prezentaciji predmeta, no valja napomenuti da ovdje dodajemo nadopunjavanje (*padding*) koje odgovara veličini jezgre umanjenoj za jedan jer nadopunjavanja nije bilo u unaprijednom prolazu. Ovakvo pravilo o nadopunjavanju vrijedi općenito kada računamo gradijent gubitka po ulazima konvolucije.

No, primjećujemo da u ovom konkretnom slučaju efektivno neće biti nadopunjavanja jer vrijedi $k_B - 1 = 1 - 1 = 0$.

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial \mathbf{h}_{A0}} &= [-1] \star \left(-0.5 \ln(2) \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}\right) = 0.5 \ln(2) \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \\ \frac{\partial \mathcal{L}}{\partial \mathbf{h}_{A1}} &= [1] \star \left(-0.5 \ln(2) \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}\right) = -0.5 \ln(2) \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}\end{aligned}$$

Dalje računamo:

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial \mathbf{g}_{A0}} &= \frac{\partial \mathcal{L}}{\partial \mathbf{h}_{A0}} \frac{\partial \mathbf{h}_{A0}}{\partial \mathbf{g}_{A0}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_{A0}} \odot [\mathbf{g}_{A0} > 0] \\ &= 0.5 \ln(2) \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \odot \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = 0.5 \ln(2) \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \\ \frac{\partial \mathcal{L}}{\partial \mathbf{g}_{A1}} &= \frac{\partial \mathcal{L}}{\partial \mathbf{h}_{A1}} \frac{\partial \mathbf{h}_{A1}}{\partial \mathbf{g}_{A1}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_{A1}} \odot [\mathbf{g}_{A1} > 0] \\ &= -0.5 \ln(2) \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \odot \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = -0.5 \ln(2) \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \\ \frac{\partial \mathcal{L}}{\partial \mathbf{w}_{A00}} &= \frac{\partial \mathcal{L}}{\partial \mathbf{g}_{A0}} \star \mathbf{x} \\ &= 0.5 \ln(2) \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \star \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} = 0.5 \ln(2) \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix} \\ \frac{\partial \mathcal{L}}{\partial \mathbf{w}_{A10}} &= \frac{\partial \mathcal{L}}{\partial \mathbf{g}_{A1}} \star \mathbf{x} \\ &= -0.5 \ln(2) \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \star \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} = -0.5 \ln(2) \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}\end{aligned}$$

Zadatak 5

Oblikujte funkciju koja implementira optimizacijski algoritam ADAM. Napišite ispitni kod koji uz pomoć te funkcije pronalazi minimum funkcije $x^4 - 4x$ tijekom 10000 iteracija, počevši iz $x = -2$. Zadatak riješite prema sljedećim uputama.

- (a) Neka vas ne zbuni što se nigdje ne spominju podatci: njih smo izostavili kako bi zadatak bio jednostavniji;
- (b) tražena funkcija treba raditi samo za skalarne funkcije skalarne varijable;
- (c) neka argumenti funkcije budu:
 - početna vrijednost parametra (`theta`),
 - funkcija koja računa gradijent (`fgrad`), te
 - broj iteracija (`n`);
- (d) neka povratna vrijednost funkcije bude konačna vrijednost parametra;
- (e) nemojte komplicirati: naša implementacija ima 20 redaka Pythona od čega 5 redaka otpada na inicijalizaciju hiperparametara.

Rješenje:

Tražena implementacija algoritma ADAM u jeziku Python:

```
1 def adam(theta, fgrad, n):
2     eta = 0.001
3     ro1 = 0.9
4     ro2 = 0.999
5     delta = 1e-8
6     s = 0
7     r = 0
8     t = 0
9
10    for _ in range(n):
11        g = fgrad(theta)
12        t += 1
13        s = ro1 * s + (1 - ro1) * g
14        r = ro2 * r + (1 - ro2) * g * g
15        s_hat = s / (1 - (ro1**t))
16        r_hat = r / (1 - (ro2**t))
17        dtheta = -eta * s_hat / (r_hat**(1/2) + delta)
18        theta += dtheta
19
20    return theta
21
22 theta = adam(-2, lambda x: (4 * x**3 - 4), 10000)
23 print(theta)
24
```

Zadatak 6

Razmatramo sloj dubokog modela koji ulazni vektor $\mathbf{p}$ transformira u izlazni vektor $\mathbf{q}$ koristeći skalarne parametre w_0, w_1 i w_2 , a može se opisati jednačbom: $q_i = w_0 \cdot p_{i-1} +$

$$w_1 \cdot p_i + w_2 \cdot p_{i+1}.$$

Odredite Jakobijan gubitka po ulazu i Jakobijan gubitka po parametrima ako je poznat Jakobijan gubitka po izlazu.

Napišite kod koji bi omogućio uklapanje sloja u duboki model proizvoljne složenosti. Sloj izrazite razredom koji implementira sučelje `Layer` iz prvog zadatka druge laboratorijske vježbe te implementira sljedeće metode: `forward(self, inputs)`, `backward_inputs(self, grads)`, te `backward_parameters(self, grads)`.

Izvedba treba osigurati da dimenzionalnost izlaznog vektora bude jednaka dimenzionalnosti ulaznog vektora.

Pomoć: numpyjev tenzor `x` proizvoljne dimenzionalnosti možemo nadopuniti nulama sljedećim pozivom: `np.lib.pad(x, 1, mode='constant')`.

Rješenje:

Neka je ulazni vektor $\mathbf{p}$ duljine n i neka je $\mathbf{p} = [p_0 \ p_1 \ \dots \ p_{n-1}]^\top$. Tada se komponente izlaznog vektora $\mathbf{q}$ računaju ovako:

$$\begin{aligned} q_0 &= w_0 \cdot p_{-1} + w_1 \cdot p_0 + w_2 \cdot p_1 \\ q_1 &= w_0 \cdot p_0 + w_1 \cdot p_1 + w_2 \cdot p_2 \\ q_2 &= w_0 \cdot p_1 + w_1 \cdot p_2 + w_2 \cdot p_3 \\ &\vdots \\ q_{n-2} &= w_0 \cdot p_{n-3} + w_1 \cdot p_{n-2} + w_2 \cdot p_{n-1} \\ q_{n-1} &= w_0 \cdot p_{n-2} + w_1 \cdot p_{n-1} + w_2 \cdot p_n \end{aligned}$$

Iz ovih izraza vidimo da se pri računanju izlaznog vektora $\mathbf{q}$, ulazni vektor $\mathbf{p}$ mora nadopuniti s po jednom nulom na početku i kraju, tako da vrijedi $p_{-1} = p_n = 0$.

Iz gornjih izraza relativno lako možemo raspisati Jakobijan izlaza po ulazima. Dobivamo matricu u kojoj se na glavnoj dijagonali nalazi w_1 , na prvoj donjoj sporednoj dijagonali w_0 , a na prvoj gornjoj sporednoj dijagonali w_2 :

$$\frac{\partial \mathbf{q}}{\partial \mathbf{p}} = \begin{bmatrix} w_1 & w_2 & & & \\ w_0 & w_1 & w_2 & & \\ & w_0 & w_1 & w_2 & \\ & & \ddots & \ddots & \ddots \\ & & & w_0 & w_1 \end{bmatrix}.$$

Nadalje, Jakobijani izlaza $\mathbf{q}$ po parametrima w_0, w_1, w_2 su vektori retci:

$$\begin{aligned} \frac{\partial \mathbf{q}}{\partial w_0} &= [0 \ p_0 \ p_1 \ \dots \ p_{n-2}] \\ \frac{\partial \mathbf{q}}{\partial w_1} &= [p_0 \ p_1 \ \dots \ p_{n-1}] \\ \frac{\partial \mathbf{q}}{\partial w_2} &= [p_1 \ p_2 \ \dots \ p_{n-1} \ 0] \end{aligned}$$

Poznavajući Jakobijan gubitka po izlazu ($\frac{\partial \mathcal{L}}{\partial \mathbf{q}}$), Jakobijan gubitka po ulazu i parametrima računamo lančanim pravilom:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{p}} = \frac{\partial \mathcal{L}}{\partial \mathbf{q}} \frac{\partial \mathbf{q}}{\partial \mathbf{p}}, \quad \frac{\partial \mathcal{L}}{\partial w_k} = \frac{\partial \mathcal{L}}{\partial \mathbf{q}} \left(\frac{\partial \mathbf{q}}{\partial w_k} \right)^\top.$$

Implementacija sloja u jeziku Python:


```

1 import numpy as np
2
3 class MyLayer(Layer):
4     def __init__(self, name="MyLayer"):
5         self.has_params = True
6         self.name = name
7         self.p = None
8         self.w0 = np.random.randn(1)
9         self.w1 = np.random.randn(1)
10        self.w2 = np.random.randn(1)
11
12    def forward(self, inputs):
13        self.p = inputs
14        padded_p = np.lib.pad(self.p, 1, mode='constant')
15        n = self.p.shape[0]
16        q = np.zeros(n)
17        for i in range(n):
18            q[i] = self.w0 * padded_p[i] + self.w1 * padded_p[i + 1] +
self.w2 * padded_p[i + 2]
19        return q
20
21    def backward_inputs(self, grads):
22        n = self.p.shape[0]
23        dq_dp = np.zeros((n, n))
24        np.fill_diagonal(dq_dp[1:, :-1], self.w0)
25        np.fill_diagonal(dq_dp, self.w1)
26        np.fill_diagonal(dq_dp[:-1, 1:], self.w2)
27        return grads @ dq_dp
28
29    def backward_params(self, grads):
30        n = self.p.shape[0]
31        dq_w0 = np.zeros(n)
32        dq_w1 = np.zeros(n)
33        dq_w2 = np.zeros(n)
34
35        dq_w0[1:] = self.p[:-1]
36        dq_w1[:] = self.p[:]
37        dq_w2[:-1] = self.p[1:]
38
39        return [[self.w0, grads @ dq_w0], [self.w1, grads @ dq_w1], [
self.w2, grads @ dq_w2], self.name]
40

```

3 MI 2018./2019.

Zadatak 1

Razmatramo klasifikacijski model s arhitekturom:

- konvolucijski sloj bez nadopunjavanja: 16 jezgara 5×5 , korak 1, aktivacija ReLU;
- sažimanje maksimumom 2×2 ;
- konvolucijski sloj bez nadopunjavanja: 32 jezgre 5×5 , korak 1, aktivacija ReLU;
- sažimanje maksimumom 2×2 ;
- pretvaranje u vektor;
- potpuno povezani sloj dimenzije 10 te aktivacija softmax.

Na ulaz mreže dovodimo RGB slike veličine 28×28 piksela.

Zadaci:

- Napišite dimenzije ulaznih i izlaznih tenzora za svaki sloj.
- Koliko ukupno parametara ima zadani model?
- Procijenite memorijsko zauzeće u trenutku nakon izračuna gubitka unakrsne entropije. Postupak učenja zadanog modela provodimo s veličinom mini-grupe 8. Tip podataka je 32-bitni float. Pomoć: u memorijsko zauzeće ubrojiti sve u tome trenutku pohranjene tenzore i parametre modela.
- Kolika su receptivna polja aktivacija u prvom sloju sažimanja odnosno aktivacija u potpuno povezanom sloju? Pomoć: receptivno polje računamo s obzirom na piksele ulazne slike.

Rješenje (a):

U tablici nije eksplicitno naveden stupac "Dimenzija ulaza" jer se pretpostavlja da ona za svaki sloj (osim naravno za ulaznu sliku) odgovara vrijednosti stupca "Dimenzija izlaza" prethodnog sloja. Nadalje, u tablici su odmah navedeni podatci koji su potrebni za naredne podzadatke.

Sloj	Broj parametara	Dimenzija izlaza	Receptivno polje
Slika	0	$3 \times 28 \times 28$ (*)	1×1
Conv1($C_{\text{in}} = 3$, $C_{\text{out}} = 16$, $k = 5$, $s = 1$, $p = 0$)	$16 \cdot 3 \cdot 5 \cdot 5 + 16$	$16 \times 24 \times 24$ (*)	5×5
ReLU1	0	$16 \times 24 \times 24$ (*)	5×5
MaxPool1($k = 2$, $s = 2$, $p = 0$)	0	$16 \times 12 \times 12$ (*)	6×6
Conv2($C_{\text{in}} = 16$, $C_{\text{out}} = 32$, $k = 5$, $s = 1$, $p = 0$)	$32 \cdot 16 \cdot 5 \cdot 5 + 32$	$32 \times 8 \times 8$ (*)	14×14
ReLU2	0	$32 \times 8 \times 8$ (*)	14×14
MaxPool2($k = 2$, $s = 2$, $p = 0$)	0	$32 \times 4 \times 4$ (*)	16×16
Flatten	0	512	16×16
FC	$10 \cdot 512 + 10$	10 (*)	28×28
Softmax	0	10 (*)	28×28

Rješenje (b):

Ukupno parametara: $\sum \text{Broj parametara} = 19178$

Rješenje (c):

S (*) su označene dimenzije svih izlaznih tenzora koji se pamte kao ulazni podatak u sljedećem sloju (uz napomenu da operacija **Flatten** ne zauzima dodatni prostor u memoriji jer dijeli isti memorijski blok s izlazom iz **MaxPool2**).

Zauzeće pohranjenih tenzora: $4 \text{ B} \cdot 8 \cdot \sum \text{Dimenzija izlaza (*)} = 886912 \text{ B}$

Zauzeće pohranjenih parametara: $4 \text{ B} \cdot \sum \text{Broj parametara} = 76712 \text{ B}$

Ukupno zauzeće: $886912 \text{ B} + 76712 \text{ B} = 963624 \text{ B} \approx 0.92 \text{ MiB}$

Rješenje (d):

Iščitavamo tražene vrijednosti iz tablice dane u rješenju (a) dijela zadatka. Receptivno polje aktivacija u:

- prvom sloju sažimanja: 6×6
- potpuno povezanom sloju: 28×28

Zadatak 2

Razmatramo regresijski model zadan nizom slojeva:

- potpuno povezani sloj s matricom težina $\mathbf{W}_1$ i pomakom $\mathbf{b}_1$;
- normalizacija po grupi, bez pomičnog prosjeka, s fiksnom afinom transformacijom $\gamma = 1$, $\beta = 0$ (identitet);
- aktivacija zglobnicom ReLU;
- potpuno povezani sloj s matricom težina $\mathbf{W}_2$ i pomakom b_2 .

Zadani su podatci $\mathbf{X}$ i parametri modela:

$$\mathbf{X} = \begin{bmatrix} 1 & -3 \\ 5 & 3 \\ 2 & 4 \\ 2 & 0 \end{bmatrix}, \quad \mathbf{W}_1 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{b}_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad \mathbf{W}_2 = \begin{bmatrix} 1 & -1 \end{bmatrix}, \quad b_2 = 1.$$

Zadaci:

- Provedite unaprijedni prolaz mreže u fazi treniranja za prvu mini-grupu, ako koristimo veličinu mini-grupe 2 sa slijednim uzorkovanjem.
- Provedite unaprijedni prolaz mreže u fazi eksploatacije za prvi primjer.

Rješenje (a):

Označimo prvu mini-grupu s $\mathbf{X}^{(1)}$:

$$\mathbf{X}^{(1)} = \begin{bmatrix} 1 & -3 \\ 5 & 3 \end{bmatrix}$$

Neka je $\mathbf{s}_1$ izlaz iz prvog potpuno povezanog sloja:

$$\mathbf{s}_1 = \mathbf{X}^{(1)}\mathbf{W}_1^\top + \text{broadcast}(\mathbf{b}_1^\top) = \begin{bmatrix} 1 & -3 \\ 5 & 3 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & -3 \\ 5 & 3 \end{bmatrix}$$

Dalje nam slijedi normalizacija po grupi. Prema tome, najprije moramo izračunati srednju vrijednost i varijancu svake značajke izlaza prethodnog sloja za danu mini-grupu.

Za prvu značajku (prvi stupac):

$$\begin{aligned}\mu_1 &= \frac{1+5}{2} = 3 \\ \sigma_1^2 &= \frac{1}{2} \left((1-3)^2 + (5-3)^2 \right) = \frac{1}{2}(4+4) = 4 \\ \sigma_1 &= 2\end{aligned}$$

Za drugu značajku (drugi stupac):

$$\begin{aligned}\mu_2 &= \frac{-3+3}{2} = 0 \\ \sigma_2^2 &= \frac{1}{2} \left((-3-0)^2 + (3-0)^2 \right) = \frac{1}{2}(9+9) = 9 \\ \sigma_2 &= 3\end{aligned}$$

Neka je $\mathbf{s}_1^{(1)}$ vektor prvog stupca matrice $\mathbf{s}_1$, a $\mathbf{s}_1^{(2)}$ vektor drugog stupca matrice $\mathbf{s}_1$. Računamo standardizirane vrijednosti:

$$\begin{aligned}\hat{\mathbf{s}}_1^{(1)} &= \frac{\mathbf{s}_1^{(1)} - \mu_1}{\sigma_1} = \frac{\begin{bmatrix} 1 \\ 5 \end{bmatrix} - 3}{2} = \begin{bmatrix} -1 \\ 1 \end{bmatrix} \\ \hat{\mathbf{s}}_1^{(2)} &= \frac{\mathbf{s}_1^{(2)} - \mu_2}{\sigma_2} = \frac{\begin{bmatrix} -3 \\ 3 \end{bmatrix} - 0}{3} = \begin{bmatrix} -1 \\ 1 \end{bmatrix}\end{aligned}$$

Spajanjem u matricu dobivamo standardiziranu matricu $\hat{\mathbf{s}}_1$:

$$\hat{\mathbf{s}}_1 = \begin{bmatrix} -1 & -1 \\ 1 & 1 \end{bmatrix}$$

Uključivanjem parametara, u ovom slučaju fiksne, afine transformacije ($\gamma = 1$, $\beta = 0$) dobivamo konačni izlaz sloja normalizacije $\mathbf{z}$:

$$\mathbf{z} = \gamma \cdot \hat{\mathbf{s}}_1 + \beta = 1 \cdot \begin{bmatrix} -1 & -1 \\ 1 & 1 \end{bmatrix} + 0 = \begin{bmatrix} -1 & -1 \\ 1 & 1 \end{bmatrix}$$

Neka je $\mathbf{h}$ izlaz iz ReLU aktivacije:

$$\mathbf{h} = \text{ReLU}(\mathbf{z}) = \begin{bmatrix} 0 & 0 \\ 1 & 1 \end{bmatrix}$$

Neka je $\mathbf{s}_2$ izlaz iz drugog potpuno povezanog sloja:

$$\mathbf{s}_2 = \mathbf{h}\mathbf{W}_2^\top + \text{broadcast}(b_2) = \begin{bmatrix} 0 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Rješenje (b):

Budući da se koristi normalizacija po grupi bez pomičnog prosjeka, nakon treniranja modela, a prije evaluacije pojedinih primjera, moramo kroz naučeni model propustiti sve podatke kako bismo za normalizacijski sloj izračunali potrebne statistike.

Neka je $\mathbf{s}_1$ izlaz iz prvog potpuno povezanog sloja za cijeli skup podataka $\mathbf{X}$:

$$\mathbf{s}_1 = \mathbf{X}\mathbf{W}_1^\top + \text{broadcast}(\mathbf{b}_1^\top) = \begin{bmatrix} 1 & -3 \\ 5 & 3 \\ 2 & 4 \\ 2 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & -3 \\ 5 & 3 \\ 2 & 4 \\ 2 & 0 \end{bmatrix}$$

Za prvu značajku (prvi stupac):

$$\mu_1 = \frac{1 + 5 + 2 + 2}{4} = 2.5$$

$$\sigma_1^2 = \frac{(1 - 2.5)^2 + (5 - 2.5)^2 + (2 - 2.5)^2 + (2 - 2.5)^2}{4} = \frac{(-1.5)^2 + 2.5^2 + (-0.5)^2 + (-0.5)^2}{4} = 2.25$$

$$\sigma_1 = 1.5$$

Za drugu značajku (drugi stupac):

$$\mu_2 = \frac{-3 + 3 + 4 + 0}{4} = 1$$

$$\sigma_2^2 = \frac{(-3 - 1)^2 + (3 - 1)^2 + (4 - 1)^2 + (0 - 1)^2}{4} = \frac{(-4)^2 + 2^2 + 3^2 + (-1)^2}{4} = 7.5$$

$$\sigma_2 = \frac{\sqrt{30}}{2}$$

Kroz ostatak unaprijednog prolaza možemo nastaviti razmatrati samo što se događa za prvi primjer.

$$\hat{\mathbf{s}}_1 = \begin{bmatrix} \frac{1-2.5}{1.5} \\ \frac{-3-1}{\frac{\sqrt{30}}{2}} \end{bmatrix} = \begin{bmatrix} -1 \\ -\frac{8}{\sqrt{30}} \end{bmatrix} \approx \begin{bmatrix} -1 \\ -1.4606 \end{bmatrix}$$

$$\mathbf{z} = \gamma \cdot \hat{\mathbf{s}}_1 + \beta = 1 \cdot \begin{bmatrix} -1 \\ -1.4606 \end{bmatrix} + 0 = \begin{bmatrix} -1 \\ -1.4606 \end{bmatrix}$$

$$\mathbf{h} = \text{ReLU}(\mathbf{z}) = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$s_2 = \mathbf{W}_2 \mathbf{h} + b_2 = \begin{bmatrix} 1 & -1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} + 1 = 1$$

Zadatak 3

Potrebno je projektirati program koji na ulazu dobiva binarnu matricu dimenzija $2 \times 3k$,

pri čemu je k prirodan broj. Jedan primjer takve matrice je sljedeći:

$$X = \begin{bmatrix} 1 & 1 & 1 & 0 & 1 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \end{bmatrix}$$

Gornji redak matrice sadrži niz 3-bitnih binarnih brojeva. Donji redak sadrži niz 3-bitnih jednojedinичnih kodova koji određuju način obrade. Zadaća programa je obraditi binarne brojeve u skladu s odgovarajućim jednojedinичnim kodom te odrediti rezultat u dekadskom sustavu. Značenja jednojedinичnih kodova su:

- 100 – identitet (rezultat je jednak ulazu);
- 010 – zbrajanje s tri;
- 001 – postavljanje na nulu.

Za dani ulazni primjer X , na izlazu treba proizvesti matricu $\begin{bmatrix} 10 & 0 & 5 \end{bmatrix}$.

Prvi broj u gornjem retku ulaza je 7, a ispod njega je zadana operacija 010, te je zato na izlazu broj 10. Nakon toga, u gornjem retku dolazi broj 3, a ispod njega je zadana opcija 001; zato je na izlazu 0. Zadnji broj gornjeg retka je 5, a ispod njega je zapisana opcija 100, te se zato na izlaz propušta broj 5.

Konstruirajte model koji rješava opisani problem koristeći neke od sljedećih slojeva: množenje matricom, softmax, sigmoida, ReLU, konvolucija, te sažimanje prosjekom ili maksimumom. Ako ste odabrali slojeve s težinama, napišite i vrijednosti tih težina.

Rješenje:

Ovaj zadatak se može riješiti korištenjem jednog konvolucijskog sloja i jednog aktivacijskog sloja ReLU. Koristit ćemo konvoluciju sa sljedećim parametrima:

- $C_{\text{in}} = 1$
- $C_{\text{out}} = 1$
- $k = (2, 3)$
- $s = (1, 3)$
- $p = (0, 0)$

Sada preostaje odrediti vrijednosti parametara konvolucijske jezgre $\mathbf{w}$, te eventualno pomaka b , ako nam zatreba, no za sada slobodno postavimo $b = 0$.

Nekako je jasno da bi prvi redak matrice težina koji klizi po 3-bitnim binarnim brojevima trebao sadržavati odgovarajuće potencije broja dva tako da težinskim zbrajanjem dobijemo dekadsku vrijednost. Prema tome, vrijedi:

$$\mathbf{w}_1 = \begin{bmatrix} 4 & 2 & 1 \end{bmatrix}$$

Pogledajmo sada što ćemo s težinama drugog retka matrice težina, $\mathbf{w}_2$. Budući da su 3-bitni kodovi koji određuju način obrade jednojedinичni, to znači da je za svaki od tih kodova relevantna točno jedna težina iz drugog retka matrice težina.

- Jednojedinичni kod operacije identiteta, 100, efektivno uzima u obzir težinu w_{21} . Budući da ova operacija ne želi promijeniti dekadsku vrijednost koju smo dobili na temelju prvog retka, postavljamo $w_{21} = 0$.

- Jednolinični kod operacije zbrajanja s 3, 010, efektivno uzima u obzir težinu w_{22} . Budući da ovom operacijom želimo dodati 3 dekadskoj vrijednosti koju smo dobili na temelju prvog retka, postavljamo $w_{22} = 3$.
- Jednolinični kod operacije postavljanja na 0, 001, efektivno uzima u obzir težinu w_{23} . Ovom težinom želimo da se poništi vrijednost dobivena na temelju prvog retka, no ne možemo znati unaprijed koja je to vrijednost. Prema tome, ovu težinu možemo postaviti na maksimalnu moguću dekadsku vrijednost prvog retka te se onda u sljedećem sloju pobrinuti da ta vrijednost postane 0. Dakle, postavljamo $w_{23} = -7$.

Konačno, matrica težina jednaka je:

$$\mathbf{w} = \begin{bmatrix} 4 & 2 & 1 \\ 0 & 3 & -7 \end{bmatrix}$$

Zbog posljednje opisane operacije postavljanja na nulu, nakon konvolucijskog sloja dodajemo sloj ReLU. Kako je najveća moguća vrijednost prvog retka 7, zbrajanjem s -7 maksimalna moguća vrijednost je 0, a za manje brojeve to je neka negativna vrijednost. ReLU će sve te vrijednosti ≤ 0 preslikati u 0, čime se postiže traženo rješenje.

Zadatak 4

Razmatramo sloj dubokog modela koji ulazni vektor $\mathbf{p}$ transformira u izlazni vektor $\mathbf{q}$ koristeći polinomnu transformaciju koja se može opisati jednadžbom:

$$q_i = w_0 + w_1 p_i + w_2 p_i^2 + \dots + w_n p_i^n$$

Odredite Jakobijan gubitka po ulazu i Jakobijan gubitka po parametrima ako je poznat Jakobijan gubitka po izlazu.

Napišite kod koji bi omogućio uklapanje sloja u duboki model proizvoljne složenosti. Sloj izrazite razredom koji nasljeđuje sučelje `Layer` iz prvog zadatka druge laboratorijske vježbe te implementira sljedeće metode: `forward(self, inputs)`, `backward_inputs(self, grads)`, te `backward_parameters(self, grads)`. Vaš razred treba kroz konstruktor primiti parametar n koji određuje stupanj polinoma.

Izvedba treba osigurati da dimenzionalnost izlaznog vektora bude jednaka dimenzionalnosti ulaznog vektora.

Rješenje:

Kao što je navedeno u tekstu zadatka, i -ta komponenta izlaznog vektora $\mathbf{q}$ može se opisati jednadžbom:

$$q_i = w_0 + w_1 p_i + w_2 p_i^2 + \dots + w_n p_i^n$$

Neka je ulazni vektor $\mathbf{p}$ duljine k , tada je i izlazni vektor $\mathbf{q}$ duljine k te vrijedi:

$$\begin{aligned} q_0 &= w_0 + w_1 p_0 + w_2 p_0^2 + \dots + w_n p_0^n \\ q_1 &= w_0 + w_1 p_1 + w_2 p_1^2 + \dots + w_n p_1^n \\ &\vdots \\ q_{k-1} &= w_0 + w_1 p_{k-1} + w_2 p_{k-1}^2 + \dots + w_n p_{k-1}^n \end{aligned}$$

Na temelju ovog izraza, možemo relativno lako napisati Jakobijan izlaza s obzirom na ulaz te Jakobijan izlaza s obzirom na parametre.

Jakobijan izlaza po ulazu je dijagonalna matrica:

$$\frac{\partial \mathbf{q}}{\partial \mathbf{p}} = \text{diag} \left(\begin{bmatrix} w_1 + 2w_2p_0 + \dots + nw_np_0^{n-1} \\ w_1 + 2w_2p_1 + \dots + nw_np_1^{n-1} \\ \vdots \\ w_1 + 2w_2p_{k-1} + \dots + nw_np_{k-1}^{n-1} \end{bmatrix} \right)$$

Jakobijan izlaza po parametrima je:

$$\frac{\partial \mathbf{q}}{\partial \mathbf{w}} = \begin{bmatrix} 1 & p_0 & p_0^2 & \dots & p_0^n \\ 1 & p_1 & p_1^2 & \dots & p_1^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & p_{k-1} & p_{k-1}^2 & \dots & p_{k-1}^n \end{bmatrix}$$

Poznavajući Jakobijan gubitka po izlazu ($\frac{\partial \mathcal{L}}{\partial \mathbf{q}}$), Jakobijan gubitka po ulazu i parametrima računamo lančanim pravilom:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{p}} = \frac{\partial \mathcal{L}}{\partial \mathbf{q}} \frac{\partial \mathbf{q}}{\partial \mathbf{p}}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \frac{\partial \mathcal{L}}{\partial \mathbf{q}} \frac{\partial \mathbf{q}}{\partial \mathbf{w}}$$

Implementacija sloja u jeziku Python:

```

1 import numpy as np
2
3 class MyLayer(Layer):
4     def __init__(self, n, name="MyLayer"):
5         self.has_params = True
6         self.name = name
7         self.n = n
8         self.w = np.random.randn(self.n + 1)
9         self.p = None
10
11     def forward(self, inputs):
12         self.p = inputs
13         powers = np.arange(0, self.n + 1)
14         p_powers = self.p[..., np.newaxis] ** powers
15         weighted_p_powers = self.w * p_powers
16         q = np.sum(weighted_p_powers, axis=1)
17         return q
18
19     def backward_inputs(self, grads):
20         powers = np.arange(0, self.n)
21         p_powers = self.p[..., np.newaxis] ** powers
22         weighted_p_powers = np.arange(1, self.n + 1) * self.w[1:] *
p_powers
23         dq_dp_diag = np.sum(weighted_p_powers, axis=1)
24         # element-wise mnozenje odgovara mnozenju s dijagonalnom
matricom
25         return grads * dq_dp_diag
26
27     def backward_parameters(self, grads):
28         powers = np.arange(0, self.n + 1)
29         dq_dw = self.p[..., np.newaxis] ** powers
30         return [[self.w, grads @ dq_dw], self.name]
31

```


Zadatak 5

Razmatramo klasifikacijski konvolucijski model koji je prilagođen obradi vektora s pet elemenata. Arhitektura modela je:

- konvolucijski sloj bez nadopunjavanja, jezgra $w_1 = \begin{bmatrix} -0.25 & 0.5 & 0.25 \end{bmatrix}$, pomak $b_1 = -1/4$, korak 1, aktivacija ReLU;
- potpuno povezani sloj, jezgra $w_2 = \begin{bmatrix} -1 & 1 & 1 \end{bmatrix}$, pomak $b_2 = -1$, sigmoidna aktivacija.

Zadatci:

- Odredite dimenzije svih aktivacijskih tenzora te broj parametara modela;
- odredite predikciju modela za podatak $x = \begin{bmatrix} 1 & 0 & 6 & 1 & 1 \end{bmatrix}^\top$ te odredite gubitak (negativnu log izglednost) ako je točna izglednost podatka jednaka nuli;
- odredite gradijente gubitka s obzirom na sve parametre modela;
- odredite indeks dimenzije ulaznog podatka čije povećanje maksimalno smanjuje gubitak (BONUS).

Rješenje (a):

Sloj	Broj parametara	Dimenzija izlaza
Ulaz	0	5
Conv1d($C_{\text{in}} = 1$, $C_{\text{out}} = 1$, $k = 3$, $s = 1$, $p = 0$)	$3 + 1 = 4$	3
ReLU	0	3
FC($D_{\text{in}} = 3$, $D_{\text{out}} = 1$)	$1 \cdot 3 + 1 = 4$	1
Sigmoida	0	1

Dimenzije aktivacijskih tenzora čitamo iz posljednjeg stupca tablice.

Ukupan broj parametara: $\sum \text{Broj parametara} = 8$

Rješenje (b):

Zadani su:

$$x = \begin{bmatrix} 1 & 0 & 6 & 1 & 1 \end{bmatrix}^\top, \quad y = 0$$

Neka je g izlaz iz konvolucije:

$$g = w_1 \star x + \text{broadcast}(b_1) = \begin{bmatrix} 1.25 \\ 3.25 \\ -0.75 \end{bmatrix} + \begin{bmatrix} -0.25 \\ -0.25 \\ -0.25 \end{bmatrix} = \begin{bmatrix} 1 \\ 3 \\ -1 \end{bmatrix}$$

Neka je h izlaz iz ReLU aktivacije:

$$h = \text{ReLU}(g) = \begin{bmatrix} 1 & 3 & 0 \end{bmatrix}^\top$$

Neka je s izlaz iz potpuno povezanog sloja:

$$s = w_2 h + b_2 = \begin{bmatrix} -1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 3 \\ 0 \end{bmatrix} - 1 = -1 + 3 - 1 = 1$$

Neka je $\hat{y}$ izlaz iz sigmoide:

$$\hat{y} = \sigma(s) = \sigma(1) = 1/(1 + e^{-1}) \approx 0.7311$$

Gubitak negativne log-izglednosti iznosi:

$$\mathcal{L} = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y}) = -\log(1 - \hat{y}) \approx 1.3134$$

Rješenje (c):

Postupak unatragnog prolaza započinjemo računanjem gradijenta gubitka po ulazu u sigmoиду s :

$$\frac{\partial \mathcal{L}}{\partial s} = \hat{y} - y \approx 0.7311$$

Zatim, po pravilu ulančavanja, računamo gradijente gubitka po parametrima potpuno povezanog sloja w_2 i b_2 :

$$\frac{\partial \mathcal{L}}{\partial w_2} = \frac{\partial \mathcal{L}}{\partial s} \frac{\partial s}{\partial w_2} = \frac{\partial \mathcal{L}}{\partial s} h^\top \approx 0.7311 \begin{bmatrix} 1 & 3 & 0 \end{bmatrix}$$

$$\frac{\partial \mathcal{L}}{\partial b_2} = \frac{\partial \mathcal{L}}{\partial s} \frac{\partial s}{\partial b_2} = \frac{\partial \mathcal{L}}{\partial s} \cdot 1 \approx 0.7311$$

Nadalje, računamo gradijent gubitka po izlazu skrivenog sloja h :

$$\frac{\partial \mathcal{L}}{\partial h} = \frac{\partial \mathcal{L}}{\partial s} \frac{\partial s}{\partial h} = \frac{\partial \mathcal{L}}{\partial s} w_2 \approx 0.7311 \begin{bmatrix} -1 & 1 & 1 \end{bmatrix}$$

Slijedi gradijent gubitka po ulazu u aktivacijsku funkciju g :

$$\frac{\partial \mathcal{L}}{\partial g} = \frac{\partial \mathcal{L}}{\partial h} \frac{\partial h}{\partial g} = \frac{\partial \mathcal{L}}{\partial h} \odot \llbracket g > 0 \rrbracket^\top \approx 0.7311 \begin{bmatrix} -1 & 1 & 1 \end{bmatrix} \odot \begin{bmatrix} 1 & 1 & 0 \end{bmatrix} = 0.7311 \begin{bmatrix} -1 & 1 & 0 \end{bmatrix}$$

Konačno, računamo gradijente gubitka po parametrima konvolucijskog sloja w_1 i b_1 :

$$\frac{\partial \mathcal{L}}{\partial w_1} = \frac{\partial \mathcal{L}}{\partial g} \star x \approx 0.7311 \begin{bmatrix} -1 & 1 & 0 \end{bmatrix} \star \begin{bmatrix} 1 & 0 & 6 & 1 & 1 \end{bmatrix} = 0.7311 \begin{bmatrix} -1 & 6 & -5 \end{bmatrix}$$

$$\frac{\partial \mathcal{L}}{\partial b_1} = \sum_i \frac{\partial \mathcal{L}}{\partial g_i} = 0$$

Rješenje (d):

Neka je k veličina jezgre w_1 , tj. $k = 3$.

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial x} &= \text{flip}(w_1) \star \text{pad} \left(\frac{\partial \mathcal{L}}{\partial g}, k - 1 \right) \\ &\approx \begin{bmatrix} 0.25 & 0.5 & -0.25 \end{bmatrix} \star \left(0.7311 \begin{bmatrix} 0 & 0 & -1 & 1 & 0 & 0 & 0 \end{bmatrix} \right) \\ &= 0.7311 \begin{bmatrix} 0.25 & -0.75 & 0.25 & 0.25 & 0 \end{bmatrix} \end{aligned}$$

Vektor $\partial\mathcal{L}/\partial x$ daje smjer u kojem pomak vektora x povećava gubitak $\mathcal{L}$. Gubitak se smanjuje u suprotnom smjeru:

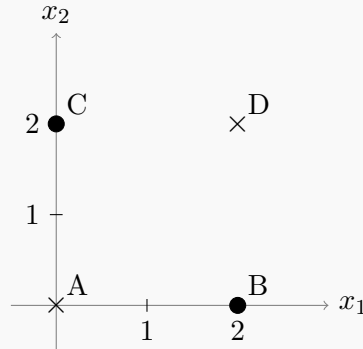
$$-\frac{\partial\mathcal{L}}{\partial x} \approx 0.7311 \begin{bmatrix} -0.25 & 0.75 & -0.25 & -0.25 & 0 \end{bmatrix}$$

Najpozitivnija vrijednost dobivenog vektora je $0.7311 \cdot 0.75$ koja se, ako indeksiranje započinje od 0, nalazi na indeksu 1 pa je upravo to indeks dimenzije ulaznog podatka čije povećanje maksimalno smanjuje gubitak.

4 MI 2021./2022.

Zadatak 1

Dizajnirajte dvoslojni potpunopovezani model koji rješava problem sa slike. Zadani problem sadrži četiri podatka koje je potrebno klasificirati u dva razreda: kružić i križić. Aktivacijska funkcija skrivenog sloja neka bude zglobnica, a izlaznog sloja sigmoida. Napišite sve parametre modela i pokažite da Vaš model ispravno klasificira sve podatke pod pretpostavkom da granicu klasa definira vjerojatnost od 50%. Pomoć: razmotrite duboki model nad varijablom $z = x_2 - x_1$.



Rješenje:

Zadani problem predstavlja varijaciju klasičnog XOR problema koji nije linearno odvojiv, pa ga nije moguće riješiti jednoslojnim modelom. Iskoristimo ponuđenu pomoć i promotrimo ponašanje varijable $z = x_2 - x_1$ za zadane ulazne podatke, uz pretpostavku da križić označava klasu $y = 0$, a kružić klasom $y = 1$:

Točka	(x_1, x_2)	Klasa y	$z = x_2 - x_1$
A	(0, 0)	0	0
B	(2, 0)	1	-2
C	(0, 2)	1	2
D	(2, 2)	0	0

Uočimo da smo preko varijable z podatke iz ulaznog prostora preslikali u jednodimenzionalni skriveni prostor. Ako razmislimo što će se dogoditi s ovim podacima nakon prolaska kroz nelinearnu aktivaciju ReLU, biti će nam jasno da ćemo izgubiti informacije potrebne za diskriminaciju. Naime, izlaz iz ReLU sloja za točku C iznosio bi $\max(0, 2) = 2$, dok bi za točke A , B i D iznosio 0. Budući da točka B pripada klasi 1, a točke A i D klasi 0, u skrivenom prostoru te se točke više ne mogu diskriminirati.

Shodno navedenom, nužno je uvesti barem još jednu skrivenu varijablu. Proširimo skriveni sloj na dvije dimenzije i definirajmo linearnu transformaciju izlaza iz skrivenog sloja kao:

$$z = W_1 x + b_1$$

gdje biramo matricu težina W_1 i vektor pomaka b_1 tako da razmatramo simetrične razlike

$$z = \begin{bmatrix} x_2 - x_1 \\ x_1 - x_2 \end{bmatrix};$$

$$W_1 = \begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix}, \quad b_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Nakon primjene aktivacijske funkcije ReLU ($h = \max(0, z)$), dobivamo sljedeće dvodimenzionalne reprezentacije u skrivenom prostoru:

Točka	$x = [x_1, x_2]^\top$	Klasa y	$z = [x_2 - x_1, x_1 - x_2]^\top$	$h = \text{ReLU}(z)$
A	$[0, 0]^\top$	0	$[0, 0]^\top$	$[0, 0]^\top$
B	$[2, 0]^\top$	1	$[-2, 2]^\top$	$[0, 2]^\top$
C	$[0, 2]^\top$	1	$[2, -2]^\top$	$[2, 0]^\top$
D	$[2, 2]^\top$	0	$[0, 0]^\top$	$[0, 0]^\top$

Na temelju varijable h uočavamo da u skrivenom prostoru podaci postaju linearno odvojivi. Za klasu 0 obje komponente skrivenog stanja poprimaju vrijednost 0, dok za klasu 1 barem jedna komponenta ima pozitivnu vrijednost.

Sada bismo željeli svaku točku iz skrivenog prostora preslikati u skalar koji bi predstavljao ulaz za sigmoidu pa stoga razmotrimo izlazni sloj modela. Potpuno povezani dio izlaznog sloja definiran je kao $s = w_2^\top h + b_2$, a slijedi ga sigmoidalna aktivacijska funkcija na temelju čije vrijednosti se provodi klasifikacija, $p = \sigma(s) = 1/(1 + e^{-s})$. Budući da je klasifikacijski prag postavljen na vjerojatnost od 50%, a $\sigma(0) = 0.5$, tražimo parametre za koje vrijedi:

$$s < 0 \iff y = 0 \quad \text{i} \quad s > 0 \iff y = 1$$

Pogledom na posljednji stupac tablice iznad, nije jako teško vidjeti da možemo odabrati sljedeće parametre:

$$w_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad b_2 = -1$$

Pokažimo da predloženi model ispravno klasificira sve zadane podatke izračunom unaprijednog prolaska:

Točka	h	Stvarna klasa y	$s = w_2^\top h + b_2$	$p = \sigma(s)$	$\hat{y} = \llbracket p \geq 0.5 \rrbracket$
A	$[0, 0]^\top$	0	-1	$\sigma(-1) \approx 0.2689$	0
B	$[0, 2]^\top$	1	1	$\sigma(1) \approx 0.7311$	1
C	$[2, 0]^\top$	1	1	$\sigma(1) \approx 0.7311$	1
D	$[0, 0]^\top$	0	-1	$\sigma(-1) \approx 0.2689$	0

Iz završne tablice vidljivo je da za sve točke vrijedi $\hat{y} = y$, čime je dokazano da model s predloženim parametrima ispravno klasificira sve podatke.

Zadatak 2

Napišite izravnu implementaciju funkcije koja obavlja unaprijedni prolaz i računa gubitak za višerazrednu logističku regresiju koristeći biblioteku `numpy`. Funkcija na ulazu prima jedan podatak x dimenzija $(D, 1)$, prirodni broj y iz intervala $[1, C]$ koji označava točni razred, te ostale potrebne parametre odgovarajućih dimenzija (kojih?). Identificirajte dijelove vašeg koda u kojima može doći do numeričkih pogrešaka. Napišite robusnu implementaciju koja rješava navedene probleme. Uputa: obratite pažnju na mogući preljev u funkcijama `exp` i `log`.

Rješenje:

Ostali potrebni parametri modela su matrica težina i vektor pomaka:

- Matrica težina W dimenzija $C \times D$
- Vektor pomaka b dimenzija $C \times 1$

Gubitak negativne log-izglednosti za višerazrednu logističku regresiju definiran je kao:

$$\mathcal{L} = -\log(P(\hat{y} = y)) = -\log(\text{softmax}(s)_y) = -\log\left(\frac{\exp(s_y)}{\sum_{k=1}^C \exp(s_k)}\right) = -s_y + \log\left(\sum_{k=1}^C \exp(s_k)\right)$$

gdje je $s = Wx + b$ vektor logita dimenzija $C \times 1$.

Izravna implementacija ovog izraza može dovesti do numeričkih pogrešaka, konkretno do *overflow*-a pri računanju eksponencijalne funkcije $\exp(s_k)$ ako su logiti s_k veliki pozitivni brojevi, što će potom onemogućiti ispravno računanje logaritma i samog gubitka.

Kako bismo to riješili, možemo iskoristiti svojstvo invarijantnosti softmaxa na translaciju, tj. dodavanje konstante svim logitima ulaza neće promijeniti izlaz. Definiramo $\hat{s} = s - \max(s)$ te dobivamo konačni izraz:

$$\mathcal{L} = -\hat{s}_y + \log\left(\sum_{k=1}^C \exp(\hat{s}_k)\right)$$

Dobiveni izraz je numerički stabilan i naziva se *Log-Sum-Exp* trikom.

Implementacija u programskom jeziku Python koristeći biblioteku `numpy` (uz napomenu da su indeksi polja u Pythonu od 0, pa zadanu oznaku razreda $y \in [1, C]$ moramo umanjiti za 1):

```

1 import numpy as np
2
3 def loss(x, y, W, b):
4     # x: (D, 1), W: (C, D), b: (C, 1)
5     # y: tocan razred iz [1, C]
6
7     # racunanje logita
8     s = W @ x + b
9
10    # stabilizacija oduzimanjem maksimuma (Log-Sum-Exp trik)
11    s_hat = s - np.max(s)
12
13    # racunanje gubitka
14    s_hat_exp = np.exp(s_hat)
15    log_sum_exp = np.log(np.sum(s_hat_exp))
16
17    loss = -s_hat[y - 1, 0] + log_sum_exp
18
19    return loss

```

Zadatak 3

Zadana je sljedeća inicijalizacija modula, te odgovarajući unaprijedni prolaz. Na ulazu se nalazi jedna RGB slika dimenzija 32×32 .

```

conv1 = nn.Conv2d(__, 16, kernel_size=7, stride=1, padding=0)
conv2 = nn.Conv2d(__, 24, kernel_size=3, stride=1, padding=1)
maxpool1 = nn.MaxPool2d(kernel_size=__, stride=__, padding=0)
conv3 = nn.Conv2d(__, 32, kernel_size=7, stride=1, padding=0)
conv_skip = nn.Conv2d(__, __, kernel_size=3, stride=__, padding=__)
avgpool1 = nn.AvgPool2d(kernel_size=2, stride=2, padding=0)
fc1 = nn.Linear(__, 256)
fc2 = nn.Linear(__, 4)

```

```
# img.shape == (1, C, H, W)
x = torch.relu(conv1(img))
x = torch.relu(conv2(x))
x = maxpool1(x)
assert(x.shape[-1] == 22)
assert(x.shape[-2] == 22)
x = torch.relu(conv3(x))
skip = torch.relu(conv_skip(img))
x = x + skip
x = avgpool1(x)
x = x.view(1, -1)
x = torch.relu(fc1(x))
x = fc2(x)
```

Zadaci:

- Dopunite inicijalizaciju zadanih slojeva hiperparametrima koji nedostaju, tako da se unaprijedni prolaz može izvršiti bez grešaka.
- Napišite broj parametara svakoga sloja, te dimenzije tenzora x nakon izvršene svake linije unaprijednog prolaza. Podrazumijevana vrijednost argumenta `bias` u konvolucijskim i potpuno povezanim slojevima je `True`.
- Odredite receptivno polje izlaza sloja `maxpool1`.

Rješenje (a, b, c):

Prolazit ćemo liniju po liniju unaprijednog prolaza te graditi tablicu sa svim relevantnim informacijama o slojevima.

Na temelju linije `img.shape == (1, C, H, W)` i na temelju toga da na ulaz dolazi jedna RGB slika dimenzija 32×32 , vrijedi:

Sloj	Broj parametara	Dimenzije izlaza	Receptivno polje
Slika	0	$3 \times 32 \times 32$	1×1

Dalje, razmatramo liniju `x = torch.relu(conv1(img))` te definiciju `conv1 = nn.Conv2d(..., 16, kernel_size=7, stride=1, padding=0)`. Dakle, nepoznat je broj ulaznih kanala ovog konvolucijskog sloja. No, taj podatak lako saznamo tako da pogledamo broj izlaznih kanala prethodnog sloja. Prema tome, vrijedi:

Sloj	Broj parametara	Dimenzije izlaza	Receptivno polje
Slika	0	$3 \times 32 \times 32$	1×1
Conv1 ($c_{in} = 3, c_{out} = 16, k = 7, s = 1, p = 0$)	$3 \cdot 16 \cdot 7 \cdot 7 + 16$	$16 \times 26 \times 26$	7×7
ReLU1	0	$16 \times 26 \times 26$	7×7

Dalje, razmatramo liniju `x = torch.relu(conv2(x))` te definiciju `conv2 = nn.Conv2d(..., 24, kernel_size=3, stride=1, padding=1)`. Dakle, nepoznat je broj ulaznih kanala ovog konvolucijskog sloja. No, taj podatak lako saznamo tako da pogledamo broj izlaznih kanala prethodnog sloja. Prema tome, vrijedi:

Sloj	Broj parametara	Dimenzije izlaza	Receptivno polje
Slika	0	$3 \times 32 \times 32$	1×1
Conv1 ($c_{in} = 3, c_{out} = 16, k = 7, s = 1, p = 0$)	$16 \cdot 3 \cdot 7 \cdot 7 + 16$	$16 \times 26 \times 26$	7×7
ReLU1	0	$16 \times 26 \times 26$	7×7
Conv2 ($c_{in} = 16, c_{out} = 24, k = 3, s = 1, p = 1$)	$24 \cdot 16 \cdot 3 \cdot 3 + 24$	$24 \times 26 \times 26$	9×9
ReLU2	0	$24 \times 26 \times 26$	9×9

Dalje, razmatramo liniju `x = maxpool1(x)` te definiciju `maxpool1 = nn.MaxPool2d(kernel_size=__, stride=__, padding=0)`. Dakle, nepoznata je veličina jezgre i korak ovog sloja sažimanja maksimumom. U ovom slučaju ne možemo ništa zaključiti na temelju slojeva koje već imamo zapisane u tablici pa moramo pogledati što dalje slijedi u kodu.

U kodu slijede linije `assert(x.shape[-1] == 22)` te `assert(x.shape[-2] == 22)`. Iz ovoga možemo zaključiti da sloj sažimanja mora smanjiti prostornu dimenziju prethodnog sloja, 26×26 , na dimenziju 22×22 . Rješavamo jednadžbu:

$$\begin{aligned}
22 &= \lfloor (26 + 2 \cdot p - k) / s \rfloor + 1 \\
21 &= \lfloor (26 - k) / s \rfloor \\
21 &\leq (26 - k) / s < 22 \\
21 \cdot s &\leq 26 - k < 22 \cdot s
\end{aligned}$$

Jasno je da za bilo koji $s > 1$ ovaj sustav ne može biti zadovoljen, npr. za $s = 2$ dobiva se:

$$42 \leq 26 - k < 44$$

pri čemu je nemoguće da vrijedi $42 \leq 26 - k$.

Stoga zaključujemo da mora vrijediti $s = 1$. Sada imamo:

$$\begin{aligned}
21 &\leq 26 - k < 22 \\
-5 &\leq -k < -4 \\
4 &< k \leq 5
\end{aligned}$$

Dakle, mora vrijediti $k = 5$.

Nadopunjavamo tablicu novim retkom:

Sloj	Broj parametara	Dimenzije izlaza	Receptivno polje
Slika	0	$3 \times 32 \times 32$	1×1
Conv1 ($c_{in} = 3, c_{out} = 16, k = 7, s = 1, p = 0$)	$16 \cdot 3 \cdot 7 \cdot 7 + 16$	$16 \times 26 \times 26$	7×7
ReLU1	0	$16 \times 26 \times 26$	7×7
Conv2 ($c_{in} = 16, c_{out} = 24, k = 3, s = 1, p = 1$)	$24 \cdot 16 \cdot 3 \cdot 3 + 24$	$24 \times 26 \times 26$	9×9
ReLU2	0	$24 \times 26 \times 26$	9×9
MaxPool1 ($k = 5, s = 1, p = 0$)	0	$24 \times 22 \times 22$	13×13

Dalje, razmatramo liniju `x = torch.relu(conv3(x))` te definiciju `conv3 = nn.Conv2d(__, 32, kernel_size=7, stride=1, padding=0)`. Dakle, nepoznat je broj ulaznih kanala ovog konvolucijskog sloja. No, taj podatak lako saznamo tako da pogledamo broj izlaznih kanala prethodnog sloja. Prema tome, vrijedi:

Sloj	Broj parametara	Dimenzije izlaza	Receptivno polje
Slika	0	$3 \times 32 \times 32$	1×1
Conv1 ($c_{in} = 3, c_{out} = 16, k = 7, s = 1, p = 0$)	$16 \cdot 3 \cdot 7 \cdot 7 + 16$	$16 \times 26 \times 26$	7×7
ReLU1	0	$16 \times 26 \times 26$	7×7
Conv2 ($c_{in} = 16, c_{out} = 24, k = 3, s = 1, p = 1$)	$24 \cdot 16 \cdot 3 \cdot 3 + 24$	$24 \times 26 \times 26$	9×9
ReLU2	0	$24 \times 26 \times 26$	9×9
MaxPool1 ($k = 5, s = 1, p = 0$)	0	$24 \times 22 \times 22$	13×13
Conv3 ($c_{in} = 24, c_{out} = 32, k = 7, s = 1, p = 0$)	$32 \cdot 24 \cdot 7 \cdot 7 + 32$	$32 \times 16 \times 16$	19×19
ReLU3	0	$32 \times 16 \times 16$	19×19

Dalje, razmatramo liniju `skip = torch.relu(conv_skip(img))` te definiciju `conv_skip = nn.Conv2d(__, __, kernel_size=3, stride=__, padding=__)`. Ovdje najprije moramo primijetiti da ulaz u sloj nije izlaz ReLU sloja kojeg smo posljednjeg napisali u tablici, već je to sama ulazna slika. Nepoznati parametri ovdje navedenog konvolucijskog sloja su broj ulaznih kanala, broj izlaznih kanala, korak i padding. Lako je odrediti broj ulaznih kanala jer znamo da na ulaz dolazi trokanalna slika. Ostale nepoznanice ne možemo odrediti na temelju trenutno poznatih informacija, već je potrebno pogledati što se događa dalje u kodu.

Slijedi linija `x = x + skip`. Ona nam govori da se izlaz iz skip zbraja s izlazom iz posljednjeg ReLU sloja, što znači da ta dva tenzora moraju biti istog oblika. Iz toga zaključujemo da broj izlaznih kanala za `conv_skip` mora biti 32, te također zaključujemo da `conv_skip` mora smanjiti prostornu dimenzionalnost s 32×32 na 16×16 . Rješavamo jednadžbu:

$$\begin{aligned}
16 &= \lfloor (32 + 2 \cdot p - k) / s \rfloor + 1 \\
15 &= \lfloor (29 + 2 \cdot p) / s \rfloor \\
15 &\leq (29 + 2 \cdot p) / s < 16 \\
15 \cdot s &\leq 29 + 2 \cdot p < 16 \cdot s
\end{aligned}$$

Postavimo li $p = 1$, tada imamo:

$$15 \cdot s \leq 31 < 16 \cdot s$$

Ovaj sustav nejednakosti je sada zadovoljen za $s = 2$:

$$30 \leq 31 < 32$$

Odredili smo sve nepoznate parametre od `conv_skip`. Budući da je ulaz u ovaj sloj sama ulazna slika, napravimo si pomoćnu tablicu pa ćemo tek onda vidjeti kako “pametno” informacije o ovom sloju ugraditi u “glavnu” tablicu.

Sloj	Broj parametara	Dimenzije izlaza	Receptivno polje
Slika	0	$3 \times 32 \times 32$	1×1
ConvSkip ($c_{in} = 3, c_{out} = 32, k = 3, s = 2, p = 1$)	$32 \cdot 3 \cdot 3 \cdot 3 + 32$	$32 \times 16 \times 16$	3×3
ReLU3Skip	0	$32 \times 16 \times 16$	3×3

Sada ćemo ReLU3Skip pribrojiti posljednjem retku “glavne” tablice. Ovdje se postavlja pitanje koja će biti vrijednost receptivnog polja budući da zbrajamo izlaze slojeva čija su receptivna polja različita, odnosno 19×19 i 3×3 . Pokušajmo to objasniti na sljedeći način. Odaberemo li proizvoljan piksel izlaza iz ReLU3 te odgovarajući piksel iz ReLU3Skip, onda piksel iz ReLU3

vidi područje od 19×19 piksela ulazne slike, a ReLUSkip područje od 3×3 piksela ulazne slike. Navedeno područje od 3×3 piksela je već sadržano u navedenom području od 19×19 piksela, a to znači da se zbrajanjem dobiva njihova unija, odnosno područje od 19×19 piksela.

Novo stanje tablice je sljedeće:

Sloj	Broj parametara	Dimenzije izlaza	Receptivno polje
Slika	0	$3 \times 32 \times 32$	1×1
Conv1 ($c_{in} = 3, c_{out} = 16, k = 7, s = 1, p = 0$)	$16 \cdot 3 \cdot 7 \cdot 7 + 16$	$16 \times 26 \times 26$	7×7
ReLU1	0	$16 \times 26 \times 26$	7×7
Conv2 ($c_{in} = 16, c_{out} = 24, k = 3, s = 1, p = 1$)	$24 \cdot 16 \cdot 3 \cdot 3 + 24$	$24 \times 26 \times 26$	9×9
ReLU2	0	$24 \times 26 \times 26$	9×9
MaxPool1 ($k = 5, s = 1, p = 0$)	0	$24 \times 22 \times 22$	13×13
Conv3 ($c_{in} = 24, c_{out} = 32, k = 7, s = 1, p = 0$)	$32 \cdot 24 \cdot 7 \cdot 7 + 32$	$32 \times 16 \times 16$	19×19
ReLU3 + ReLUSkip	0	$32 \times 16 \times 16$	19×19

Ovdje bi bilo dobro napomenuti još jednu stvar. Budući da se tijekom cijelog ovog postupka koristi formula (1) za računanje receptivnog polja izlaza sloja, a u kojoj se razmatraju stride-ovi prethodnih slojeva, bitno je razumjeti uzimamo li sada u narednim koracima stride sloja Conv3 ili ConvSkip. Odgovor je taj da se u obzir uzima stride sloja koji ima veće receptivno polje, a zaključili smo da je to Conv3.

Dalje, razmatramo liniju `x = avgpool1(x)` te definiciju `avgpool1 = nn.AvgPool2d(kernel_size=2, stride=2, padding=0)`. Za ovaj sloj nema nepoznatih parametara, novo stanje tablice dano je u nastavku.

Sloj	Broj parametara	Dimenzije izlaza	Receptivno polje
Slika	0	$3 \times 32 \times 32$	1×1
Conv1 ($c_{in} = 3, c_{out} = 16, k = 7, s = 1, p = 0$)	$16 \cdot 3 \cdot 7 \cdot 7 + 16$	$16 \times 26 \times 26$	7×7
ReLU1	0	$16 \times 26 \times 26$	7×7
Conv2 ($c_{in} = 16, c_{out} = 24, k = 3, s = 1, p = 1$)	$24 \cdot 16 \cdot 3 \cdot 3 + 24$	$24 \times 26 \times 26$	9×9
ReLU2	0	$24 \times 26 \times 26$	9×9
MaxPool1 ($k = 5, s = 1, p = 0$)	0	$24 \times 22 \times 22$	13×13
Conv3 ($c_{in} = 24, c_{out} = 32, k = 7, s = 1, p = 0$)	$32 \cdot 24 \cdot 7 \cdot 7 + 32$	$32 \times 16 \times 16$	19×19
ReLU3 + ReLUSkip	0	$32 \times 16 \times 16$	19×19
AvgPool1 ($k = 2, s = 2, p = 0$)	0	$32 \times 8 \times 8$	20×20

Dalje, razmatramo liniju `x = x.view(1, -1)`, a koja će u ovom slučaju “ispeglati” 3D tenzor u vektor redak, no mi u tablici možemo jednostavno pisati veličinu kroz jednu dimenziju. Novo stanje tablice dano je u nastavku.

Sloj	Broj parametara	Dimenzije izlaza	Receptivno polje
Slika	0	$3 \times 32 \times 32$	1×1
Conv1 ($c_{in} = 3, c_{out} = 16, k = 7, s = 1, p = 0$)	$16 \cdot 3 \cdot 7 \cdot 7 + 16$	$16 \times 26 \times 26$	7×7
ReLU1	0	$16 \times 26 \times 26$	7×7
Conv2 ($c_{in} = 16, c_{out} = 24, k = 3, s = 1, p = 1$)	$24 \cdot 16 \cdot 3 \cdot 3 + 24$	$24 \times 26 \times 26$	9×9
ReLU2	0	$24 \times 26 \times 26$	9×9
MaxPool1 ($k = 5, s = 1, p = 0$)	0	$24 \times 22 \times 22$	13×13
Conv3 ($c_{in} = 24, c_{out} = 32, k = 7, s = 1, p = 0$)	$32 \cdot 24 \cdot 7 \cdot 7 + 32$	$32 \times 16 \times 16$	19×19
ReLU3 + ReLUSkip	0	$32 \times 16 \times 16$	19×19
AvgPool1 ($k = 2, s = 2, p = 0$)	0	$32 \times 8 \times 8$	20×20
Flatten	0	2048	20×20

Zadnje dvije linije koda i odgovarajuće definicije možemo razmotriti zajedno. Dakle imamo liniju `x = torch.relu(fc1(x))` te definiciju `fc1 = nn.Linear(..., 256)` te liniju `x = fc2(x)` te definiciju `fc2 = nn.Linear(..., 4)`. I za `fc1` i za `fc2` je nepoznat parametar veličine ulaza. Za `fc1` on odgovara veličini izlaza prethodnog Flatten sloja, a za `fc2` on odgovara veličini izlaza od `fc1`.

Konačna tablica:

Sloj	Broj parametara	Dimenzije izlaza	Receptivno polje
Slika	0	$3 \times 32 \times 32$	1×1
Conv1 ($c_{in} = 3, c_{out} = 16, k = 7, s = 1, p = 0$)	$16 \cdot 3 \cdot 7 \cdot 7 + 16$	$16 \times 26 \times 26$	7×7
ReLU1	0	$16 \times 26 \times 26$	7×7
Conv2 ($c_{in} = 16, c_{out} = 24, k = 3, s = 1, p = 1$)	$24 \cdot 16 \cdot 3 \cdot 3 + 24$	$24 \times 26 \times 26$	9×9
ReLU2	0	$24 \times 26 \times 26$	9×9
MaxPool1 ($k = 5, s = 1, p = 0$)	0	$24 \times 22 \times 22$	13×13 (c)
Conv3 ($c_{in} = 24, c_{out} = 32, k = 7, s = 1, p = 0$)	$32 \cdot 24 \cdot 7 \cdot 7 + 32$	$32 \times 16 \times 16$	19×19
ReLU3 + ReLUSkip	0	$32 \times 16 \times 16$	19×19
AvgPool1 ($k = 2, s = 2, p = 0$)	0	$32 \times 8 \times 8$	20×20
Flatten	0	2048	20×20
FC1 ($d_{in} = 2048, d_{out} = 256$)	$256 \cdot 2048 + 256$	256	32×32
ReLU4	0	256	32×32
FC2 ($d_{in} = 256, d_{out} = 4$)	$4 \cdot 256 + 4$	4	32×32

Odgovori na sve podzadatke su dani ovom tablicom.

Zadatak 4

Razmatramo klasifikacijski model u tri klase. Učenje provodimo na podacima koji su označeni indeksima $y \in \{0, 1, 2, 3\}$. Indeksi 0, 1, 2 označavaju razrede unutar distribucijskih podataka. Na takve podatke primjenjujemo gubitak negativne log izglednosti $\mathcal{L}_{NLL}$. Indeks 3 označava tzv. negativne podatke za učenje. To su izvan distribucijski podatci koji ne pripadaju niti jednom od 3 poznata razreda. Na takve podatke primjenjujemo

gubitak KL divergencije $\mathcal{L}_{KL}$. Takav je model opisan sljedećim jednadžbama:

$$\mathbf{s} = \mathbf{W} \cdot \mathbf{x} + \mathbf{b}$$

$$\mathbf{p} = \text{softmax}(\mathbf{s})$$

$$\mathcal{L} = \begin{cases} \mathcal{L}_{NLL}(p_y), & y \in \{0, 1, 2\} \\ \mathcal{L}_{KL}(\mathbf{p}, \text{uniform}), & y = 3; \end{cases} \quad \mathcal{L}_{KL}(\mathbf{p}, \mathbf{q}) = -\sum_i q_i \log\left(\frac{p_i}{q_i}\right)$$

Zadaci:

- Izvedite gradijente KL divergencije po logitima $\frac{\partial \mathcal{L}_{KL}}{\partial \mathbf{s}}$ za zadanu proizvoljnu distribuciju $\mathbf{q}$.
- Pojasnite odnos između gubitka negativne log izglednosti i gubitka unakrsne entropije.
- Pojasnite odnos između gubitka unakrsne entropije i KL divergencije.
- Pojednostavite funkciju gubitka koristeći prethodna razmatranja.
- Izračunajte gubitak i gradijente po parametrima u podacima $\mathbf{x}_1 = \begin{bmatrix} -5 \\ 2 \end{bmatrix}$, $y_1 = 0$ i

$$\mathbf{x}_2 = \begin{bmatrix} 6 \\ 8 \end{bmatrix}, y_2 = 3, \text{ ako su zadani parametri } \mathbf{W} = \begin{bmatrix} 1 & 1 & -1 \\ -1 & 1 & 1 \end{bmatrix}^T \text{ i } \mathbf{b} = 0.$$

Rješenje (a):

Najprije ćemo raspisati izraz za gubitak KL divergencije:

$$\begin{aligned} \mathcal{L}_{KL}(\mathbf{p}, \mathbf{q}) &= -\sum_i q_i \log\left(\frac{p_i}{q_i}\right) \\ &= -\sum_i q_i (\log(p_i) - \log(q_i)) \\ &= -\sum_i q_i \log(p_i) + \sum_i q_i \log(q_i) \\ &= -\sum_i q_i \log(\text{softmax}(\mathbf{s})_i) + \sum_i q_i \log(q_i) \\ &= -\sum_i q_i \log\left(\frac{e^{s_i}}{\sum_k e^{s_k}}\right) + \sum_i q_i \log(q_i) \\ &= \sum_i q_i \log\left(\frac{\sum_k e^{s_k}}{e^{s_i}}\right) + \sum_i q_i \log(q_i) \\ &= \sum_i q_i \left(\log\left(\sum_k e^{s_k}\right) - s_i \right) + \sum_i q_i \log(q_i) \\ &= \log\left(\sum_k e^{s_k}\right) \sum_i q_i - \sum_i q_i s_i + \sum_i q_i \log(q_i) \end{aligned}$$

Budući da je $\mathbf{q}$ distribucija vjerojatnosti, vrijedi $\sum_i q_i = 1$, pa izraz možemo pojednostaviti:

$$\mathcal{L}_{KL}(\mathbf{p}, \mathbf{q}) = \log\left(\sum_k e^{s_k}\right) - \sum_i q_i s_i + \sum_i q_i \log(q_i)$$

Pogledajmo sada gradijent ovog gubitka s obzirom na jednu komponentu vektora logita s_i :

$$\frac{\partial \mathcal{L}_{KL}(\mathbf{p}, \mathbf{q})}{\partial s_i} = \frac{e^{s_i}}{\sum_k e^{s_k}} - q_i = p_i - q_i$$

Dakle, za gradijent gubitka po vektoru logita $\mathbf{s}$ vrijedi:

$$\frac{\partial \mathcal{L}_{KL}}{\partial \mathbf{s}} = \mathbf{p} - \mathbf{q}$$

Rješenje (b):

Neka je $\mathbf{p}$ prediktirana distribucija vjerojatnosti po klasama, a $\mathbf{q}$ stvarna distribucija vjerojatnosti po klasama. Tada se gubitak unakrsne entropije definira kao:

$$\mathcal{L}_{CE}(\mathbf{p}, \mathbf{q}) = - \sum_i q_i \log(p_i)$$

U klasifikacijskim problemima, primjeri pripadaju samo jednoj klasi, tj. vjerojatnost pripadnosti ispravnoj klasi je 1, a vjerojatnost pripadanja ostalim klasama je 0. Neka je y indeks točnog razreda. Tada je $\mathbf{q}$ jednojedinčni (engl. *one-hot*) vektor, odnosno vektor za čije komponente vrijedi:

$$q_i = \begin{cases} 1, & \text{ako je } i = y \\ 0, & \text{inače} \end{cases}$$

Uvrštavanjem ove činjenice u izraz za gubitak unakrsne entropije, dobivamo gubitak negativne log-izglednosti:

$$\begin{aligned} \mathcal{L}_{CE}(\mathbf{p}, \mathbf{q}) &= - \left(1 \cdot \log(p_y) + \sum_{i \neq y} 0 \cdot \log(p_i) \right) \\ &= -\log(p_y) = \mathcal{L}_{NLL}(\mathbf{p}, y) \end{aligned}$$

Dakle, u konačnici možemo reći da je gubitak negativne log-izglednosti $\mathcal{L}_{NLL}$ specijalan slučaj gubitka unakrsne entropije $\mathcal{L}_{CE}$ u situaciji kada su ciljne distribucije zadane kao jednojedinčni vektori.

Rješenje (c):

Ako napravimo raspis gubitka KL divergencije:

$$\mathcal{L}_{KL}(\mathbf{p}, \mathbf{q}) = - \sum_i q_i \log\left(\frac{p_i}{q_i}\right) = - \sum_i q_i \log(p_i) + \sum_i q_i \log(q_i)$$

Možemo uočiti da prvi član zbroja odgovara gubitku unakrsne entropije, dok drugi član predstavlja negativnu entropiju distribucije $\mathbf{q}$, odnosno $-H(\mathbf{q})$, gdje je entropija definirana kao $H(\mathbf{q}) = - \sum_i q_i \log(q_i)$.

Prema tome, vrijedi:

$$\mathcal{L}_{KL}(\mathbf{p}, \mathbf{q}) = \mathcal{L}_{CE}(\mathbf{p}, \mathbf{q}) - H(\mathbf{q})$$

Iz ovog izraza se jasno vidi veza između ta dva gubitka: KL divergencija jednaka je razlici unakrsne entropije i entropije ciljne distribucije $\mathbf{q}$.

Rješenje (d):

Na temelju zaključka iz (c) dijela zadatka, gubitak KL divergencije za uniformnu distribuciju možemo raspisati kao:

$$\begin{aligned}\mathcal{L}_{KL}(\mathbf{p}, \text{uniform}) &= \mathcal{L}_{CE}(\mathbf{p}, \text{uniform}) - H(\text{uniform}) \\ &= -\sum_{i=0}^2 \left(\frac{1}{3} \log(p_i) \right) + \sum_{i=0}^2 \left(\frac{1}{3} \log\left(\frac{1}{3}\right) \right) \\ &= -\sum_{i=0}^2 \left(\frac{1}{3} \log(p_i) \right) - \log(3)\end{aligned}$$

Budući da drugi član, konstanta $-\log(3)$, ne ovisi o parametrima modela, on ne utječe na gradijente pa ga pri optimizaciji slobodno smijemo izostaviti. Dobivamo:

$$\mathcal{L}'_{KL} = -\sum_{i=0}^2 \left(\frac{1}{3} \log(p_i) \right) = \mathcal{L}_{CE}(\mathbf{p}, \text{uniform}(3, 1/3))$$

Nadalje, iz (b) dijela zadatka znamo vezu gubitka negativne log-izglednosti i unakrsne entropije, pa možemo uočiti sljedeće:

$$\mathcal{L}_{NLL}(p_y) = \mathcal{L}_{NLL}(\mathbf{p}, y) = \mathcal{L}_{CE}(\mathbf{p}, \text{oh}(3, y))$$

(kao i obično, pretpostavlja se da je $0 \log(0) = 0$).

Konačno, koristeći prethodna razmatranja, cjelokupnu funkciju gubitka možemo zapisati kao:

$$\mathcal{L} = \mathcal{L}_{CE}(\mathbf{p}, \mathbf{q}), \quad \mathbf{q} = \begin{cases} \text{oh}(3, y), & \text{ako } y = 0, 1, 2 \\ \text{uniform}(3, 1/3), & \text{inače} \end{cases}$$

Pri tome $\text{oh}(n, y)$ označava jednodinični vektor dimenzije n s jedinicom na indeksu y , a $\text{uniform}(n, p)$ označava vektor dimenzije n čiji su svi elementi jednaki p .

Rješenje (e):

Računamo unaprijedni prolaz za podatak $\mathbf{x}_1$:

$$\mathbf{s}_1 = \mathbf{W}\mathbf{x}_1 + \mathbf{b} = \begin{bmatrix} 1 & -1 \\ 1 & 1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} -5 \\ 2 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} -7 \\ -3 \\ 7 \end{bmatrix}$$

$$\begin{aligned}\mathbf{p}_1 &= \text{softmax}(\mathbf{s}_1) = \begin{bmatrix} e^{-7}/(e^{-7} + e^{-3} + e^7) & e^{-3}/(e^{-7} + e^{-3} + e^7) & e^7/(e^{-7} + e^{-3} + e^7) \end{bmatrix}^\top \\ &\approx \begin{bmatrix} e^{-7}/e^7 & e^{-3}/e^7 & e^7/e^7 \end{bmatrix}^\top = \begin{bmatrix} e^{-14} & e^{-10} & 1 \end{bmatrix}^\top\end{aligned}$$

$$\mathcal{L}_1 = \mathcal{L}_{CE}(\mathbf{p}_1, \text{oh}(3, 0)) = -\log(p_{1,0}) = 14$$

Računamo unatrazni prolaz za podatak $\mathbf{x}_1$:

$$\mathbf{q}_1 = \text{oh}(3, 0) = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix}^\top$$

$$\frac{\partial \mathcal{L}_1}{\partial \mathbf{s}_1} = (\mathbf{p}_1 - \mathbf{q}_1)^\top = \begin{bmatrix} e^{-14} - 1 & e^{-10} & 1 \end{bmatrix}$$

$$\begin{aligned}
\frac{\partial \mathcal{L}_1}{\partial \mathbf{W}} &= \left(\frac{\partial \mathcal{L}_1}{\partial \mathbf{s}_1} \right)^\top \mathbf{x}_1^\top = \begin{bmatrix} e^{-14} - 1 \\ e^{-10} \\ 1 \end{bmatrix} \begin{bmatrix} -5 & 2 \end{bmatrix} \\
&= \begin{bmatrix} -5(e^{-14} - 1) & 2(e^{-14} - 1) \\ -5e^{-10} & 2e^{-10} \\ -5 & 2 \end{bmatrix} \\
\frac{\partial \mathcal{L}_1}{\partial \mathbf{b}} &= \frac{\partial \mathcal{L}_1}{\partial \mathbf{s}_1} \frac{\partial \mathbf{s}_1}{\partial \mathbf{b}} = \frac{\partial \mathcal{L}_1}{\partial \mathbf{s}_1} \mathbf{I} = \begin{bmatrix} e^{-14} - 1 & e^{-10} & 1 \end{bmatrix}
\end{aligned}$$

Računamo unaprijedni prolaz za podatak $\mathbf{x}_2$:

$$\mathbf{s}_2 = \mathbf{W} \mathbf{x}_2 + \mathbf{b} = \begin{bmatrix} 1 & -1 \\ 1 & 1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 6 \\ 8 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} -2 \\ 14 \\ 2 \end{bmatrix}$$

$$\begin{aligned}
\mathbf{p}_2 = \text{softmax}(\mathbf{s}_2) &= \begin{bmatrix} e^{-2}/(e^{-2} + e^{14} + e^2) & e^{14}/(e^{-2} + e^{14} + e^2) & e^2/(e^{-2} + e^{14} + e^2) \end{bmatrix}^\top \\
&\approx \begin{bmatrix} e^{-2}/e^{14} & e^{14}/e^{14} & e^2/e^{14} \end{bmatrix}^\top = \begin{bmatrix} e^{-16} & 1 & e^{-12} \end{bmatrix}^\top
\end{aligned}$$

$$\mathcal{L}_2 = \mathcal{L}_{CE}(\mathbf{p}_2, \text{uniform}(3, 1/3)) = -\frac{1}{3}(-16 + 0 - 12) = \frac{28}{3} \approx 9.333$$

Računamo unatražni prolaz za podatak $\mathbf{x}_2$:

$$\begin{aligned}
\mathbf{q}_2 &= \text{uniform}(3, 1/3) = \begin{bmatrix} 1/3 & 1/3 & 1/3 \end{bmatrix}^\top \\
\frac{\partial \mathcal{L}_2}{\partial \mathbf{s}_2} &= (\mathbf{p}_2 - \mathbf{q}_2)^\top = \begin{bmatrix} e^{-16} - 1/3 & 2/3 & e^{-12} - 1/3 \end{bmatrix}
\end{aligned}$$

$$\begin{aligned}
\frac{\partial \mathcal{L}_2}{\partial \mathbf{W}} &= \left(\frac{\partial \mathcal{L}_2}{\partial \mathbf{s}_2} \right)^\top \mathbf{x}_2^\top = \begin{bmatrix} e^{-16} - 1/3 \\ 2/3 \\ e^{-12} - 1/3 \end{bmatrix} \begin{bmatrix} 6 & 8 \end{bmatrix} \\
&= \begin{bmatrix} 6(e^{-16} - 1/3) & 8(e^{-16} - 1/3) \\ 4 & 16/3 \\ 6(e^{-12} - 1/3) & 8(e^{-12} - 1/3) \end{bmatrix} \\
\frac{\partial \mathcal{L}_2}{\partial \mathbf{b}} &= \frac{\partial \mathcal{L}_2}{\partial \mathbf{s}_2} \frac{\partial \mathbf{s}_2}{\partial \mathbf{b}} = \frac{\partial \mathcal{L}_2}{\partial \mathbf{s}_2} \mathbf{I} = \begin{bmatrix} e^{-16} - 1/3 & 2/3 & e^{-12} - 1/3 \end{bmatrix}
\end{aligned}$$

Zadatak 5

Razmatramo klasifikacijski model zadan nizom slojeva:

$$\mathbf{s}_1 = \text{conv1D}(\mathbf{x}, \mathbf{w}_1, b_1)$$

$$\mathbf{h}_1 = \text{ReLU}(\mathbf{s}_1)$$

$$\mathbf{g}_1 = \text{concat}(\mathbf{h}_1, \mathbf{x})$$

$$s_2 = \mathbf{w}_2^T \mathbf{g}_1$$

$$p = \sigma(s_2)$$

Izračunajte gradijente gubitka binarne unakrsne entropije obzirom na sve parametre modela, ako je zadan ulazni podatak $\mathbf{x} = \begin{bmatrix} 1 & 0 & -1 \end{bmatrix}^T$, parametri modela $\mathbf{w}_1 = \begin{bmatrix} 1 & 1 \end{bmatrix}^T$, $b_1 = 0$, $\mathbf{w}_2 = \begin{bmatrix} 1 & 1 & 0 & 1 & 1 \end{bmatrix}^T$, te točna oznaka podatka $y = 0$.

Napomena: operacija `concat` označava konkatenciju dva vektora dimenzija n_1 i n_2 ; rezultat te operacije je novi vektor s $n_1 + n_2$ elemenata, gdje prvih n_1 elemenata odgovara prvome vektoru, a preostalih n_2 elemenata drugome vektoru.

Rješenje:

Unaprijedni prolaz:

$$\mathbf{s}_1 = \text{conv1D}(\mathbf{x}, \mathbf{w}_1, b_1) = \mathbf{w}_1 \star \mathbf{x} + \text{broadcast}(b_1) = \begin{bmatrix} 1 & 1 \end{bmatrix}^T \star \begin{bmatrix} 1 & 0 & -1 \end{bmatrix}^T + \begin{bmatrix} 0 & 0 \end{bmatrix}^T = \begin{bmatrix} 1 & -1 \end{bmatrix}^T$$

$$\mathbf{h}_1 = \text{ReLU}(\mathbf{s}_1) = \begin{bmatrix} 1 & 0 \end{bmatrix}^T$$

$$\mathbf{g}_1 = \text{concat}(\mathbf{h}_1, \mathbf{x}) = \begin{bmatrix} 1 & 0 & 1 & 0 & -1 \end{bmatrix}^T$$

$$s_2 = \mathbf{w}_2^T \mathbf{g}_1 = \begin{bmatrix} 1 & 1 & 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \\ -1 \end{bmatrix} = 0$$

$$p = \sigma(s_2) = 0.5$$

$$y = 0 \implies \mathcal{L} = -y \log(p) - (1 - y) \log(1 - p) = -\log(0.5) = \log(2)$$

Unatražni prolaz:

$$\frac{\partial \mathcal{L}}{\partial s_2} = p - y = 0.5$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}_2} = \frac{\partial \mathcal{L}}{\partial s_2} \frac{\partial s_2}{\partial \mathbf{w}_2} = \frac{\partial \mathcal{L}}{\partial s_2} \mathbf{g}_1^T = 0.5 \begin{bmatrix} 1 & 0 & 1 & 0 & -1 \end{bmatrix}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{g}_1} = \frac{\partial \mathcal{L}}{\partial s_2} \frac{\partial s_2}{\partial \mathbf{g}_1} = \frac{\partial \mathcal{L}}{\partial s_2} \mathbf{w}_2^T = 0.5 \begin{bmatrix} 1 & 1 & 0 & 1 & 1 \end{bmatrix}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{g}_1} \frac{\partial \mathbf{g}_1}{\partial \mathbf{h}_1}$$

Ovdje bi nas možda na prvu moglo zbuniti kako izračunati $\frac{\partial \mathbf{g}_1}{\partial \mathbf{h}_1}$ budući da se radi o operaciji konkatencije čiji gradijent ne računamo baš toliko često na ovom predmetu. No, intuicija iza

ovoga nije jako komplicirana. Naime, promjena u nekoj komponenti od $\mathbf{h}_1$ utječe na promjenu samo u onoj komponenti od $\mathbf{g}_1$ na koju je “sjela” tijekom konkatencije.

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial \mathbf{h}_1} &= \frac{\partial \mathcal{L}}{\partial \mathbf{g}_1} \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} = 0.5 \begin{bmatrix} 1 & 1 & 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} = 0.5 \begin{bmatrix} 1 & 1 \end{bmatrix} \\ \frac{\partial \mathcal{L}}{\partial \mathbf{s}_1} &= \frac{\partial \mathcal{L}}{\partial \mathbf{h}_1} \frac{\partial \mathbf{h}_1}{\partial \mathbf{s}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_1} \odot \llbracket \mathbf{s}_1 > 0 \rrbracket^\top = 0.5 \begin{bmatrix} 1 & 1 \end{bmatrix} \odot \begin{bmatrix} 1 & 0 \end{bmatrix} = \begin{bmatrix} 0.5 & 0 \end{bmatrix} \\ \frac{\partial \mathcal{L}}{\partial \mathbf{w}_1} &= \frac{\partial \mathcal{L}}{\partial \mathbf{s}_1} \star \mathbf{x}^\top = \begin{bmatrix} 0.5 & 0 \end{bmatrix} \star \begin{bmatrix} 1 & 0 & -1 \end{bmatrix} = \begin{bmatrix} 0.5 & 0 \end{bmatrix} \\ \frac{\partial \mathcal{L}}{\partial b_1} &= \sum \left(\frac{\partial \mathcal{L}}{\partial \mathbf{s}_1} \right) = 0.5\end{aligned}$$

5 MI 2022./2023.

Zadatak 1

Razmatramo klasifikacijsku konvolucijsku mrežu s arhitekturom:

- konvolucijski sloj bez nadopunjavanja: dvije jezgre dimenzija $k \times k$, bez pomaka, korak 1;
- aktivacija ReLU;
- sažimanje maksimumom 2×2 s korakom 2;
- pretvaranje u vektor;
- potpuno povezani sloj dimenzije 2 te aktivacijom softmax.

Poznata je ulazna slika X i parametri W potpuno povezanog sloja (nema pomaka!):

$$X = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & -2 & 3 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}, \quad W = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 & 1 & 0 & 1 \end{bmatrix}.$$

Zadatci:

- Odredite nepoznati hiperparametar k u konvolucijskom sloju, takav da omogućiti validan unaprijedni prolaz kroz model ako je poznato da vrijedi $k > 1$.
- Odredite izlaze mreže ako su parametri konvolucijske jezgre zadani na sljedeći način. Težina prve jezgre u gornjem lijevom kutu jednaka je 1, u gornjem desnom -1, a 0 inače. Težine druge jezgre u gornjem lijevom i gornjem desnom kutu jednake su 1, a 0 inače.
- Odredite gradijente negativne log-izglednosti po izlazima konvolucijskog sloja, ako je točna oznaka za zadanu sliku $y = 0$.

Napomena: Kod sloja sažimanja maksimumom, u slučaju više jednakih kandidata izabrati onoga s manjim indeksom.

Rješenje (a):

Za početak napravimo tablicu koja prikazuje sve slojeve, uključujući dimenzije njihovih izlaza.

Sloj	Dimenzija izlaza
Slika	$1 \times 5 \times 5$
Conv($C_{\text{in}} = 1$, $C_{\text{out}} = 2$, bias = F, ker_size = k , $s = 1$, $p = 0$)	$2 \times H_1 \times W_1$
MaxPool(ker_size = 2, $s = 2$, $p = 0$)	$2 \times H_2 \times W_2$
Flatten	$2 \cdot H_2 \cdot W_2$
FC($D_{\text{in}} = 8$, $D_{\text{out}} = 2$)	2
Softmax	2

Trenutno smo u tablicu zapisali sve parametre slojeva koji su nam poznati direktno iz zadanih podataka. Možda će nekoga zbuniti od kuda D_{in} i D_{out} za FC sloj, no to se vidi iz dimenzija matrice W .

Sada se vidi da mora vrijediti:

$$2 \cdot H_2 \cdot W_2 = 8 \iff H_2 \cdot W_2 = 4$$

Budući da je ulazna slika kvadratna te budući da konvolucijski sloj te sloj sažimanja imaju parametre (k, s, p) koji su simetrični za obje osi, izlaz nakon tih slojeva će ostati kvadratan.

Prema tome vrijedi:

$$H_2 = W_2 = 2$$

Nadalje, zbog načina na koji sloj sažimanja mijenja prostornu dimenzionalnost vrijedi:

$$\begin{aligned} H_2 &= \lfloor (H_1 + 2 \cdot p - k) / s \rfloor + 1 \\ 2 &= \lfloor (H_1 - 2) / 2 \rfloor + 1 \\ 1 &= \lfloor (H_1 - 2) / 2 \rfloor \end{aligned}$$

$$\begin{aligned} 1 &\leq \frac{H_1 - 2}{2} < 2 \\ 2 &\leq H_1 - 2 < 4 \\ 4 &\leq H_1 < 6 \end{aligned}$$

Prema ove dvije nejednakosti, vrijedi $H_1 = 4$ ili $H_1 = 5$, no budući da je H_1 visina izlaza iz konvolucijskog sloja bez nadopunjavanja i budući da je $k < 1$, onda je nemoguće da ta visina bude ista kao i visina ulaza u konvolucijski sloj. Prema tome zaključujemo da vrijedi:

$$H_1 = 4$$

Analogno ovom za H_1 , dolazimo do toga da vrijedi:

$$W_1 = 4$$

Konačno, veličinu jezgre dobivamo iz sljedeće jednadžbe:

$$\begin{aligned} H_1 &= \lfloor (5 + 2 \cdot p_{\text{conv}} - k) / s_{\text{conv}} \rfloor + 1 \\ 4 &= \lfloor 5 - k \rfloor + 1 \\ 3 &= \lfloor 5 - k \rfloor \end{aligned}$$

$$\begin{aligned} 3 &\leq 5 - k < 4 \\ -2 &\leq -k < -1 \\ 1 &< k \leq 2 \end{aligned}$$

Dakle, vrijedi da je $k = 2$.

Sloj	Dimenzija izlaza
Slika	$1 \times 5 \times 5$
Conv($C_{\text{in}}=1, C_{\text{out}}=2, \text{bias}=\text{F}, \text{ker_size}=2, s=1, p=0$)	$2 \times 4 \times 4$
MaxPool($\text{ker_size}=2, s=2, p=0$)	$2 \times 2 \times 2$
Flatten	8
FC($D_{\text{in}}=8, D_{\text{out}}=2$)	2
Softmaks	2

Rješenje (b):

Zapišimo matrice težina konvolucijskih jezgri na temelju teksta zadatka:

$$W_1^{(1)} = \begin{bmatrix} 1 & -1 \\ 0 & 0 \end{bmatrix}, \quad W_1^{(2)} = \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix}$$

Računamo vrijednosti oba izlazna kanala konvolucijskog sloja:

$$g^{(1)} = W_1^{(1)} \star X = \begin{bmatrix} 0 & 0 & 0 & 0 \\ -1 & 1 & 0 & 0 \\ -1 & 1 & 0 & 0 \\ 0 & 0 & 2 & -5 \end{bmatrix}$$

$$g^{(2)} = W_1^{(2)} \star X = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & -2 & 1 \end{bmatrix}$$

Slijedi ReLU aktivacija:

$$h^{(1)} = \text{ReLU}(g^{(1)}) = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 2 & 0 \end{bmatrix}, \quad h^{(2)} = \text{ReLU}(g^{(2)}) = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Slijedi sažimanje maksimumom s jezgrom 2×2 i korakom 2:

$$f^{(1)} = \text{MaxPool}(h^{(1)}) = \begin{bmatrix} 1 & 0 \\ 1 & 2 \end{bmatrix}, \quad f^{(2)} = \text{MaxPool}(h^{(2)}) = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$$

Slijedi vektorizacija:

$$g = \text{Flatten}(f^{(1)}, f^{(2)}) = [1 \ 0 \ 1 \ 2 \ 1 \ 0 \ 1 \ 1]^\top$$

Slijedi potpuno povezani sloj:

$$s = Wg = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 & 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \\ 2 \\ 1 \\ 0 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 3 \end{bmatrix}$$

Konačno, primjenom softmax funkcije na logite, dobivamo izlazni vektor vjerojatnosti modela:

$$p = \text{softmax}(s) = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}$$

Rješenje (c):

$$y = 0 \implies y_{\text{oh}} = \begin{bmatrix} 1 & 0 \end{bmatrix}^\top$$

$$\frac{\partial \mathcal{L}}{\partial s} = (p - y_{\text{oh}})^\top = \begin{bmatrix} -0.5 & 0.5 \end{bmatrix}$$

$$\frac{\partial \mathcal{L}}{\partial g} = \frac{\partial \mathcal{L}}{\partial s} \frac{\partial s}{\partial g} = \frac{\partial \mathcal{L}}{\partial s} W = 0.5 \begin{bmatrix} -1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 & 1 & 0 & 1 \end{bmatrix} = 0.5 \begin{bmatrix} 0 & -1 & 1 & 0 & 0 & 1 & -1 & 0 \end{bmatrix}$$

Gradijente gubitka po $g^{(1)}$ i $g^{(2)}$ dobivamo “de-konkateniranjem” gradijenta po g .

$$\frac{\partial \mathcal{L}}{\partial g^{(1)}} = \frac{\partial \mathcal{L}}{\partial g} [0 : 4] = 0.5 \begin{bmatrix} 0 & -1 & 1 & 0 \end{bmatrix}$$

$$\frac{\partial \mathcal{L}}{\partial g^{(2)}} = \frac{\partial \mathcal{L}}{\partial g} [4 : 8] = 0.5 \begin{bmatrix} 0 & 1 & -1 & 0 \end{bmatrix}$$

Gradijent gubitka po $f^{(1)}$, odnosno $f^{(2)}$, dobivamo “de-vektorizacijom” gradijenta po $g^{(1)}$, odnosno $g^{(2)}$.

$$\frac{\partial \mathcal{L}}{\partial f^{(1)}} = \text{Unflatten} \left(\frac{\partial \mathcal{L}}{\partial g^{(1)}} \right) = 0.5 \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

$$\frac{\partial \mathcal{L}}{\partial f^{(2)}} = \text{Unflatten} \left(\frac{\partial \mathcal{L}}{\partial g^{(2)}} \right) = 0.5 \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$$

Gradijent gubitka po ulazu (h) u MaxPool, $\partial \mathcal{L} / \partial h$, računamo na sljedeći način. Razmatramo pozicije (i, j) ulaza h s kojih je rezultirao maksimum u nekom koraku MaxPool-a. Taj maksimum je zapisan u rezultat f na neki indeks (i', j'). Uzimamo vrijednost s indeksa (i', j') jakobijana $\partial \mathcal{L} / \partial f$ te zapisujemo na indeks (i, j) jakobijana $\partial \mathcal{L} / \partial h$.

$$\frac{\partial \mathcal{L}}{\partial h^{(1)}} = \begin{bmatrix} 0 & 0 & -0.5 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0.5 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad \frac{\partial \mathcal{L}}{\partial h^{(2)}} = \begin{bmatrix} 0 & 0 & 0.5 & 0 \\ 0 & 0 & 0 & 0 \\ -0.5 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial s^{(1)}} &= \frac{\partial \mathcal{L}}{\partial h^{(1)}} \odot \llbracket s^{(1)} > 0 \rrbracket \\ &= \begin{bmatrix} 0 & 0 & -0.5 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0.5 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \odot \begin{bmatrix} 0 & 0 & 0 & 0 \\ -1 & 1 & 0 & 0 \\ -1 & 1 & 0 & 0 \\ 0 & 0 & 2 & -5 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0.5 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \end{aligned}$$

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial s^{(2)}} &= \frac{\partial \mathcal{L}}{\partial h^{(2)}} \odot \llbracket s^{(2)} > 0 \rrbracket \\ &= \begin{bmatrix} 0 & 0 & 0.5 & 0 \\ 0 & 0 & 0 & 0 \\ -0.5 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \odot \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & -2 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ -0.5 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \end{aligned}$$

Zadatak 2

Potrebno je napraviti model za detekciju znaka plus u binarnoj ulaznoj slici veličine 4×4 .

Plusom smatramo isječak izgleda $\begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix}$. Plus se može naći u bilo kojem dijelu slike.

Ako se plus nalazi u slici, model na izlazu mora predvidjeti 1, inače mora predvidjeti 0.
Zadaci:

- Konstruirajte potpuno povezani model koji može izvoditi zadani zadatak.
- Konstruirajte konvolucijski model koji može izvoditi zadani zadatak.
- Prikažite rad konstruiranih modela na slikama:
 - intenzitet 1 u gornjem lijevom pikselu, svi ostali intenziteti 0
 - plus u donjem desnom kutu, svi ostali intenziteti 0
- Izračunajte broj parametara za predložene modele i usporedite ih.
- Objasnite koja je od ovih arhitektura prikladnija za zadani problem.

Rješenje (a):

Razmotrimo najprije sve moguće ulazne slike veličine 4×4 na kojima se nalazi znak plus. Neka su $*$ označeni oni elementi matrice za koje nije bitno jesu li 0 ili 1:

$$\begin{bmatrix} 0 & 1 & 0 & * \\ 1 & 1 & 1 & * \\ 0 & 1 & 0 & * \\ * & * & * & * \end{bmatrix}, \begin{bmatrix} * & 0 & 1 & 0 \\ * & 1 & 1 & 1 \\ * & 0 & 1 & 0 \\ * & * & * & * \end{bmatrix}, \begin{bmatrix} * & * & * & * \\ 0 & 1 & 0 & * \\ 1 & 1 & 1 & * \\ 0 & 1 & 0 & * \end{bmatrix}, \begin{bmatrix} * & * & * & * \\ * & 0 & 1 & 0 \\ * & 1 & 1 & 1 \\ * & 0 & 1 & 0 \end{bmatrix}$$

Potpuno povezani modeli rade s vektorskim ulazima, što znači da ćemo matricu X koja dolazi na njegov ulaz morati vektorizirati:

$$x = \text{Flatten}(X)$$

Na temelju gore navedenih matrica koje predstavljaju slike na kojima se nalazi plus, pogledajmo pripadne vektorizacije:

$$\begin{aligned} x_1 &= [0 \ 1 \ 0 \ * \ 1 \ 1 \ 1 \ * \ 0 \ 1 \ 0 \ * \ * \ * \ * \ *]^T \\ x_2 &= [* \ 0 \ 1 \ 0 \ * \ 1 \ 1 \ 1 \ * \ 0 \ 1 \ 0 \ * \ * \ * \ *]^T \\ x_3 &= [* \ * \ * \ * \ 0 \ 1 \ 0 \ * \ 1 \ 1 \ 1 \ * \ 0 \ 1 \ 0 \ *]^T \\ x_4 &= [* \ * \ * \ * \ * \ 0 \ 1 \ 0 \ * \ 1 \ 1 \ 1 \ * \ 0 \ 1 \ 0]^T \end{aligned}$$

Sada trebamo konstruirati prvi potpuno povezani sloj, neka su njegovi parametri W_1 i b_1 . Izlaz sloja zadan je izrazom:

$$s_1 = W_1 x + b_1$$

Ideja je da bismo željeli da svaki redak matrice W_1 bude zadužen za jednu konfiguraciju ulaza koja sadrži znak plus. Gledajući gornje vektorizirane ulaze, intuitivno možemo doći do zaključka da redak matrice zadužen za neku konfiguraciju mora imati 1 na pozicijama na kojima

u toj konfiguraciji leži znak plus. Nadalje, redak mora imati -1 na rubovima oko plusa, tj. tamo gdje su u gore navedenim konfiguracijama nule. Elemente retka matrice koji odgovaraju oznaci $*$, možemo jednostavno postaviti na 0, pošto oni ne doprinose, a niti ne odmažu znaku plus. Shodno navedenom, matrica težina W_1 je:

$$W_1 = \begin{bmatrix} -1 & 1 & -1 & 0 & 1 & 1 & 1 & 0 & -1 & 1 & -1 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 1 & -1 & 0 & 1 & 1 & 1 & 0 & -1 & 1 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -1 & 1 & -1 & 0 & 1 & 1 & 1 & 0 & -1 & 1 & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 & -1 & 1 & -1 & 0 & 1 & 1 & 1 & 0 & -1 & 1 & -1 \end{bmatrix}$$

Na trenutak postavimo b_1 na vektor nula, tada je izlaz ovog sloja:

$$s_1 = W_1 x$$

Ako je na nekoj poziciji pronađen plus, tada će odgovarajući element vektora s_1 imati vrijednost točno 5, u suprotnom se ne radi o plusu na toj poziciji. Budući da će ovaj sloj slijediti nelinearna aktivacija ReLU, mi želimo da ReLU propusti isključivo one elemente vektora s_1 koji govore da je plus pronađen. To možemo ostvariti tako da vektor pomaka postavimo na sljedeći način:

$$b_1 = [-4 \quad -4 \quad -4 \quad -4]^\top$$

jer će nakon dodavanja ove pristranosti ostati pozitivni samo oni elementi vektora s_1 koji su imali vrijednost 5.

Slijedi ReLU aktivacija:

$$h_1 = \text{ReLU}(s_1)$$

Vektor h_1 će se sastojati isključivo od elemenata 0 i 1, pri čemu može biti maksimalno jedna vrijednost 1 jer nije moguće da na slici budu dva plusa kakvi su traženi u zadatku.

Sada želimo ovaj vektor transformirati u konačni izlaz modela z za koji će vrijediti:

$$\begin{aligned} z &= 1 && \text{ako je } 1 \in h_1 \\ z &= 0 && \text{inače} \end{aligned}$$

Ovo možemo ostvariti tako da uvedemo drugi potpuno povezani sloj, s parametrima w_2 i b_2 . Izlaz tog sloja je:

$$z = w_2^\top h_1 + b_2$$

Parametre možemo postaviti na sljedeći način:

$$w_2 = [1 \quad 1 \quad 1 \quad 1]^\top, \quad b_2 = 0$$

Budući da se u vektoru h_1 nalazi najviše jedna jedinica, dok su ostali elementi nule, ovakvim množenjem jednostavno zbrajamo elemente vektora h_1 , čime izlaz modela z poprima točno tražene vrijednosti 0 ili 1, bez potrebe za izlaznom aktivacijskom funkcijom.

Konačni pregled arhitekture potpuno povezanog modela:

Sloj	Broj parametara	Dimenzija izlaza
Slika	0	4×4
Flatten	0	16
FC1 ($D_{\text{in}} = 16, D_{\text{out}} = 4$)	$4 \cdot 16 + 4 = 68$	4
ReLU	0	4
FC2 ($D_{\text{in}} = 4, D_{\text{out}} = 1$)	$1 \cdot 4 + 1 = 5$	1

Rješenje (b):

Rješenje ovog zadatka korištenjem konvolucijskog modela dolazi puno prirodnije nego u slučaju potpuno povezanog modela. Najprije definiramo konvolucijski sloj s jednom konvolucijskom jezgrom. Neka je jezgra W_1 , a pomak b_1 . Izlaz iz sloja zadan je izrazom:

$$g = W_1 \star x + b_1$$

Jezgra W_1 zadana je kao:

$$W_1 = \begin{bmatrix} -1 & 1 & -1 \\ 1 & 1 & 1 \\ -1 & 1 & -1 \end{bmatrix}$$

Vođeni praktički istom intuicijom kao u (a) dijelu zadatka, 1 smo postavili na mjesta gdje leži plus, a -1 na mjesta oko plusa gdje svakako mora biti 0 na originalnoj slici.

Postavljamo $b_1 = -4$ te ovdje također vrijedi objašnjenje dano u (a) dijelu zadatka. Izlaz iz konvolucije, tenzor g , bit će prostornih dimenzija 2×2 . Zanima nas je li jedan od elemenata poprimio vrijednost 1.

Najprije provedimo g kroz ReLU aktivaciju:

$$h = \text{ReLU}(g)$$

Potom, provedimo operaciju sažimanja maksimumom (**MaxPool**) s veličinom jezgre 2 i korakom 2:

$$f = \text{MaxPool}(h)$$

Budući da **MaxPool** standardno vraća tenzor (u ovom slučaju prostornih dimenzija i kanala $1 \times 1 \times 1$), primijenit ćemo operaciju **Flatten** kako bismo dobili konačni skalarni izlaz z :

$$z = \text{Flatten}(f)$$

Izlaz z je sada skalar te je on jednak 1 samo u slučaju da je plus bio pronađen, a u suprotnom je 0. Budući da zadatak traži da model predvidi točno 1 ako se plus nalazi na slici, a 0 inače, ovo predstavlja konačno rješenje.

Konačni pregled arhitekture konvolucijskog modela:

Sloj	Broj parametara	Dimenzija izlaza
Slika	0	$1 \times 4 \times 4$
Conv1 ($C_{\text{in}} = 1, C_{\text{out}} = 1, k = 3, s = 1, p = 0$)	$1 \cdot 1 \cdot 3 \cdot 3 + 1 = 10$	$1 \times 2 \times 2$
ReLU	0	$1 \times 2 \times 2$
MaxPool ($k = 2, s = 2, p = 0$)	0	$1 \times 1 \times 1$
Flatten	0	1

Rješenje (c):

i. Zadan je ulaz intenziteta 1 u gornjem lijevom pikselu:

$$X = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Za potpuno povezani model:

$$x = \text{Flatten}(X) = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}^\top$$

$$s_1 = W_1 x + b_1$$

$$= \begin{bmatrix} -1 & 1 & -1 & 0 & 1 & 1 & 1 & 0 & -1 & 1 & -1 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 1 & -1 & 0 & 1 & 1 & 1 & 0 & -1 & 1 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -1 & 1 & -1 & 0 & 1 & 1 & 1 & 0 & -1 & 1 & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 & -1 & 1 & -1 & 0 & 1 & 1 & 1 & 0 & -1 & 1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} + \begin{bmatrix} -4 \\ -4 \\ -4 \\ -4 \end{bmatrix}$$

$$= \begin{bmatrix} -1 \\ 0 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} -4 \\ -4 \\ -4 \\ -4 \end{bmatrix} = \begin{bmatrix} -5 \\ -4 \\ -4 \\ -4 \end{bmatrix}$$

$$h_1 = \text{ReLU}(s_1) = \begin{bmatrix} 0 & 0 & 0 & 0 \end{bmatrix}^\top$$

$$z = w_2^\top h_1 + b_2 = \begin{bmatrix} 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} + 0 = 0$$

Za konvolucijski model:

$$g = W_1 \star X + b_1 = \begin{bmatrix} -1 & 1 & -1 \\ 1 & 1 & 1 \\ -1 & 1 & -1 \end{bmatrix} \star \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} -4 & -4 \\ -4 & -4 \end{bmatrix} = \begin{bmatrix} -1 & 0 \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} -4 & -4 \\ -4 & -4 \end{bmatrix} = \begin{bmatrix} -5 & -4 \\ -4 & -4 \end{bmatrix}$$

$$h = \text{ReLU}(g) = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

$$f = \text{MaxPool}(h) = \begin{bmatrix} 0 \end{bmatrix}$$

$$z = \text{Flatten}(f) = 0$$

ii. Zadan je plus u donjem desnom kutu:

$$X = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

Za potpuno povezani model:

$$x = \text{Flatten}(X) = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 0 \end{bmatrix}^\top$$

$$s_1 = W_1 x + b_1$$

$$= \begin{bmatrix} -1 & 1 & -1 & 0 & 1 & 1 & 1 & 0 & -1 & 1 & -1 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 1 & -1 & 0 & 1 & 1 & 1 & 0 & -1 & 1 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -1 & 1 & -1 & 0 & 1 & 1 & 1 & 0 & -1 & 1 & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 & -1 & 1 & -1 & 0 & 1 & 1 & 1 & 0 & -1 & 1 & -1 \end{bmatrix} \begin{bmatrix} 0 \\ \vdots \\ 1 \\ 0 \end{bmatrix} + \begin{bmatrix} -4 \\ -4 \\ -4 \\ -4 \end{bmatrix}$$

$$= \begin{bmatrix} 1 \\ 0 \\ 0 \\ 5 \end{bmatrix} + \begin{bmatrix} -4 \\ -4 \\ -4 \\ -4 \end{bmatrix} = \begin{bmatrix} -3 \\ -4 \\ -4 \\ 1 \end{bmatrix}$$

$$h_1 = \text{ReLU}(s_1) = \begin{bmatrix} 0 & 0 & 0 & 1 \end{bmatrix}^\top$$

$$z = w_2^\top h_1 + b_2 = \begin{bmatrix} 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} + 0 = 1$$

Za konvolucijski model:

$$g = W_1 \star X + b_1 = \begin{bmatrix} -1 & 1 & -1 \\ 1 & 1 & 1 \\ -1 & 1 & -1 \end{bmatrix} \star \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} + \begin{bmatrix} -4 & -4 \\ -4 & -4 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 5 \end{bmatrix} + \begin{bmatrix} -4 & -4 \\ -4 & -4 \end{bmatrix} = \begin{bmatrix} -3 & -4 \\ -4 & 1 \end{bmatrix}$$

$$h = \text{ReLU}(g) = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$$

$$f = \text{MaxPool}(h) = \begin{bmatrix} 1 \end{bmatrix}$$

$$z = \text{Flatten}(f) = 1$$

Rješenje (d):

Ukupan broj parametara za potpuno povezani model dobiva se iz konačne tablice u (a) dijelu zadatka zbrajanjem vrijednosti u stupcu za broj parametara:

$$68 \text{ (FC1)} + 5 \text{ (FC2)} = 73 \text{ parametra}$$

Ukupan broj parametara za konvolucijski model dobiva se iz konačne tablice u (b) dijelu zadatka zbrajanjem vrijednosti u odgovarajućem stupcu:

$$10 \text{ (Conv1)} = 10 \text{ parametara}$$

Zaključujemo da potpuno povezani model ima čak 7.3 puta više parametara od konvolucijskog modela.

Rješenje (e):

Za zadani problem prikladnija je konvolucijska arhitektura. Ključni razlozi za to:

- **Dijeljenje parametara:** Model traži isti lokalni uzorak, znak plus veličine 3×3 , neovisno o njegovoj poziciji na slici. Konvolucijski model koristi jednu te istu jezgru za cijelu sliku, dok potpuno povezani model mora učiti odvojene težine za svaku moguću lokaciju znaka plus, što rezultira nepotrebnom redundancijom i znatno većim brojem parametara, kao što je vidljivo u (d) dijelu.
- **Ekvivarijantnost i invarijantnost na pomak:** Potpuno povezani model ovisi isključivo o pozicijama koje je “zapamtio” u retcima matrice težina. S druge strane konvolucijska arhitektura donosi svojstva ekvivarijantnosti i invarijantnosti na pomak. Konvolucijski sloj je ekvivarijantan na pomak, tj. ako se plus pomakne na ulazu, pomaknut će se njegova aktivacija u mapi značajki. Sloj sažimanja maksimumom je invarijantan na pomak, što znači da model uspješno prepoznaje znak plus bez obzira na to u kojem se dijelu slike nalazi.

Zadatak 3

slučaj	točnost (%)			
	T_m	T_v	V_m	V_v
I.	98.4	98.5	52.3	53.0
II.	99.0	50.1	93.5	47.2
III.	50.4	50.2	49.3	49.0

Točnost konvolucijskog klasifikacijskog modela evaluiramo na podskupovima T_m, T_v, V_m, V_v . Podskupovi T_m, T_v sadrže slike iz skupa za **treniranje** podijeljene po veličini isključivo mali (m) ili veliki (v) objekti. Analogno, podskupovi V_m, V_v sadrže slike iz skupa za **validaciju** podijeljene na isti način.

Razmatramo rezultate evaluacije načelnih modela u tri odvojena slučaja kao što je prikazano u tablici. Nijedan od slučajeva ne pruža zadovoljavajuću točnost modela, stoga bismo u svakome od njih htjeli poduzeti neke promjene i naučiti nove poboljšane instance modela. Dostupne promjene su sljedeće:

- povećanje broja kanala u konvolucijskim slojevima,
- povećanje broja epoha treniranja modela,
- povećanje prostorne dimenzije svih konvolucijskih jezgara,
- povećanje regularizacijskog hiperparametra λ koji kažnjava L2 normu parametara modela,
- dodavanje slojeva sažimanja.

Svaku od mogućih promjena uparite s točno jednim od slučajeva. Komentirajte svaki od slučajeva. Objasnite zašto ste određenu promjenu pridružili baš tome slučaju i zašto očekujete poboljšanje točnosti modela.

Rješenje:

Hajdemo najprije za svaki od tri slučaja analizirati ostvarenu točnost na podskupovima, utvrditi mogući uzrok takvih rezultata te ih povezati s ponuđenim promjenama arhitekture i hiperparametara.

Slučaj I.

Analiza: Vidimo da model ostvaruje visoku točnost na skupovima za treniranje ($T_m = 98.4\%$, $T_v = 98.5\%$), dok na validacijskim skupovima ostvaruje točnost koja praktički odgovara nasumičnom pogađanju ($V_m = 52.3\%$, $V_v = 53.0\%$).

Uzrok: Riječ je o prenaučenosti. Model se previše prilagodio podacima iz skupa za učenje, uključujući i šum, te nema sposobnost generalizacije na neviđene podatke.

Odgovarajuća promjena:

- (d) **Povećanje regularizacijskog hiperparametra λ :** L2 regularizacija kažnjava velike iznose težina u modelu. Kazna je proporcionalna hiperparametru λ pa se njegovim povećanjem jače kažnjavaju velike vrijednosti težina. Na taj način se ograničava efektivni kapacitet modela pa se model teže može previše prilagoditi podacima za treniranje. Model se na taj način “prisiljava” da nauči robusnije značajke, što dovodi do bolje generalizacije na validacijskom skupu.

Slučaj II.

Analiza: Model ostvaruje visoku točnost na skupovima koji sadrže male objekte ($T_m = 99.0\%$, $V_m = 93.5\%$), dok na skupovima s velikim objektima ostvaruje točnost koja praktički odgovara nasumičnom pogađanju ($T_v = 50.1\%$, $V_v = 47.2\%$).

Uzrok: Glavni problem je premalo receptivno polje. Receptivno polje značajke u nekom sloju predstavlja područje ulazne slike koje utječe na vrijednost te značajke. Budući da model uspješno klasificira male objekte, on očito ima sposobnost detektiranja lokalnih značajki, no isto tako očito ne vidi dovoljan skup aktivacija ulaznog podatka da bi uspješno obuhvatio cijeli veliki objekt.

Odgovarajuće promjene:

- (c) **Povećanje prostorne dimenzije svih konvolucijskih jezgara:** Korištenjem jezgre veličine k , receptivno polje sloja se uvećava za $k - 1$ u odnosu na prethodni sloj. Izborom prostorno većih jezgri (npr. 5×5 umjesto 3×3), receptivno polje kroz dubinu modela raste znatno brže, omogućavajući modelu da ranije obuhvati veće dijelove slike.
- (e) **Dodavanje slojeva sažimanja:** Slojevi sažimanja značajno ubrzavaju rast receptivnog polja kroz dubinu modela zbog načina na koji smanjuju prostornu rezoluciju mape značajki. Na taj način konvolucijska jezgra u idućem sloju indirektno obuhvaća šire područje originalne slike. Dodatno, slojevi sažimanja uvode u model svojstvo invarijantnosti na pomak, što općenito pomaže pri robusnom prepoznavanju i malih i velikih objekata.

Slučaj III.

Analiza: U ovom slučaju model na svim podskupovima ostvaruje točnost koja praktički odgovara nasumičnom pogađanju ($T_m = 50.4\%$, $T_v = 50.2\%$, $V_m = 49.3\%$, $V_v = 49.0\%$).

Uzrok: Ovdje se radi o podnaučenosti, što se lako vidi jer model nije uspio modelirati čak ni podatke iz skupova za učenje. Dva su ključna moguća uzroka: nedovoljan kapacitet modela ili nedovoljan broj epoha za učenje.

Odgovarajuće promjene:

- (a) **Povećanje broja kanala u konvolucijskim slojevima:** Povećanjem broja izlaznih kanala po sloju izravno se povećava broj parametara, a time i ukupni kapacitet modela, što će dati modelu priliku da nauči složenije značajke.

- (b) **Povećanje broja epoha treniranja modela:** Čisto je moguće da model ima dovoljan kapacitet, ali da jednostavno nismo proveli optimizaciju kroz dovoljan broj iteracija. Treniranje modela kroz veći broj epoha daje mu više vremena za konvergirati prema boljim parametrima.

Konačno uparivanje promjena sa slučajevima:

- (a) $\rightarrow$ Slučaj III.
- (b) $\rightarrow$ Slučaj III.
- (c) $\rightarrow$ Slučaj II.
- (d) $\rightarrow$ Slučaj I.
- (e) $\rightarrow$ Slučaj II.

Zadatak 4

Razmatramo aktivacijsku funkciju softmaxs.

- (a) Pokažite da za vektor $\mathbf{s}$ i konstantu k vrijedi sljedeće: $\text{softmax}(\mathbf{s}) = \text{softmax}(\mathbf{s} + k)$.
- (b) Implementirajte funkciju `softmax_k(logits, k)` u numpy-u koja se oslanja na prethodno opisano svojstvo i logitima prije izračuna softmaxa dodaje argument k . Funkcija treba omogućiti rad s minigrupom podataka, odnosno pretpostavlja da je argument `logits` numpy polje dimenzija $N \times C$, gdje je N veličina minigrupe, a C broj razreda. Kakve oblike (eng. *shape*) argumenta k podržava vaša funkcija?
- (c) Po potrebi modificirajte implementaciju funkcije `softmax_k` tako da se ona može koristiti za ostvarivanje numerički stabilnog softmaxa; pokažite tu mogućnost na primjeru poziva funkcije pod pretpostavkom da je zadana matrica `logits`. Objasnite.

Rješenje (a):

Neka je C broj klasa. Za i -tu komponentu vektora $\text{softmax}(\mathbf{s})$ po definiciji vrijedi:

$$\text{softmax}(\mathbf{s})_i = \frac{\exp(s_i)}{\sum_{j=1}^C \exp(s_j)} \quad (6)$$

Ako sada u definiciju uvrstimo vektor kojemu je dodana konstanta k , odnosno $\mathbf{s} + k$, dobivamo:

$$\begin{aligned} \text{softmax}(\mathbf{s} + k)_i &= \frac{\exp(s_i + k)}{\sum_{j=1}^C \exp(s_j + k)} \\ &= \frac{\exp(s_i) \cdot \exp(k)}{\sum_{j=1}^C (\exp(s_j) \cdot \exp(k))} \\ &= \frac{\exp(s_i) \cdot \exp(k)}{\exp(k) \sum_{j=1}^C \exp(s_j)} \\ &= \frac{\exp(s_i)}{\sum_{j=1}^C \exp(s_j)} \end{aligned}$$

Usporedbom dobivenog rezultata s jednadžbom (6), dokazali smo da vrijedi:

$$\text{softmax}(\mathbf{s}) = \text{softmax}(\mathbf{s} + \mathbf{k})$$

Rješenje (b):

Implementacija tražene funkcije u jeziku Python pomoću `numpy`-ja:

```
1 import numpy as np
2
3 def softmax_k(logits, k):
4     logits = logits + k
5     exp_logits = np.exp(logits)
6     probs = exp_logits / exp_logits.sum(axis=1, keepdims=True)
7     return probs
8
```

Ako je matrica `logits` dimenzija (N, C) , `numpy`-jev *broadcasting* mehanizam omogućava da argument `k` ima jedan od sljedećih oblika:

- skalar
- $(1,)$
- $(C,)$
- $(1, C)$
- $(N, 1)$
- (N, C)

Rješenje (c):

Poziv funkcije za ostvarivanje numerički stabilnog softmaxa izgleda ovako:

```
softmax_k(logits, k=-logits.max(axis=1, keepdims=True))
```

Objašnjenje: Izraz `logits.max(axis=1, keepdims=True)` vraća `numpy` polje oblika $(N, 1)$ gdje se u i -tom retku nalazi maksimalni logit za i -ti primjer u minigrupi. Dobiveni vektor moramo negirati kako bi maksimalni logiti bili oduzeti od odgovarajućeg vektora logita.

Oduzimanjem najveće vrijednosti osiguravamo da je najveća vrijednost koja ulazi u eksponencijalnu funkciju točno 0 ($\exp(0) = 1$), a sve ostale vrijednosti negativne i za njih vrijednost funkcije `exp` teži u 0. Time smo uspješno spriječili numerički *overflow* koji bi rezultirao NaN vrijednostima u slučaju da su početni logiti veliki pozitivni brojevi, a zbog svojstva dokazanog pod (a) znamo da nismo promijenili konačnu razdiobu vjerojatnosti.

Zadatak 5

U modulu `layers.py` želimo dodati gubitak za učenje višezadaćnog modela koji nam omogućuje da jednoj slici istovremeno pridijelimo više razreda. Primjerice, u sustavu za klasifikaciju slika koji prepoznaje mačke, pse i hrčke, ulazna slika koja sadrži i psa i mačku trebala bi imati oznaku $[1 \ 1 \ 0]$. Predložite implementaciju razreda

`MultiLabelLossWithLogits` s metodama `forward` i `backward_inputs`. Metode na izlazu vraćaju iznos gubitka odnosno gradijente gubitka po ulazima, a kao argumente primaju logite i točne oznake. Uputa: tretirajte svaki izlaz kao neovisni binarni klasifikator. Bonus: interpretirajte vaš gubitak sa stanovišta teorije vjerojatnosti.

Rješenje:

Podsjetimo se funkcije gubitka logističke regresije za jedan binarni klasifikator. Gubitak je definiran kao:

$$L(s, y) = -y \log(\sigma(s)) - (1 - y) \log(1 - \sigma(s))$$

gdje je s logit, a y točna oznaka. Ako uvedemo oznaku za prediktiranu vjerojatnost $p = \sigma(s)$, taj gubitak možemo zapisati kao:

$$L(p, y) = -y \log(p) - (1 - y) \log(1 - p)$$

Nadalje, znamo da izraz za gradijent ovog gubitka po logitu iznosi:

$$\frac{\partial L(s, y)}{\partial s} = \sigma(s) - y = p - y$$

Pogledajmo sada što ćemo napraviti za slučaj modela zadanog u ovom zadatku. Uputa nam sugerira da svaki izlaz modela intepretiramo kao neovisni binarni klasifikator. Prema tome, ukupni gubitak za jedan primjer možemo dobiti tako da gornji izraz za gubitak binarnog klasifikatora vektoriziramo, a konačan skalar gubitka dobivamo sumiranjem svih komponenata toga vektora:

$$\begin{aligned} L(\mathbf{s}, \mathbf{y}) &= \sum (-\mathbf{y} \odot \log(\sigma(\mathbf{s})) - (1 - \mathbf{y}) \odot \log(1 - \sigma(\mathbf{s}))) \\ &= \sum_j (-y_j \log(\sigma(s_j)) - (1 - y_j) \log(1 - \sigma(s_j))) \end{aligned}$$

Što se tiče gradijenta ukupnog gubitka po vektoru logita $\mathbf{s}$, on se može dobiti istim postupkom vektorizacije gornjeg izraza gradijenta, čime se dobiva vektor:

$$\frac{\partial L(\mathbf{s}, \mathbf{y})}{\partial \mathbf{s}} = \sigma(\mathbf{s}) - \mathbf{y} = \mathbf{p} - \mathbf{y}$$

Budući da radimo u okviru modula `layers.py`, očekuje se da funkcija gubitka radi nad minigrupama veličine N . Ako su logiti i točne oznake zadani u obliku matrica $\mathbf{S}, \mathbf{Y} \in \mathbb{R}^{N \times C}$, ukupni gubitak definiramo kao prosječni gubitak preko svih primjera u minigrupi kako stopa učenja ne bi ovisila o njezinoj veličini:

$$J(\mathbf{S}, \mathbf{Y}) = \frac{1}{N} \sum_{i=1}^N L(\mathbf{s}^{(i)}, \mathbf{y}^{(i)})$$

Iz ovoga slijedi da matrica gradijenata gubitka po ulaznim logitima iznosi:

$$\frac{\partial J(\mathbf{S}, \mathbf{Y})}{\partial \mathbf{S}} = \frac{1}{N} (\mathbf{P} - \mathbf{Y})$$

gdje je matrica prediktiranih vjerojatnosti $\mathbf{P} = \sigma(\mathbf{S})$.

U nastavku je dana implementacija traženog razreda u programskom jeziku Python pomoću `numpy`-ja.

```

1 import numpy as np
2
3 class MultiLabelLossWithLogits:
4     def __init__(self, name="MultiLabelLossWithLogits"):
5         self.has_params = False
6         self.name = name
7
8     def forward(self, logits, y):
9         probs = 1 / (1 + np.exp(-logits))
10        loss_i = np.sum(-y * np.log(probs) - (1 - y) * np.log(1 - probs)
11        , axis=1)
12        loss = np.mean(loss_i)
13        return loss
14
15    def backward_inputs(self, logits, y):
16        N = logits.shape[0]
17        probs = 1 / (1 + np.exp(-logits))
18        dloss_dlogits = (probs - y) / N
19        return dloss_dlogits

```

Bonus: Sa stanovišta teorije vjerojatnosti, ovaj gubitak predstavlja minimizaciju negativne log-izglednosti uz pretpostavku da su izlazi Bernoullijeve slučajne varijable uvjetno nezavisne s obzirom na ulaz.

6 MI 2024./2025.

Napomena: Tekstovi zadataka s ovog međuispita rekonstruirani su isključivo prema sjećanju kolega studenata te možda ne odgovaraju u potpunosti originalnim ispitnim zadacima.

Zadatak 1

Ovaj zadatak je identičan 1. zadatku s međuispita 2021./2022. Tekst zadatka i pripadajuće rješenje možete pogledati tamo.

Zadatak 2

Razmatramo klasifikacijski model s arhitekturom koja se sastoji od skrivenog potpuno povezanog sloja s aktivacijskom funkcijom ReLU i izlaznog potpuno povezanog sloja s aktivacijskom funkcijom softmaks. Oba sloja imaju pomak (engl. *bias*). Parametre modela optimiramo postupkom stohastičkog gradijentnog spusta na razini mini-grupe, uz korištenje gubitka unakrsne entropije.

Zadaci:

- (a) Izvedite analitičke izraze za ažuriranje svih parametara modela (težina i pomaka).
- (b) Napišite `numpy` implementaciju unatražnog prolaza i ažuriranja parametara.

Rješenje (a):

U ovom rješenju će svi izračuni biti provedeni na razini mini-grupe.

Neka je N broj primjera u mini-grupi, D broj značajki te C broj klasa.

Neka je $\mathbf{X} \in \mathbb{R}^{N \times D}$ matrica svih primjera, gdje se u i -tom retku matrice nalazi vektor redak i -tog primjera, tj. $\mathbf{x}_i^\top$.

Neka je $\mathbf{Y} \in \mathbb{R}^{N \times C}$ matrica točnih oznaka primjera, gdje se u i -tom retku nalazi jednojedinичni vektor redak točne oznake i -tog primjera, $\mathbf{y}_{\text{oh},i}^\top$.

Pogledajmo sada što se događa u unaprijednom prolazu. Neka je $\mathbf{W}_1$ matrica težina, a $\mathbf{b}_1$ vektor pomaka prvog potpuno povezanog sloja. Tada je izlaz prvog potpuno povezanog sloja $\mathbf{S}_1$ dan kao:

$$\mathbf{S}_1 = \mathbf{X}\mathbf{W}_1^\top + \text{broadcast}(\mathbf{b}_1^\top)$$

Neka je $\mathbf{H}_1$ izlaz iz aktivacije ReLU koja slijedi prvi potpuno povezani sloj:

$$\mathbf{H}_1 = \text{ReLU}(\mathbf{S}_1)$$

Neka je $\mathbf{W}_2$ matrica težina, a $\mathbf{b}_2$ vektor pomaka drugog potpuno povezanog sloja. Tada je izlaz drugog potpuno povezanog sloja $\mathbf{S}_2$ dan kao:

$$\mathbf{S}_2 = \mathbf{H}_1\mathbf{W}_2^\top + \text{broadcast}(\mathbf{b}_2^\top)$$

Neka je $\mathbf{P}$ izlaz iz aktivacije softmaks koja slijedi drugi potpuno povezani sloj. Ovdje je i -ti redak matrice $\mathbf{P}$ dobiven djelovanjem softmaks nad i -tim retkom matrice $\mathbf{S}_2$. Izraz je dan kao:

$$\mathbf{P} = \text{softmax}(\mathbf{S}_2)$$

Neka je $\mathbf{p}_i$ i -ti redak matrice $\mathbf{P}$ te neka je y_i oznaka točnog razreda i -tog primjera. Gubitak unakrsne entropije za zadanu mini-grupu definiran je kao prosjek gubitaka pojedinačnih primjera:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N \mathcal{L}_i = -\frac{1}{N} \sum_{i=1}^N \log(p_{i,y_i})$$

Sada možemo pogledati što se događa u unatražnom prolazu. Ovdje navedeni izrazi gradijenata poznati su nam iz prve laboratorijske vježbe:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{S}_2} = \frac{1}{N}(\mathbf{P} - \mathbf{Y})$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_2} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{S}_2} \right)^\top \mathbf{H}_1$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}_2} = \sum_{i=1}^N \left(\frac{\partial \mathcal{L}}{\partial \mathbf{S}_2} \right)_{i,:}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{H}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{S}_2} \mathbf{W}_2$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{S}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{H}_1} \odot \llbracket \mathbf{S}_1 > 0 \rrbracket$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_1} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{S}_1} \right)^\top \mathbf{X}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}_1} = \sum_{i=1}^N \left(\frac{\partial \mathcal{L}}{\partial \mathbf{S}_1} \right)_{i,:}$$

Sad kad smo izveli gradijente za sve parametre modela, možemo napisati izraze za njihovo ažuriranje. Neka je zadan hiperparametar η kao korak učenja. Ažuriranje se provodi na temelju sljedećih izraza:

$$\begin{aligned} \mathbf{W}_1 &\leftarrow \mathbf{W}_1 - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{W}_1} \\ \mathbf{b}_1 &\leftarrow \mathbf{b}_1 - \eta \left(\frac{\partial \mathcal{L}}{\partial \mathbf{b}_1} \right)^\top \\ \mathbf{W}_2 &\leftarrow \mathbf{W}_2 - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{W}_2} \\ \mathbf{b}_2 &\leftarrow \mathbf{b}_2 - \eta \left(\frac{\partial \mathcal{L}}{\partial \mathbf{b}_2} \right)^\top \end{aligned}$$

Rješenje (b):

Implementacija u programskom jeziku Python pomoću biblioteke `numpy`:

```

1 import numpy as np
2
3 def train_step(X, Y, W1, b1, W2, b2, eta):
4     N = X.shape[0]
5
6     # Unaprijedni prolaz
7     S1 = X @ W1.T + b1
8     H1 = (S1 > 0) * S1
9     S2 = H1 @ W2.T + b2
10
11     S2_stable = S2 - S2.max(axis=1, keepdims=True)
12     S2_stable_exp = np.exp(S2_stable)
13     P = S2_stable_exp / S2_stable_exp.sum(axis=1, keepdims=True)
14

```

```

15     loss = -1 / N * (np.log(P) * Y).sum()
16
17     # Unatrazni prolaz
18     dL_dS2 = 1 / N * (P - Y)
19     dL_dW2 = dL_dS2.T @ H1
20     dL_db2 = dL_dS2.sum(axis=0)
21
22     dL_dH1 = dL_dS2 @ W2
23     dL_dS1 = dL_dH1 * (S1 > 0)
24
25     dL_dW1 = dL_dS1.T @ X
26     dL_db1 = dL_dS1.sum(axis=0)
27
28     # Azuriranje parametara
29     W1 -= eta * dL_dW1
30     b1 -= eta * dL_db1
31     W2 -= eta * dL_dW2
32     b2 -= eta * dL_db2
33
34     return loss
35

```

Zadatak 3

Ovaj zadatak je identičan [5. zadatku s međuispita 2021./2022.](#) Tekst zadatka i pripadajuće rješenje možete pogledati tamo.

Zadatak 4

Razmatramo učenje višerazredne logističke regresije s djelomično označenim primjerima. Koristimo dva različita gubitka: jedan za označene primjere, a drugi za neoznačene. Za označene primjere koristimo negativnu log-izglednost, dok za neoznačene koristimo gubitak takav da je predikcija što različitija od uniformne razdiobe.

Zadaci:

- Napišite jednadžbu gubitka za označene primjere.
- Predložite dosljedan gubitak za učenje neoznačenih primjera. Možete li to povezati s formalnom mjerom različitosti distribucija? Koja je to mjera?
- Izvedite jednadžbu gradijenta gubitka po klasifikacijskim mjerama (logitima) $\partial \mathcal{L} / \partial \mathbf{s}$.

Rješenje (a):

Neka je $\mathbf{x}$ označen primjer s oznakom y te neka je $\mathbf{y}_{\text{oh}}$ pripadna jednojedinичna oznaka. Neka je $\mathbf{s}$ vektor logita koji je model izračunao. Tada je gubitak negativne log-izglednosti dan kao:

$$\mathcal{L}_{\text{NLL}} = -\log(p_y) = -\log(\text{softmax}(\mathbf{s})_y)$$

Rješenje (b):

Neka je C broj klasa. Zadatak kaže da za neoznačene primjere želimo koristiti gubitak takav da je predikcija što različitija od uniformne distribucije. Također, tekst ovog podzadatka nas navodi da ovo povežemo s formalnom mjerom različitosti distribucija. Jedna takva mjera koja se spominje na ovom predmetu je KL divergencija.

Neka je $\mathbf{p}$ prediktirana distribucija te neka je $\mathbf{q}$ ciljna distribucija. KL divergencija između tih distribucija se definira kao:

$$D_{KL}(\mathbf{p} \parallel \mathbf{q}) = \mathcal{L}_{CE}(\mathbf{p}, \mathbf{q}) - H(\mathbf{q})$$

Dakle, što je izraz $D_{KL}(\mathbf{p} \parallel \mathbf{q})$ veći, to je udaljenost distribucije $\mathbf{p}$ od distribucije $\mathbf{q}$ veća. U našem slučaju, mi želimo maksimizirati udaljenost prediktirane distribucije $\mathbf{p}$ od uniformne distribucije. Budući da se gubitak minimizira, on mora odgovarati negativnoj KL divergenciji jer minimizacija negativne KL divergencije odgovara maksimizaciji KL divergencije.

Dakle, gubitak temeljen na KL divergenciji zadan je kao:

$$\begin{aligned} \mathcal{L}_{KL}(\mathbf{p}, \mathbf{q}) &= H(\mathbf{q}) - \mathcal{L}_{CE}(\mathbf{p}, \mathbf{q}) \\ &= - \sum_{i=1}^C (q_i \log(q_i)) + \sum_{i=1}^C (q_i \log(p_i)) \end{aligned}$$

U našem slučaju, ciljna distribucija $\mathbf{q}$ odgovara uniformnoj distribuciji preko C klasa pa vrijedi $q_i = \frac{1}{C}$:

$$\begin{aligned} \mathcal{L}_{\text{neoznačeno}} &= - \sum_{i=1}^C \left(\frac{1}{C} \log \left(\frac{1}{C} \right) \right) + \sum_{i=1}^C \left(\frac{1}{C} \log(p_i) \right) \\ &= \log(C) + \frac{1}{C} \sum_{i=1}^C \log(p_i) \\ &= \log(C) + \frac{1}{C} \sum_{i=1}^C \log(\text{softmax}(\mathbf{s})_i) \end{aligned}$$

Rješenje (c):

Pretpostavimo da je za sve neoznačene primjere oznaka y postavljena na -1 . Tada gubitak možemo zapisati po slučajevima:

$$\mathcal{L}(\mathbf{s}, y) = \begin{cases} -\log(\text{softmax}(\mathbf{s})_y), & \text{ako je } y \in \{0, 1, \dots, C-1\} \\ \log(C) + \frac{1}{C} \sum_{i=1}^C \log(\text{softmax}(\mathbf{s})_i), & \text{ako je } y = -1 \end{cases}$$

Za označene primjere, vrijedi:

$$\begin{aligned} \mathcal{L}(\mathbf{s}, y) &= -\log(\text{softmax}(\mathbf{s})_y) \\ &= -\log \left(\frac{\exp(s_y)}{\sum_{j=1}^C \exp(s_j)} \right) \\ &= -s_y + \log \left(\sum_{j=1}^C \exp(s_j) \right) \end{aligned}$$

Razmotrimo gradijent po k -toj komponenti vektora logita $\mathbf{s}$:

$$\begin{aligned} \frac{\partial \mathcal{L}(\mathbf{s}, y)}{\partial s_k} &= \frac{\exp(s_k)}{\sum_{j=1}^C \exp(s_j)} - \frac{\partial s_y}{\partial s_k} \\ &= p_k - \mathbb{I}[y = k] \end{aligned}$$

Gradijent po čitavom vektoru logita $\mathbf{s}$ je:

$$\frac{\partial \mathcal{L}(\mathbf{s}, y)}{\partial \mathbf{s}} = \mathbf{p} - \mathbf{y}_{\text{oh}}$$

Za neoznačene primjere ($y = -1$), gubitak iznosi:

$$\begin{aligned} \mathcal{L}(\mathbf{s}, -1) &= \log(C) + \frac{1}{C} \sum_{i=1}^C \log(\text{softmax}(\mathbf{s})_i) \\ &= \log(C) + \frac{1}{C} \sum_{i=1}^C \log\left(\frac{\exp(s_i)}{\sum_{j=1}^C \exp(s_j)}\right) \\ &= \log(C) + \frac{1}{C} \sum_{i=1}^C \left(s_i - \log\left(\sum_{j=1}^C \exp(s_j)\right)\right) \\ &= \log(C) + \frac{1}{C} \left(\sum_{i=1}^C s_i - \sum_{i=1}^C \log\left(\sum_{j=1}^C \exp(s_j)\right)\right) \\ &= \log(C) + \frac{1}{C} \sum_{i=1}^C s_i - \log\left(\sum_{j=1}^C \exp(s_j)\right) \end{aligned}$$

Razmotrimo gradijent po k -toj komponenti vektora logita $\mathbf{s}$:

$$\begin{aligned} \frac{\partial \mathcal{L}(\mathbf{s}, -1)}{\partial s_k} &= \frac{1}{C} - \frac{\exp(s_k)}{\sum_{j=1}^C \exp(s_j)} \\ &= \frac{1}{C} - p_k \end{aligned}$$

Konačno, kada spojimo oba slučaja, ukupni gradijent gubitka po cijelom vektoru logita $\mathbf{s}$ možemo zapisati kao:

$$\frac{\partial \mathcal{L}(\mathbf{s}, y)}{\partial \mathbf{s}} = \begin{cases} \mathbf{p} - \mathbf{y}_{\text{oh}}, & \text{ako je } y \in \{0, 1, \dots, C-1\} \\ \frac{1}{C} \mathbf{1} - \mathbf{p}, & \text{ako je } y = -1 \end{cases}$$

Zadatak 5

Razmatramo razvoj modela za klasifikaciju 3-bitnih RGB slika dimenzije 5×5 piksela. Ovaj slikovni format raspoznaje ukupno 8 boja, a vrijednost svakog piksela predstavljena je trobitnim binarnim brojem prema sljedećoj tablici:

Boja	Crna	Plava	Zelena	Tirkizna	Crvena	Roza	Žuta	Bijela
Binarni	000	001	010	011	100	101	110	111
Dekadski	0	1	2	3	4	5	6	7

Za rješavanje problema konstruiramo model s dva sigmoidalno aktivirana izlaza: y_1 i y_2 . Izlaz y_1 govori je li broj crvenih piksela u ulaznoj slici strogo veći od broja bijelih piksela, dok izlaz y_2 govori je li broj crnih piksela u ulaznoj slici strogo veći od broja crvenih piksela. U oba slučaja klasifikacijski prag iznosi 0.5 (vidi metodu `forward`). U nastavku prikazujemo skicu izvornog koda modela.

```
class ColorCounter(nn.Module):
```

```

def __init__(self):
    super(ColorCounter, self).__init__()
    self.conv = nn.Conv2d(
        in_channels=3, out_channels=___,
        kernel_size=3, stride=1, padding=1, bias=True
    )
    self.relu = nn.ReLU()
    self.flatten = nn.Flatten()
    self.fc = nn.Linear(
        in_features=___, out_features=2, bias=True
    )
    self.sigmoid = nn.Sigmoid()

def forward(self, x):
    s = self.sigmoid(
        self.fc(
            self.flatten(
                self.relu(self.conv(x))
            )
        )
    )
    y = s >= 0.5 # klasifikacijski prag
    return y

```

Zadaci:

- Popunite sve vrijednosti koje nedostaju u definiciji modela i odredite vrijednosti parametara tako da model obavlja traženi zadatak. Vaše rješenje ne smije imati više od 240 parametara.
- Koliko parametara ima konstruirani model?
- Demonstrirajte da vaš model obavlja traženi zadatak unaprijednim prolazom na primjeru ulazne slike (indekse boja treba prevesti u 3-bitne piksele).

$$\begin{bmatrix} 4 & 4 & 4 & 0 & 6 \\ 4 & 5 & 5 & 0 & 6 \\ 4 & 1 & 1 & 0 & 6 \\ 0 & 0 & 0 & 0 & 6 \\ 7 & 7 & 7 & 7 & 7 \end{bmatrix}$$

Rješenje (a):

Čitajući programski kod, možemo vidjeti da su nepoznate vrijednosti broj izlaznih kanala konvolucijskog sloja (`out_channels`) te broj ulaznih značajki potpuno povezanog sloja (`in_features`).

Hajdemo vidjeti koja će biti uloga konvolucijskog sloja u ovome zadatku. Nažalost, nema nekog univerzalnog “recepta” na temelju kojeg se uvijek dođe do tog zaključka, no često možemo krenuti od osnovne ideje da u operaciji konvolucije želimo da jedna konvolucijska jezgra bude specijalizirana za ekstrakciju jedne značajke. Ako to znamo, onda bi nas to moglo navesti na ideju da će konvolucijske jezgre u ovoj operaciji konvolucije brojati koliko ima piksela određene boje, pri čemu će svaka jezgra biti zadužena za točno jednu boju.

Nadalje, iz teksta zadatka se može zaključiti da nećemo trebati konvolucijsku jezgru za svaku

od boja, već nas interesiraju samo **bijela**, **crna** i **crvena**. Prema tome, broj izlaznih kanala ove konvolucije je **3**.

Svaka boja je prikazana 3-bitnim binarnim kodom, gdje je svaki bit zapisan u odgovarajućem kanalu 3-kanalne ulazne slike. Kako bi to bilo jasnije i kako bismo stekli još bolju intuiciju za konstrukciju konvolucijskih jezgri, raspišimo po kanalima primjer ulazne slike dan u c) dijelu zadatka. Vrijedi:

$$\mathbf{X} = \begin{bmatrix} 4 & 4 & 4 & 0 & 6 \\ 4 & 5 & 5 & 0 & 6 \\ 4 & 1 & 1 & 0 & 6 \\ 0 & 0 & 0 & 0 & 6 \\ 7 & 7 & 7 & 7 & 7 \end{bmatrix}$$

Sada dekadsku vrijednost svakog piksela zamijenimo vektorom odgovarajućeg 3-bitnog koda koji ide kroz semantičku dimenziju:

$$\mathbf{X}_{\text{bin}} = \begin{bmatrix} [1, 0, 0] & [1, 0, 0] & [1, 0, 0] & [0, 0, 0] & [1, 1, 0] \\ [1, 0, 0] & [1, 0, 1] & [1, 0, 1] & [0, 0, 0] & [1, 1, 0] \\ [1, 0, 0] & [0, 0, 1] & [0, 0, 1] & [0, 0, 0] & [1, 1, 0] \\ [0, 0, 0] & [0, 0, 0] & [0, 0, 0] & [0, 0, 0] & [1, 1, 0] \\ [1, 1, 1] & [1, 1, 1] & [1, 1, 1] & [1, 1, 1] & [1, 1, 1] \end{bmatrix}$$

Dodatno, možemo ovo raspisati kroz 3 matrice po pojedinim kanalima:

$$\mathbf{X}^{(1)} = \begin{bmatrix} 1 & 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}, \quad \mathbf{X}^{(2)} = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}, \quad \mathbf{X}^{(3)} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Primijetimo sada sljedeće. Budući da želimo prepoznati boju na razini piksela, dakle na prostornoj dimenziji 1×1 , bilo bi dovoljno da konvolucijska jezgra ima prostornu dimenzionalnost 1×1 . No, u danom programskom kodu već imamo zadano da je prostorna dimenzionalnost konvolucijske jezgre 3×3 . Je li to problem? Pa i nije, jednostavno ćemo “natjerati” jezgru 3×3 da se efektivno ponaša kao jezgra 1×1 na način da joj središnji element postavimo na vrijednost koju bismo postavili i jezgri 1×1 , a u sve druge elemente stavimo vrijednost 0.

1. Konvolucijska jezgra za crnu boju

Idemo sada vidjeti kako konstruirati konvolucijsku jezgru koja će prepoznati crnu boju. Budući da je 3-bitni kod crne boje 000, to znači da se mora “kažnjavati” pojava bita 1 u bilo kojem od tri kanala. Postavljamo središnje elemente sva tri kanala jezgre $\mathbf{W}_{\text{crna}}$ na -1 pa imamo:

$$\mathbf{W}_{\text{crna}}^{(1)} = \mathbf{W}_{\text{crna}}^{(2)} = \mathbf{W}_{\text{crna}}^{(3)} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Sada moramo razmotriti koliki će biti pomak b_{crna} za ovu jezgru. Uzimamo u obzir da ovaj sloj slijedi aktivacija ReLU za koju želimo da bude točno 1 samo na crnim pikselima, a 0 na svim ostalim. Prema tome, ako za pomak odaberemo $b_{\text{crna}} = 1$, vrijednosti koje jezgra proizvede za svaku od boja su sljedeće:

Boja (kod)	Crna (000)	Plava (001)	Zelena (010)	Tirkizna (011)	Crvena (100)	Roza (101)	Žuta (110)	Bijela (111)
Izlaz konvolucije	0	-1	-1	-2	-1	-2	-2	-3
S pomakom ($b = 1$)	1	0	0	-1	0	-1	-1	-2
Nakon ReLU	1	0	0	0	0	0	0	0

Sada je jasno da će nakon prolaska kroz ReLU svi pikseli osim crnih biti 0.

2. Konvolucijska jezgra za bijelu boju

Idemo sada vidjeti kako konstruirati konvolucijsku jezgru koja će prepoznati bijelu boju. Budući da je 3-bitni kod bijele boje 111, to znači da moramo “nagrađivati” pojavu bita 1 u svim kanalima. Postavljamo središnje elemente sva tri kanala jezgre $\mathbf{W}_{\text{bijela}}$ na 1 pa imamo:

$$\mathbf{W}_{\text{bijela}}^{(1)} = \mathbf{W}_{\text{bijela}}^{(2)} = \mathbf{W}_{\text{bijela}}^{(3)} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Sada moramo razmotriti koliki će biti pomak b_{bijela} za ovu jezgru. Uzimamo u obzir da ovaj sloj slijedi aktivacija ReLU za koju želimo da bude točno 1 samo na bijelim pikselima, a 0 na svim ostlima. Prema tome, ako za pomak odaberemo $b_{\text{bijela}} = -2$, vrijednosti koje jezgra proizvede za svaku od boja su sljedeće:

Boja (kod)	Crna (000)	Plava (001)	Zelena (010)	Tirkizna (011)	Crvena (100)	Roza (101)	Žuta (110)	Bijela (111)
Izlaz konvolucije	0	1	1	2	1	2	2	3
S pomakom ($b = -2$)	-2	-1	-1	0	-1	0	0	1
Nakon ReLU	0	0	0	0	0	0	0	1

Sada je jasno da će nakon prolaska kroz ReLU svi pikseli osim bijelih biti 0.

3. Konvolucijska jezgra za crvenu boju

Idemo sada vidjeti kako konstruirati konvolucijsku jezgru koja će prepoznati crvenu boju. Budući da je 3-bitni kod crvene boje 100, to znači da moramo “nagrađivati” pojavu bita 1 u prvom kanalu te “kažnjavati” pojavu bita 1 u drugom ili trećem kanalu. Postavljamo središnji element prvog kanal jezgre $\mathbf{W}_{\text{crvena}}$ na -1, a središnji element druga dva kanala na 1 pa imamo:

$$\mathbf{W}_{\text{crvena}}^{(1)} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{W}_{\text{crvena}}^{(2)} = \mathbf{W}_{\text{crvena}}^{(3)} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Sada moramo razmotriti koliki će biti pomak b_{crvena} za ovu jezgru. Uzimamo u obzir da ovaj sloj slijedi aktivacija ReLU za koju želimo da bude točno 1 samo na crvenim pikselima, a 0 na ostalima. Prema tome, ako za pomak odaberemo $b_{\text{crvena}} = 0$, vrijednosti koje jezgra proizvede za svaku od boja su sljedeće:

Boja (kod)	Crna (000)	Plava (001)	Zelena (010)	Tirkizna (011)	Crvena (100)	Roza (101)	Žuta (110)	Bijela (111)
Izlaz konvolucije	0	-1	-1	-2	1	0	0	-1
S pomakom ($b = 0$)	0	-1	-1	-2	1	0	0	-1
Nakon ReLU	0	0	0	0	1	0	0	0

Sada je jasno da će nakon prolaska kroz ReLU svi pikseli osim crvenih biti 0.

Potpuno povezani sloj

Budući da smo u konvolucijskom sloju definirali 3 jezgre, da je ulazni podatak slika 5×5 te da se koristi padding, izlazni tenzor konvolucije, a i ReLU aktivacije je tenzor dimenzija $3 \times 5 \times 5$. Prolazak kroz **Flatten** ga vektorizira u vektor $\mathbf{x}$ dimenzije $3 \times 25 = 75$. Prema tome, broj ulaznih značajki (**in_features**) potpuno povezanog sloja je **75**.

U tom vektoru $\mathbf{x}$, prvih 25 elemenata predstavlja elemente prvog kanala koji detektira crne piksele, idućih 25 predstavlja elemente drugog kanala koji detektira bijele piksele, a zadnjih 25 predstavlja elemente trećeg kanala koji detektira crvene piksele.

Budući da je zadani broj izlaznih kanala potpuno povezanog sloja 2, to znači da je pripadna matrica težina $\mathbf{W}$ dimenzija 2×75 . Prvi redak te matrice ćemo zadužiti da izračuna razliku između broja crvenih i bijelih piksela, a drugi redak ćemo zadužiti da izračuna razliku između broja crnih i crvenih piksela.

Prema tome, matrica $\mathbf{W}$ mora biti zadana ovako:

$$\mathbf{W} = \begin{bmatrix} \underbrace{0, \dots, 0}_{25} & \underbrace{-1, \dots, -1}_{25} & \underbrace{1, \dots, 1}_{25} \\ \underbrace{1, \dots, 1}_{25} & \underbrace{0, \dots, 0}_{25} & \underbrace{-1, \dots, -1}_{25} \end{bmatrix}$$

Još preostaje odrediti vektor pomaka $\mathbf{b}$ za ovaj sloj. Imajmo na umu da vrijedi $\sigma(0) = 0.5$ te pogledajmo sljedeće.

Neka je $\mathbf{x}$ ulazni vektor potpuno povezanog sloja, $\mathbf{w}_1$ prvi redak matrice $\mathbf{W}$, $\mathbf{w}_2$ drugi redak matrice $\mathbf{W}$, b_1 prvi element vektora pomaka $\mathbf{b}$ te b_2 drugi element vektora pomaka $\mathbf{b}$. Tada vrijedi:

- $\mathbf{w}_1 \mathbf{x} > 0$ ako je broj crvenih piksela strogo veći od broja bijelih piksela,
- $\mathbf{w}_1 \mathbf{x} < 0$ ako je broj crvenih piksela strogo manji od broja bijelih piksela,
- $\mathbf{w}_1 \mathbf{x} = 0$ ako je broj crvenih piksela jednak broju bijelih piksela.

Budući da je u zadatku zadan klasifikacijski prag od 0.5 za koji želimo da se prijede samo u slučaju strogo većeg broja crvenih piksela, onda moramo spriječiti da sigmoida vrati vrijednost 0.5 u slučaju jednakosti broja piksela. To lako postizemo postavljanjem pomaka $b_1 = -1$ jer tad vrijedi:

- $\mathbf{w}_1 \mathbf{x} + b_1 \geq 0$ ako je broj crvenih strogo veći od broja bijelih piksela,
- $\mathbf{w}_1 \mathbf{x} + b_1 < 0$ ako je broj crvenih manji ili jednak broju bijelih piksela.

Nadalje:

- $\mathbf{w}_2 \mathbf{x} > 0$ ako je broj crnih piksela strogo veći od broja crvenih piksela,
- $\mathbf{w}_2 \mathbf{x} < 0$ ako je broj crnih piksela strogo manji od broja crvenih piksela,
- $\mathbf{w}_2 \mathbf{x} = 0$ ako je broj crnih piksela jednak broju crvenih piksela.

I ovdje moramo spriječiti da sigmoida vrati vrijednost 0.5 u slučaju jednakosti broja piksela, pa postavljamo $b_2 = -1$ jer tad vrijedi:

- $\mathbf{w}_2 \mathbf{x} + b_2 \geq 0$ ako je broj crnih strogo veći od broja crvenih piksela,
- $\mathbf{w}_2 \mathbf{x} + b_2 < 0$ ako je broj crnih manji ili jednak broju crvenih piksela.

Dakle,

$$\mathbf{b} = \begin{bmatrix} -1 & -1 \end{bmatrix}^\top$$

Rješenje (b):

- **Konvolucijski sloj** (`in_channels=3`, `out_channels=3`, `kernel_size=3`, `bias=True`):

$$\#params_{conv} = (3 \cdot 3 \cdot 3 \cdot 3) + 3 = 84$$

- **Potpuno povezani sloj** (`in_features=75`, `out_features=2`, `bias=True`):

$$\#params_{fc} = (2 \cdot 75) + 2 = 152$$

Ukupan broj parametara u modelu iznosi:

$$\#params_{ukupno} = 84 + 152 = 236 < \mathbf{240}$$

Rješenje (c):

Zadana dekadaska matrica ulazne slike je:

$$\mathbf{X}_{dec} = \begin{bmatrix} 4 & 4 & 4 & 0 & 6 \\ 4 & 5 & 5 & 0 & 6 \\ 4 & 1 & 1 & 0 & 6 \\ 0 & 0 & 0 & 0 & 6 \\ 7 & 7 & 7 & 7 & 7 \end{bmatrix}$$

Kao što smo raspisali u podzadatku (a), nakon pretvorbe u 3-bitni kod i razdvajanja po kanalima dobivamo matrice $\mathbf{X}^{(1)}$, $\mathbf{X}^{(2)}$, $\mathbf{X}^{(3)}$:

$$\mathbf{X}^{(1)} = \begin{bmatrix} 1 & 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}, \quad \mathbf{X}^{(2)} = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}, \quad \mathbf{X}^{(3)} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Za konvolucijski sloj zadan je *padding* od 1. To znači da se oko matrica dodaje rub od nula, pa one postaju dimenzija 7×7 , označimo ih s $\mathbf{X}_{pad}^{(k)}$. Međutim, budući da naše 3×3 jezgre imaju vrijednosti različite od nule isključivo u središnjem elementu, operacija unakrsne korelacije s *paddingom* svodi se na množenje svake 5×5 matrice ulaza s tim središnjim elementom jezgre.

Djelovanje konvolucijske jezgre $\mathbf{W}_{\text{crna}}$

Središnji elementi sva tri kanala jezgre su -1 , a pomak je $b_{\text{crna}} = 1$.

$$\begin{aligned}\mathbf{g}^{(1)} &= \mathbf{W}_{\text{crna}}^{(1)} \star \mathbf{X}_{\text{pad}}^{(1)} + \mathbf{W}_{\text{crna}}^{(2)} \star \mathbf{X}_{\text{pad}}^{(2)} + \mathbf{W}_{\text{crna}}^{(3)} \star \mathbf{X}_{\text{pad}}^{(3)} + b_{\text{crna}} \\ &= (-1)\mathbf{X}^{(1)} + (-1)\mathbf{X}^{(2)} + (-1)\mathbf{X}^{(3)} + 1 \\ &= \begin{bmatrix} 0 & 0 & 0 & 1 & -1 \\ 0 & -1 & -1 & 1 & -1 \\ 0 & 0 & 0 & 1 & -1 \\ 1 & 1 & 1 & 1 & -1 \\ -2 & -2 & -2 & -2 & -2 \end{bmatrix}\end{aligned}$$

Djelovanje konvolucijske jezgre $\mathbf{W}_{\text{bijela}}$

Središnji elementi sva tri kanala jezgre su 1 , a pomak je $b_{\text{bijela}} = -2$.

$$\begin{aligned}\mathbf{g}^{(2)} &= \mathbf{W}_{\text{bijela}}^{(1)} \star \mathbf{X}_{\text{pad}}^{(1)} + \mathbf{W}_{\text{bijela}}^{(2)} \star \mathbf{X}_{\text{pad}}^{(2)} + \mathbf{W}_{\text{bijela}}^{(3)} \star \mathbf{X}_{\text{pad}}^{(3)} + b_{\text{bijela}} \\ &= (1)\mathbf{X}^{(1)} + (1)\mathbf{X}^{(2)} + (1)\mathbf{X}^{(3)} - 2 \\ &= \begin{bmatrix} -1 & -1 & -1 & -2 & 0 \\ -1 & 0 & 0 & -2 & 0 \\ -1 & -1 & -1 & -2 & 0 \\ -2 & -2 & -2 & -2 & 0 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}\end{aligned}$$

Djelovanje konvolucijske jezgre $\mathbf{W}_{\text{crvena}}$

Središnji element prvog kanala je 1 , drugog i trećeg je -1 , a pomak je $b_{\text{crvena}} = 0$.

$$\begin{aligned}\mathbf{g}^{(3)} &= \mathbf{W}_{\text{crvena}}^{(1)} \star \mathbf{X}_{\text{pad}}^{(1)} + \mathbf{W}_{\text{crvena}}^{(2)} \star \mathbf{X}_{\text{pad}}^{(2)} + \mathbf{W}_{\text{crvena}}^{(3)} \star \mathbf{X}_{\text{pad}}^{(3)} + b_{\text{crvena}} \\ &= (1)\mathbf{X}^{(1)} + (-1)\mathbf{X}^{(2)} + (-1)\mathbf{X}^{(3)} + 0 \\ &= \begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 1 & -1 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ -1 & -1 & -1 & -1 & -1 \end{bmatrix}\end{aligned}$$

Aktivacija ReLU i vektorizacija

Nakon prolaska matrica kroz funkciju $\text{ReLU}(z)$ dobivamo izlaze $\mathbf{h}^{(k)}$:

$$\mathbf{h}^{(1)} = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{h}^{(2)} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}, \quad \mathbf{h}^{(3)} = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Prolazak kroz sloj **Flatten** radi ulančavanje elemenata ovih matrica u jedan vektor $\mathbf{x}$ dimenzije 75. Iz matrica lako možemo prebrojati sumu elemenata za svaki pripadajući kanal vektora $\mathbf{x}$:

- $\#crni = (\sum \mathbf{h}^{(1)}) = 7$
- $\#bijeli = (\sum \mathbf{h}^{(2)}) = 5$
- $\#crveni = (\sum \mathbf{h}^{(3)}) = 5$

Potpuno povezani sloj

Vektor $\mathbf{z}$ računamo kao $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$. Uzimajući težine matrice $\mathbf{W}$ koje zbrajaju i oduzimaju iznose iz pojedinih kanala pomak $\mathbf{b} = [-1, -1]^\top$, dobivamo:

$$\begin{aligned} z_1 &= (\#crveni - \#bijeli) + b_1 = (5 - 5) - 1 = -1 \\ z_2 &= (\#crni - \#crveni) + b_2 = (7 - 5) - 1 = 1 \end{aligned}$$

Nakon prolaska kroz sigmoidu dobivamo predikcije $\mathbf{s}$:

$$\begin{aligned} s_1 &= \sigma(-1) = \frac{1}{1 + e^1} \approx 0.2689 \\ s_2 &= \sigma(1) = \frac{1}{1 + e^{-1}} \approx 0.7311 \end{aligned}$$

Konačno, primjenom klasifikacijskog praga od 0.5 izlazni vektor modela iznosi:

$$\mathbf{y} = \mathbf{s} \geq 0.5 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Model ispravno zaključuje da broj crvenih piksela (5) nije strogo veći od bijelih (5), ali da broj crnih piksela (7) jest strogo veći od crvenih (5).